

Visual Recognition using Deep Learning

Object Detection

林彥宇 教授

Yen-Yu Lin, Professor

國立陽明交通大學 資訊工程學系

Computer Science, National Yang Ming Chiao Tung University

Some slides are modified from Fei-Fei Li, Justin Johnson, Andrej Karpathy, Weijie Su, and Ross Girshick



Outline

- Object detection
 - R-CNN
 - Fast R-CNN
 - Faster R-CNN
 - YOLO
 - FCOS
 - DETR
 - Deformable DETR



Outline

- Object detection
 - R-CNN
 - Fast R-CNN
 - Faster R-CNN
 - YOLO
 - FCOS
 - DETR
 - Deformable DETR

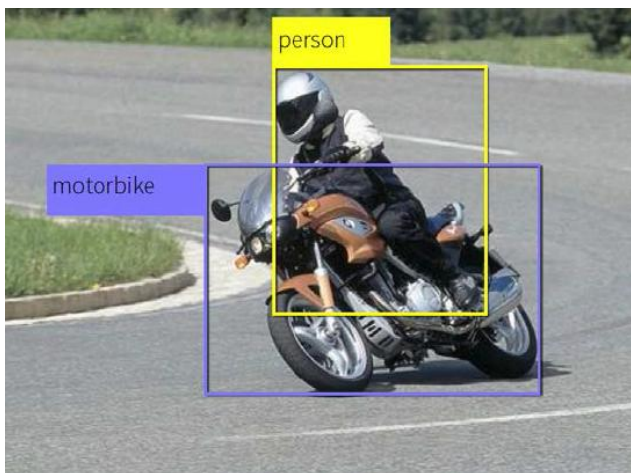


Detection and segmentation

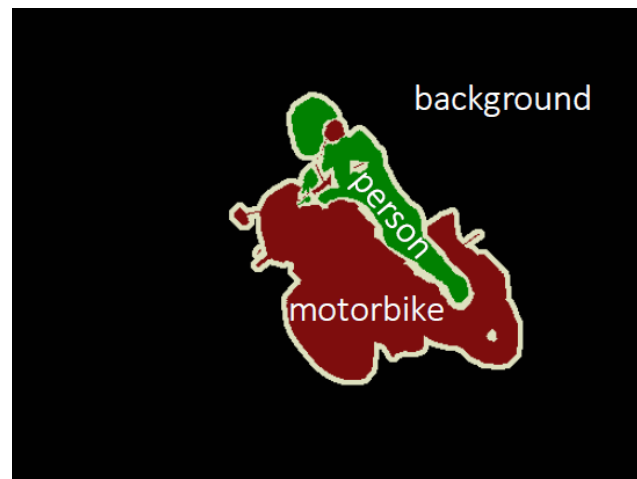
input image



object detection

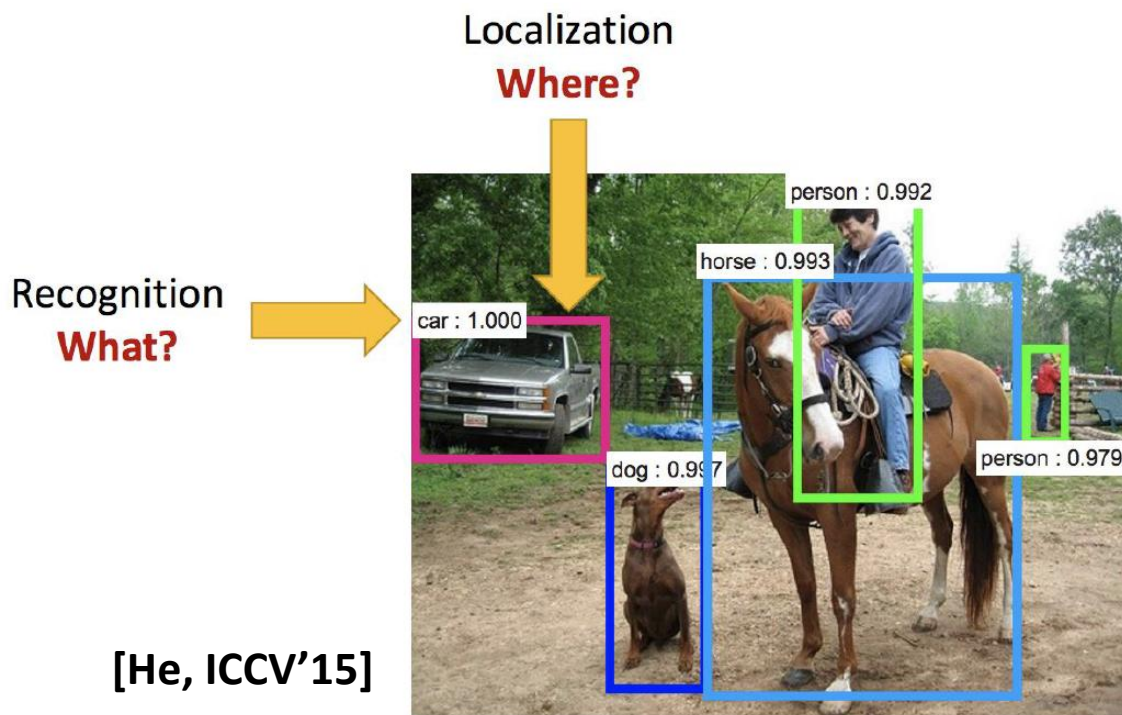


semantic segmentation



Object detection

- Goal: Detecting instances of semantic objects of certain classes
- Critical to high-level vision tasks such as surveillance, self-driving car, and scene understanding



Pascal VOC dataset

- PASCAL Visual Object Classes Challenge
- 20 classes, ~10K images, ~25K annotated objects



- Training, validation, and test data sets



Difficulties



deformation



occlusion



fine-grained categories

orientation and size



雞



鴨



鵞



鶴



鸛

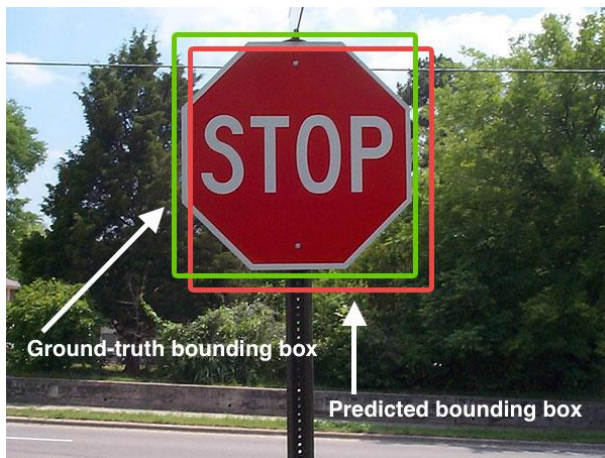


鴿



Detection performance metric: IoU

- Intersection over union (IoU)

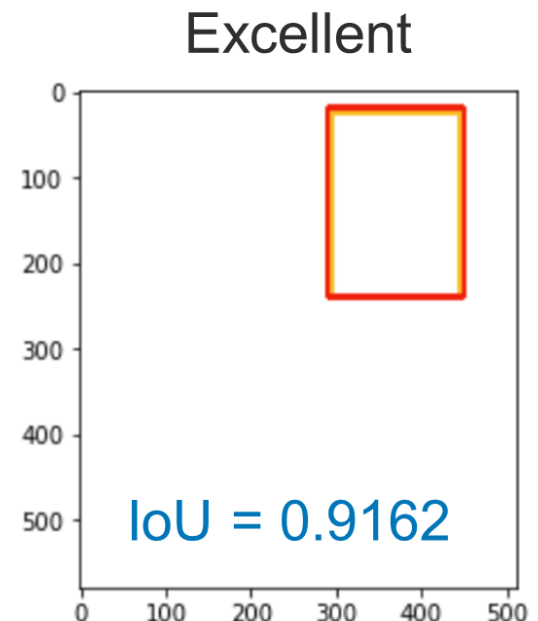
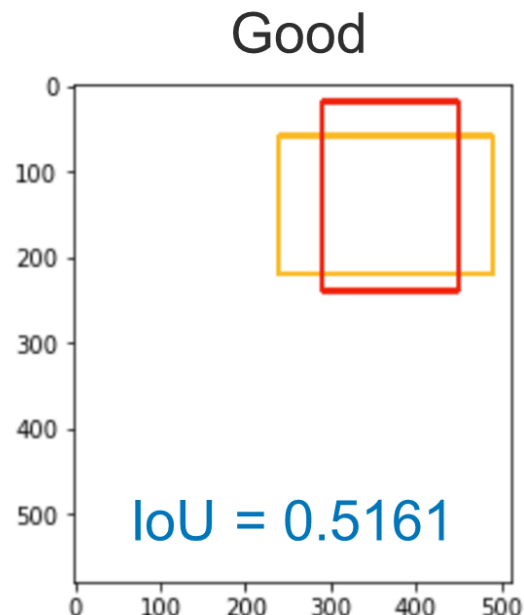
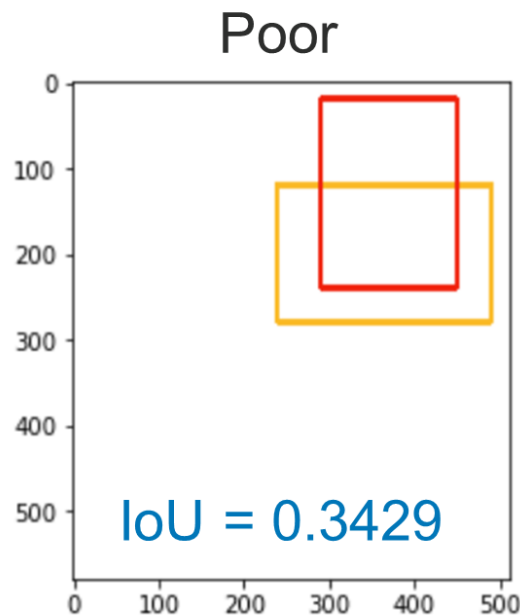


$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$


- IoU with a threshold, e.g., 0.5, to determine if an object is correctly detected
- Average Precision (AP): the average precision over thresholds
- mean AP (mAP): the mean of APs over classes

Detection performance metric: IoU

- In object detection tasks, an IoU greater than 0.5 is considered good



Detection performance metric: Precision and Recall

- Confusion matrix

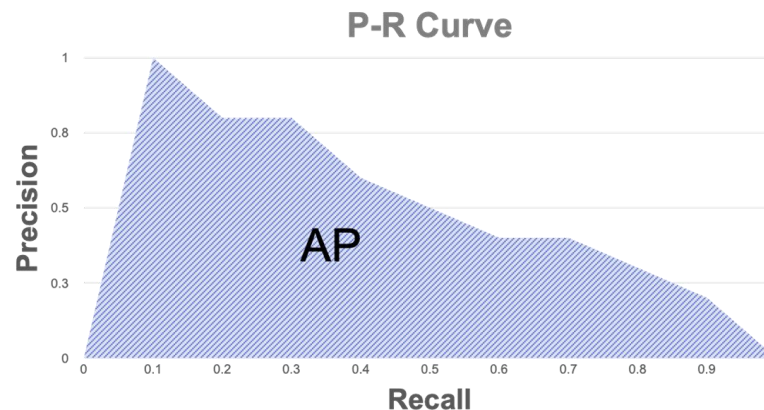
	Ground truth: YES	Ground truth: NO
Prediction: YES	TP (True Positive)	FP (False Positive)
Prediction: NO	FN (False Negative)	TN (True Negative)

- Precision**: Of all the items the model predicted as positive, how many were actually positive?
 - Precision = $\text{True Positives} / (\text{True Positives} + \text{False Positives})$
- Recall**: Of all the actual positive items, how many did the model correctly detect
 - Recall = $\text{True Positives} / (\text{True Positives} + \text{False Negatives})$



Detection performance metric: Average Precision

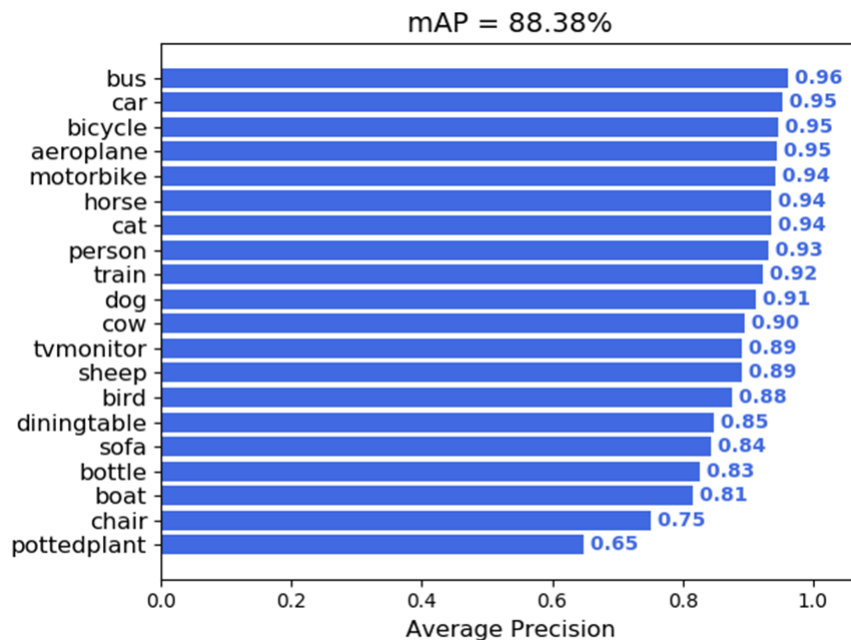
- Precision-recall curve:
 - Object detection models produce bounding boxes and confidence scores for some particular object class
 - By varying the confidence threshold, we get different precision and recall values



- Average precision (AP): It calculates the average precision across different recall values
 - It represents the area under the precision-recall curve

Detection performance metric: Mean Average Precision

- Average Precision (AP) for each single class
- **Mean Average Precision (mAP):** Once AP is calculated for each class, mAP is simply the average of these AP values.



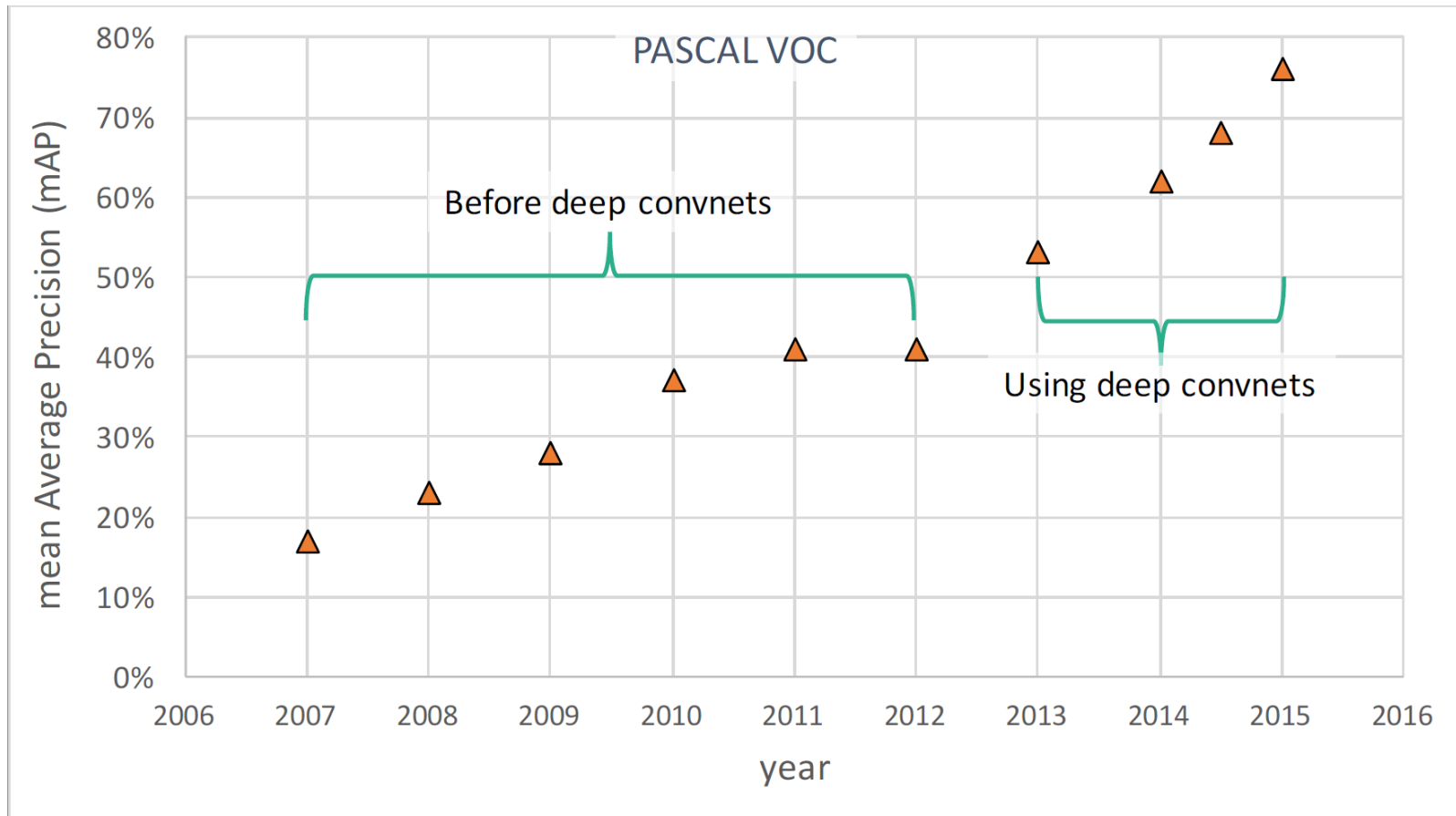
Outline

- Object detection
 - R-CNN
 - Fast R-CNN
 - Faster R-CNN
 - YOLO
 - FCOS
 - DETR
 - Deformable DETR



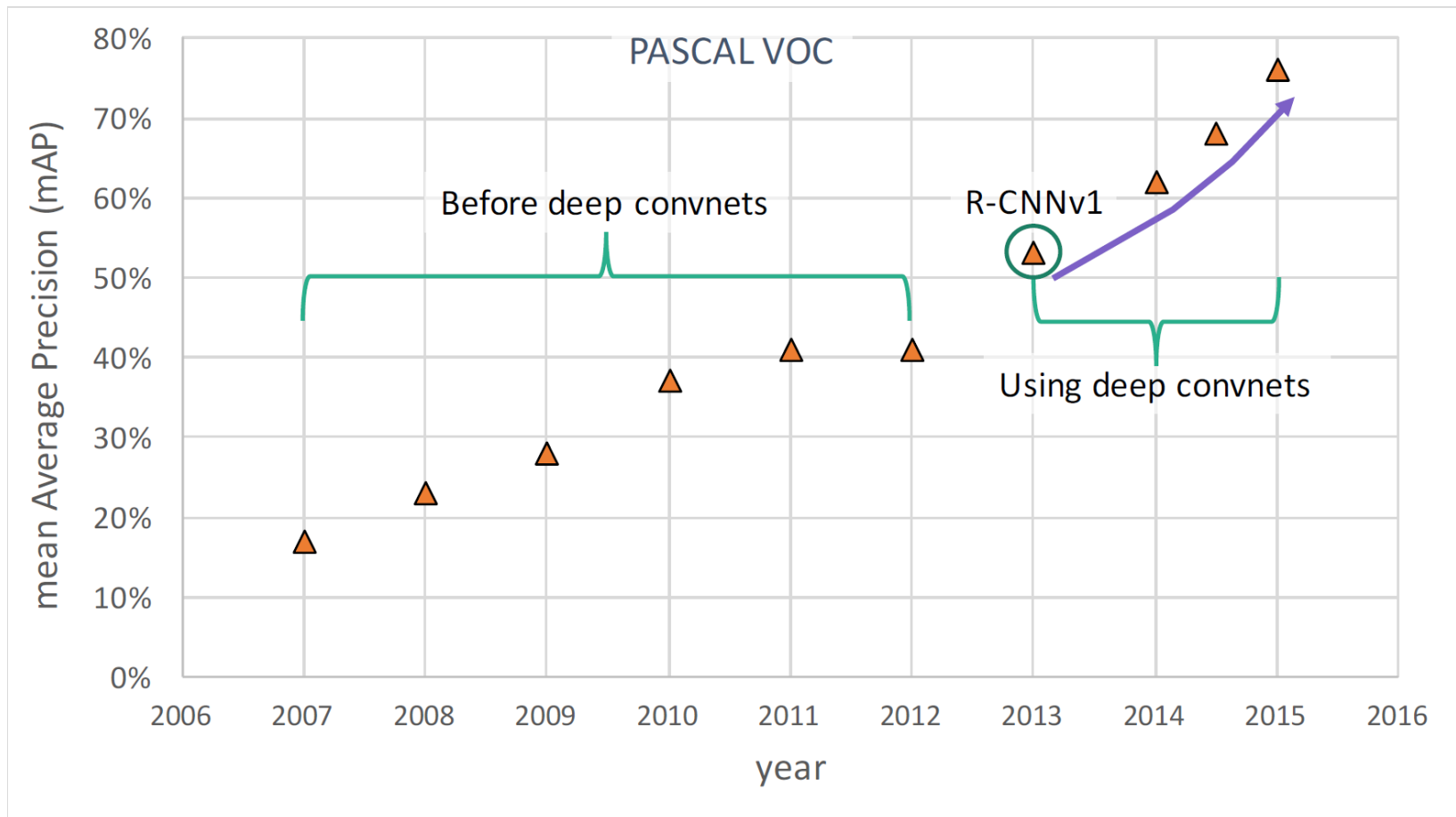
Deep learning for object detection

- Why deep learning?



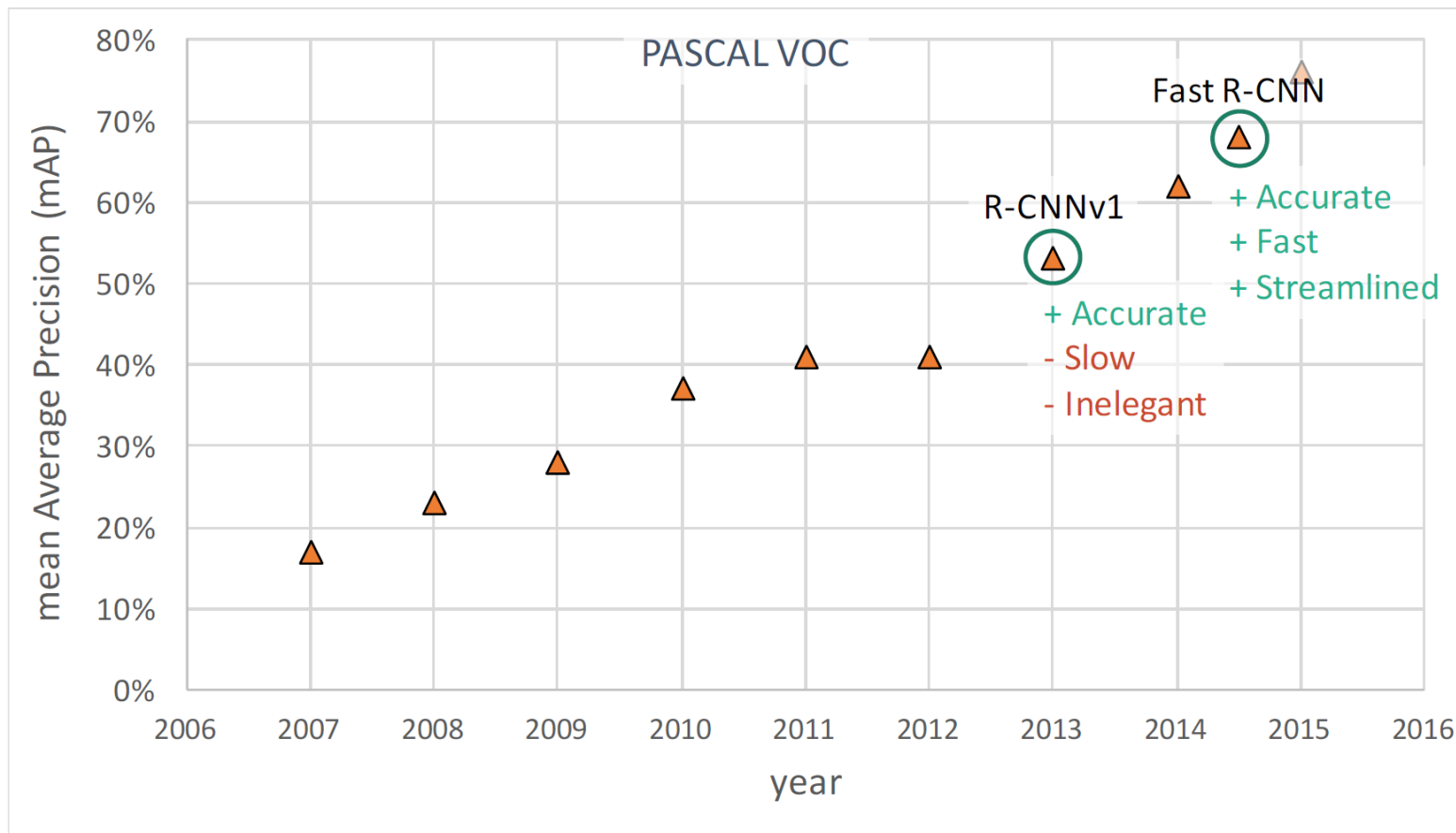
Deep learning for object detection

- Why deep learning



Deep learning for object detection

- Why deep learning



Formulation of deep learning for object detection

- Detection = Classification + Localization

- Classification: C classes

- Input: Image
- Output: Class label
- Evaluation metric: Accuracy



→ CAT

- Localization:

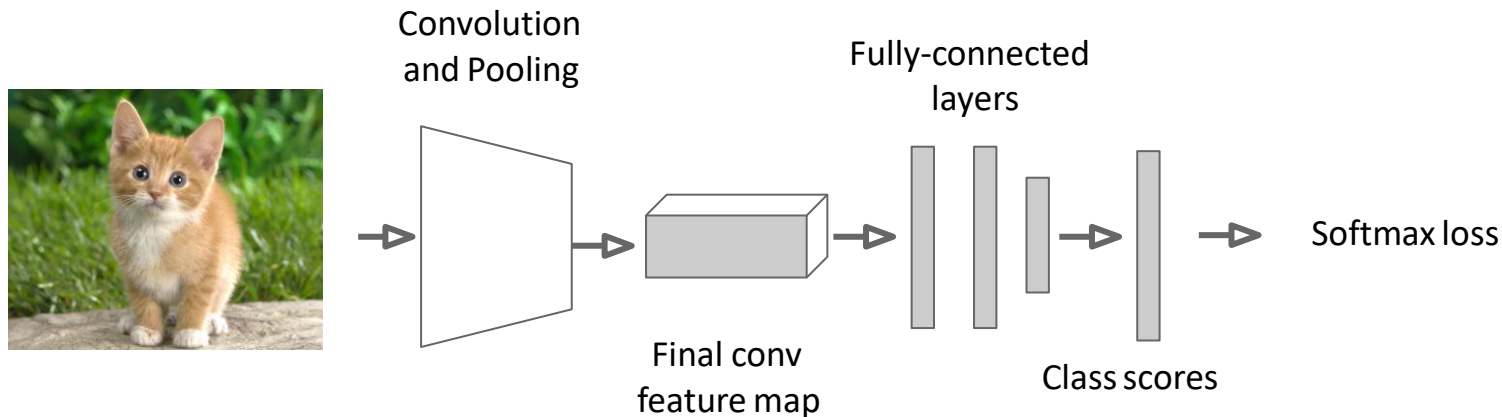
- Input: Image
- Output: Box in the image (x, y, w, h)
- Evaluation metric: IoU



→ (x, y, w, h)

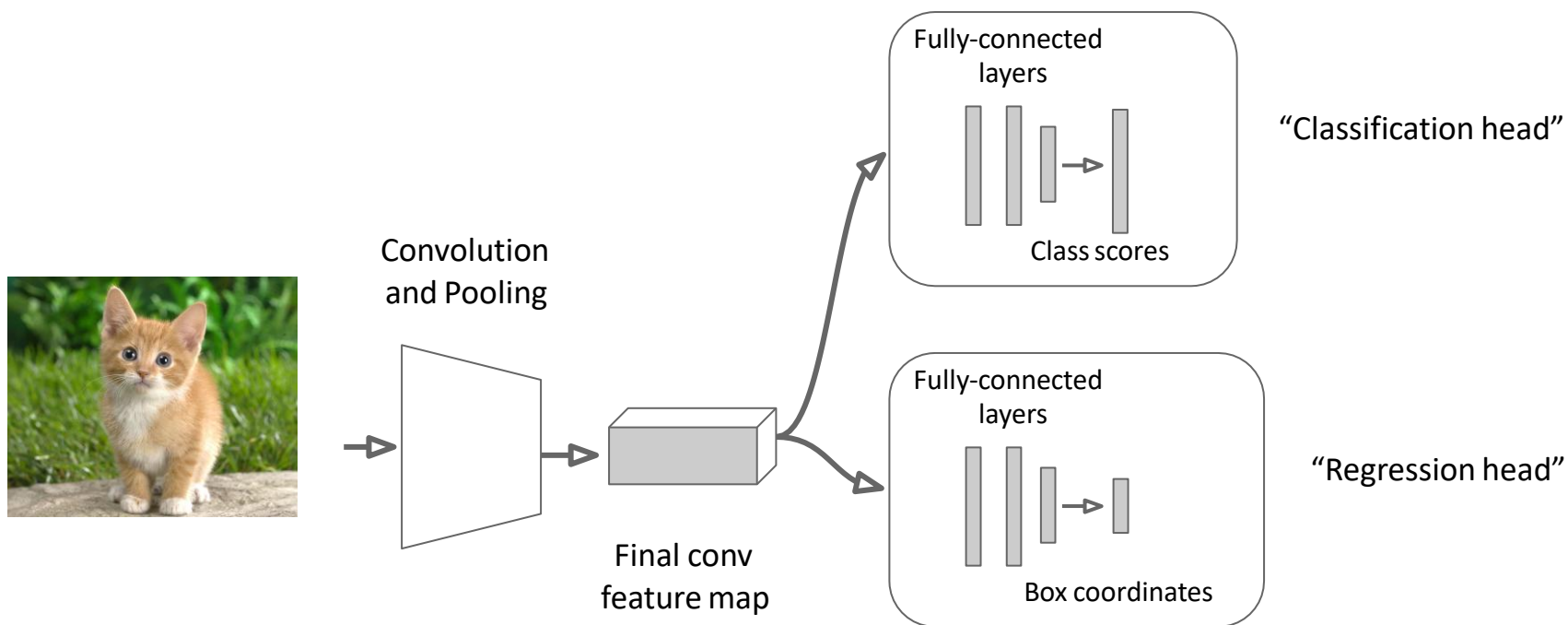
Formulation #1: Detection as a classification + regression problem

- Suppose there is an object in an image
- Train a CNN head for **object class classification**



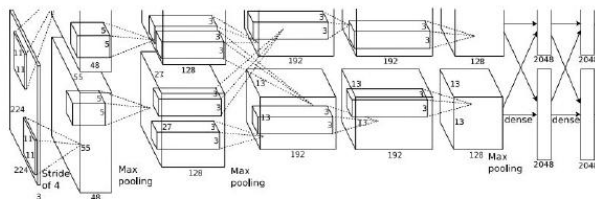
Formulation #1: Detection as a classification + regression problem

- Suppose there is an object in an image
- Train a CNN head for **object class classification**
- Train another CNN head for **bounding box regression**

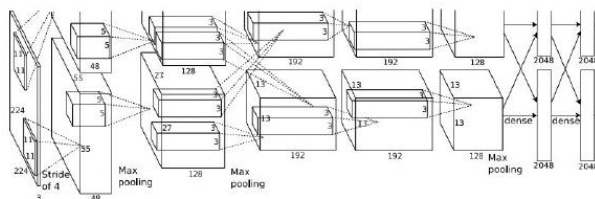


Problem of Formulation #1

- Formulation #1 works for images with a single object
- In practice, object number varies from image to image



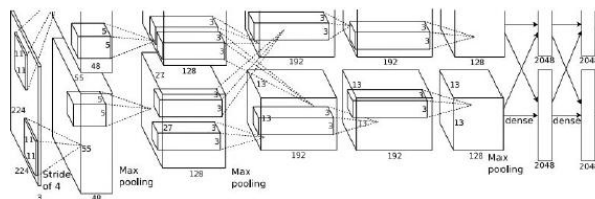
CAT: (x, y, w, h)



DOG: (x, y, w, h)

DOG: (x, y, w, h)

CAT: (x, y, w, h)



DUCK: (x, y, w, h)

DUCK: (x, y, w, h)

....

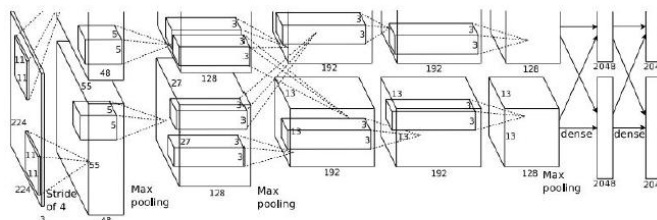
Formulation #2: **Sliding-window**-based detection

- Sliding a window across an image and evaluating an object detection model at every location and every scale



Formulation #2: **Sliding-window**-based detection

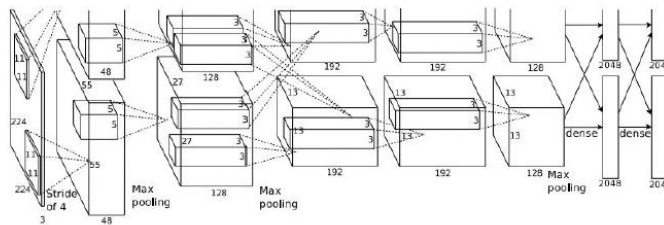
- When using CNN for object detection, we can apply a CNN model to different locations, scales, and aspect ratios



Dog? NO
Cat? NO
Background? YES

Formulation #2: **Sliding-window**-based detection

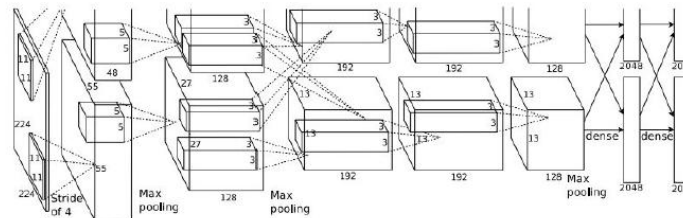
- When using CNN for object detection, we can apply a CNN model to different locations, scales, and aspect ratios



Dog? YES
Cat? NO
Background? NO

Formulation #2: **Sliding-window**-based detection

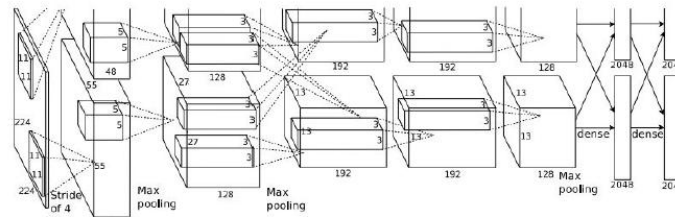
- When using CNN for object detection, we can apply a CNN model to different locations, scales, and aspect ratios



Dog? YES
Cat? NO
Background? NO

Formulation #2: **Sliding-window**-based detection

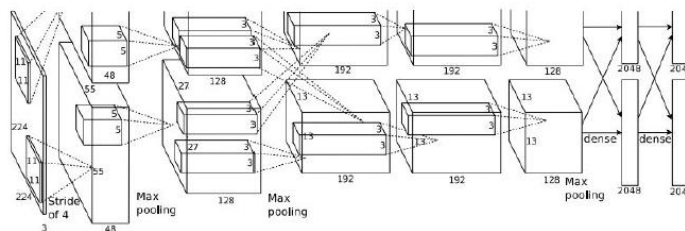
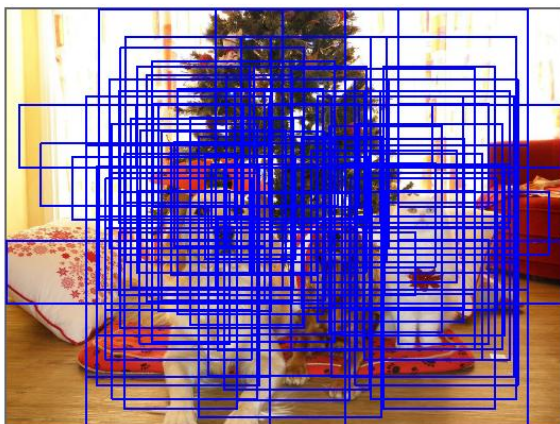
- When using CNN for object detection, we can apply a CNN model to different locations, scales, and aspect ratios



Dog? NO
Cat? YES
Background? NO

Formulation #2: Sliding-window-based detection

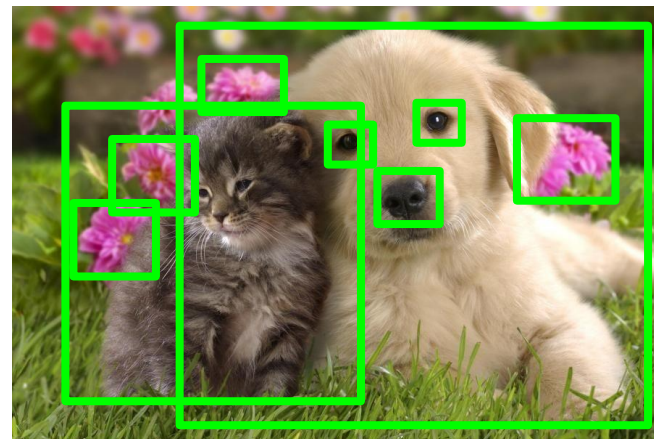
- When using CNN for object detection, we can apply a CNN model to **different locations, scales, and aspect ratios**
 - A boosted classifier with rectangle features is efficient
 - CNN is not as efficient as the boosted classifier



Dog? NO
Cat? YES
Background? NO

Region proposal

- A **proposal** is a rectangular region that likely covers an object
- Neighboring and similar pixels tend to belong to the same object or background
- Blob-like regions may cover objects

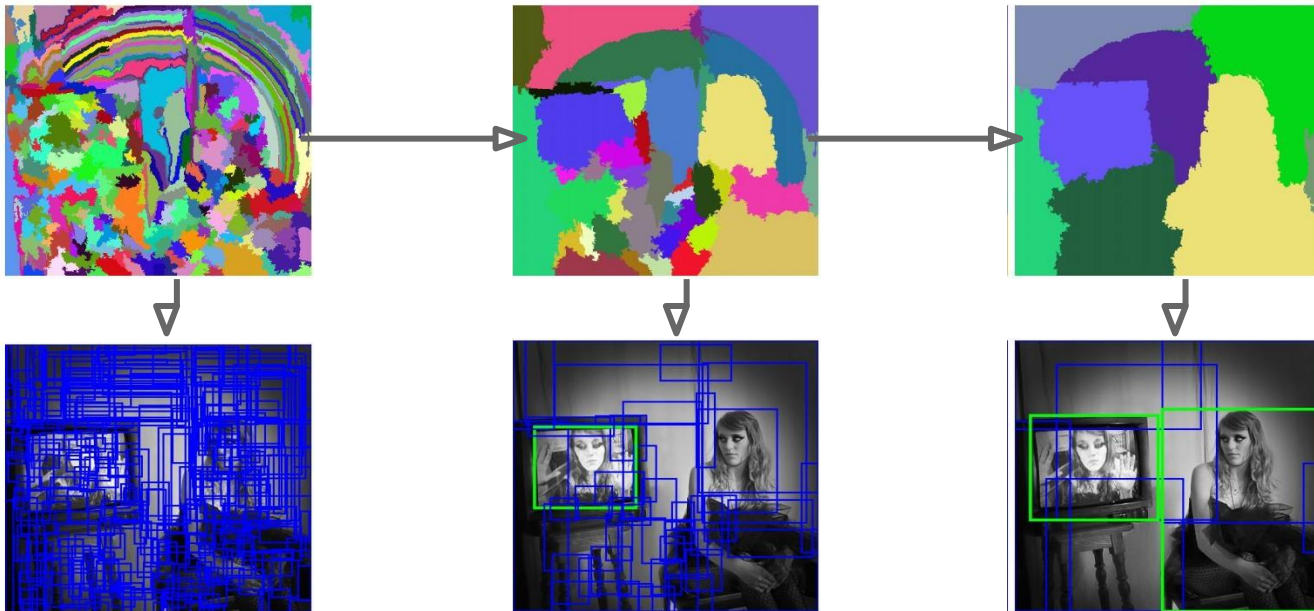


Selective search for region proposal generation

- With a high probability, an object will well be covered by at least one proposal

Bottom-up segmentation, merging regions at multiple scales

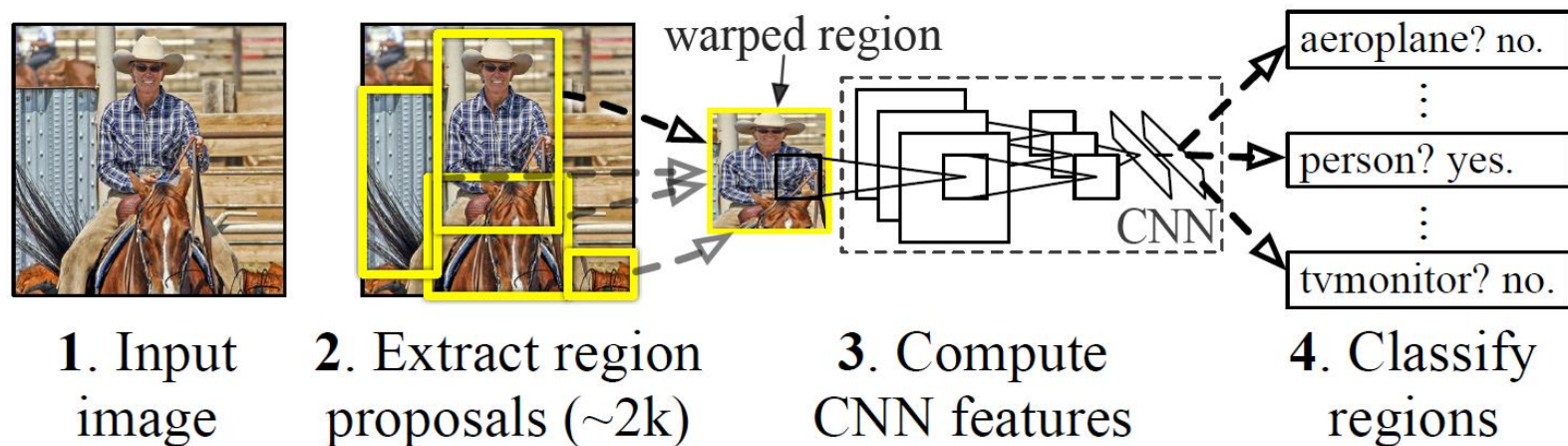
Convert
regions
to
boxes



R-CNN: Regions with CNN Features

[Girshick et al., CVPR'14]

- System overview



- Proposal extraction: Using selective search [Uijlin et al., IJCV'13]
- Compute CNN features in the layer 'fc7' of Caffe CNN
- Region classification: linear SVMs or a softmax classifier
- Regression-based bounding box refinement

R-CNN

- Given an image



Input image



R-CNN

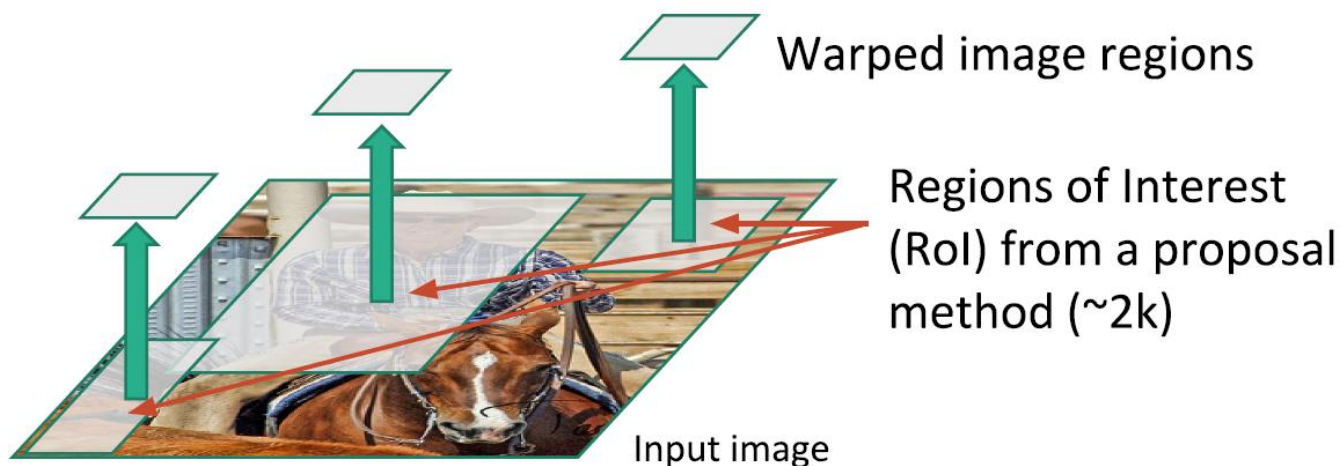
- Apply the selective search and get 2K object proposals



Regions of Interest
(RoI) from a proposal
method (~2k)

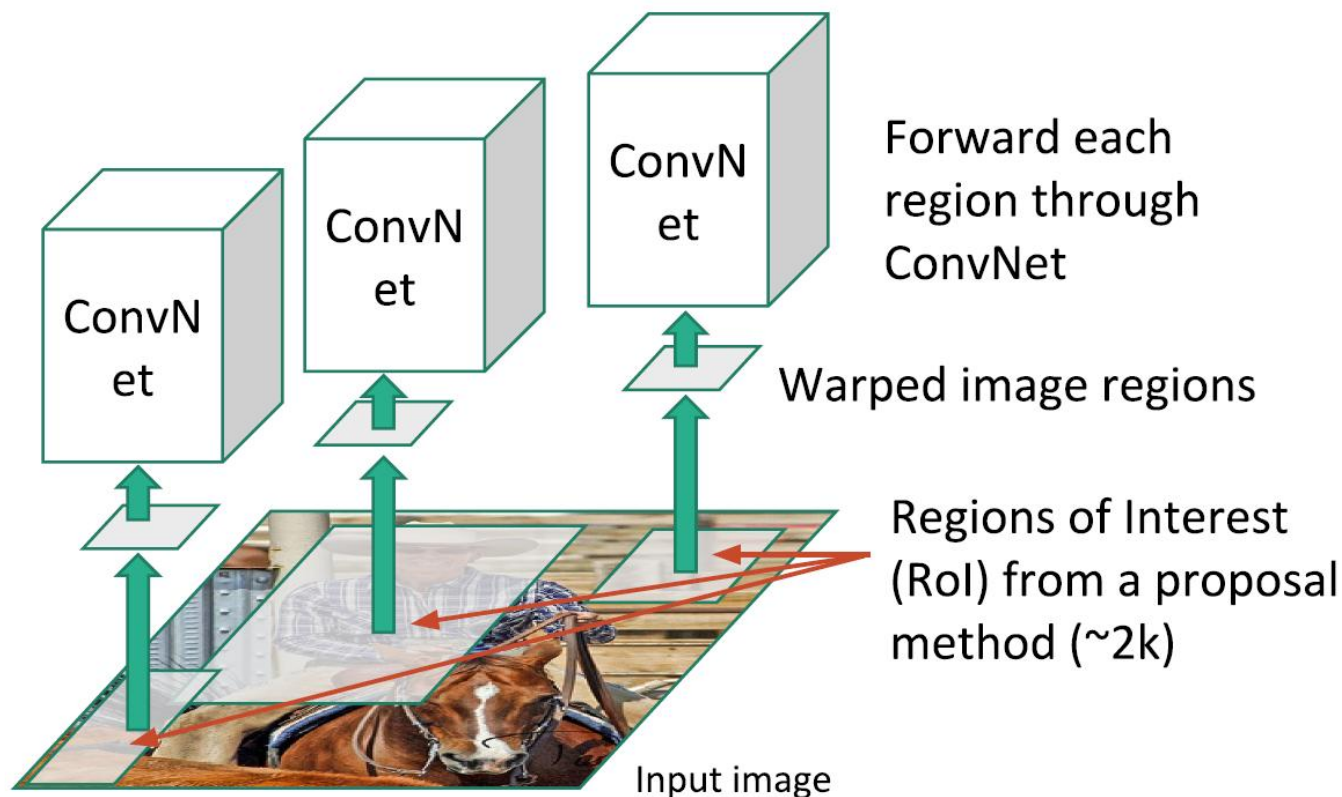
R-CNN

- Warp each proposal to serve as the input to CNN



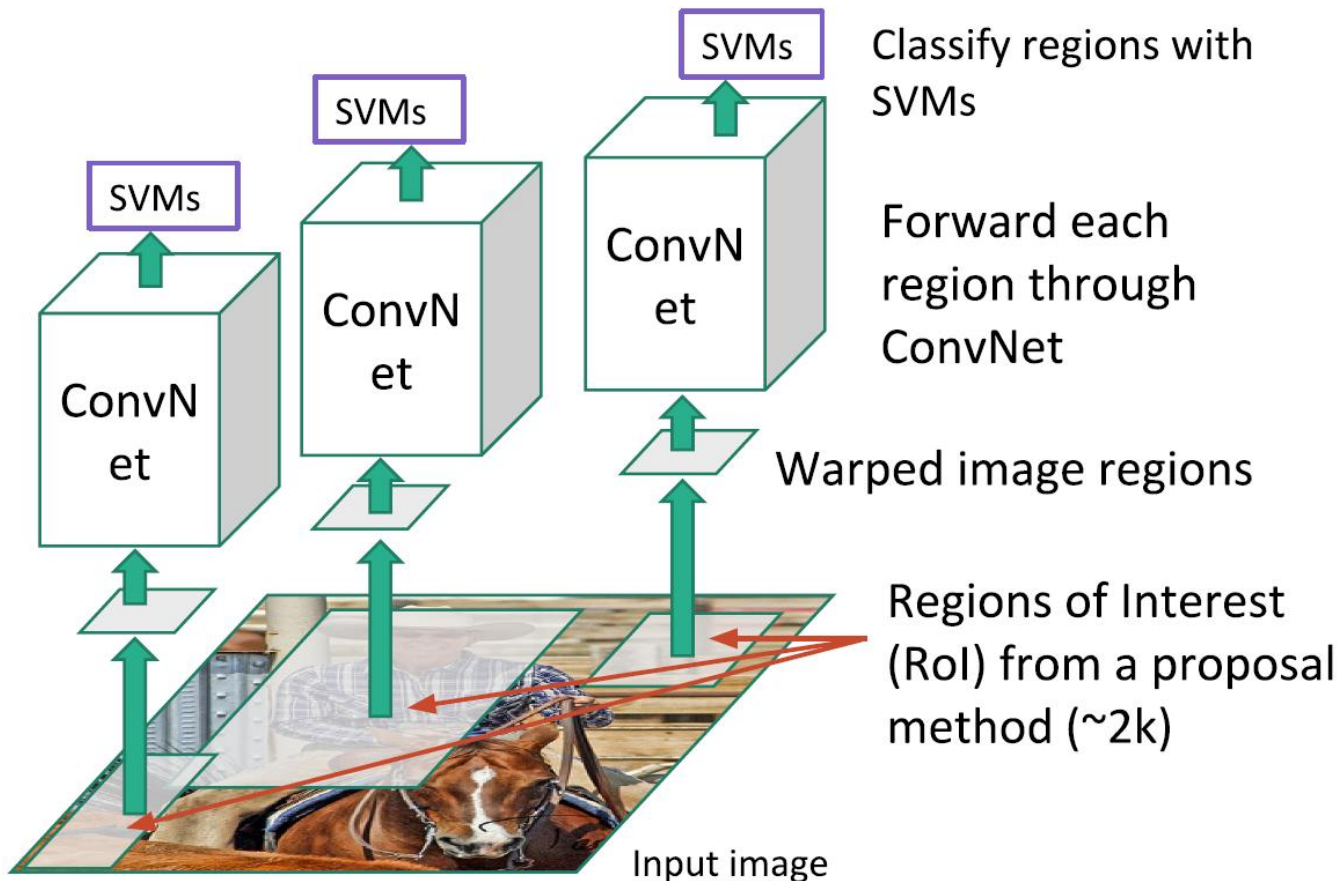
R-CNN

- Apply the CNN model to each warped proposal



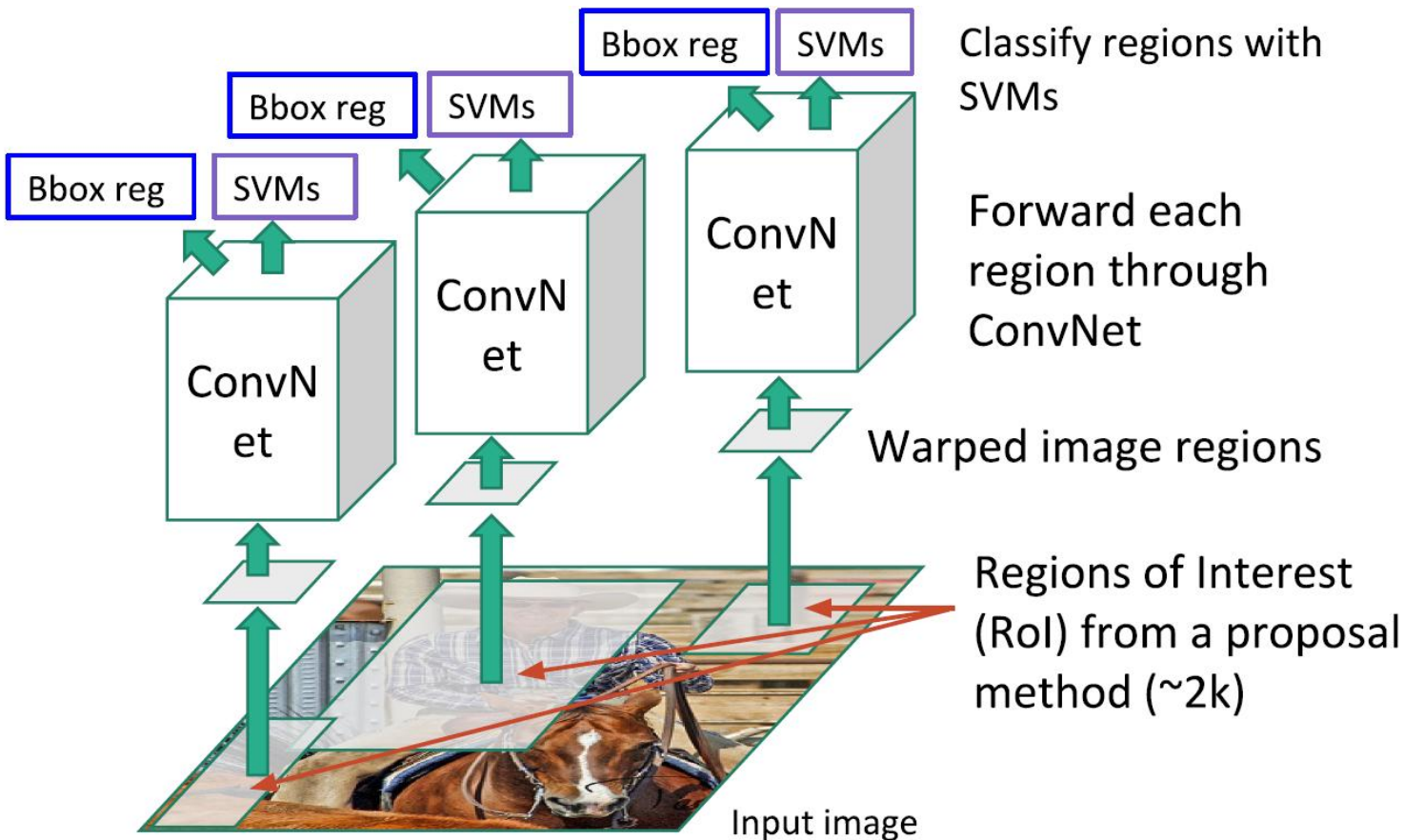
R-CNN

- Use SVMs as the classifier to separate objects from non-objects



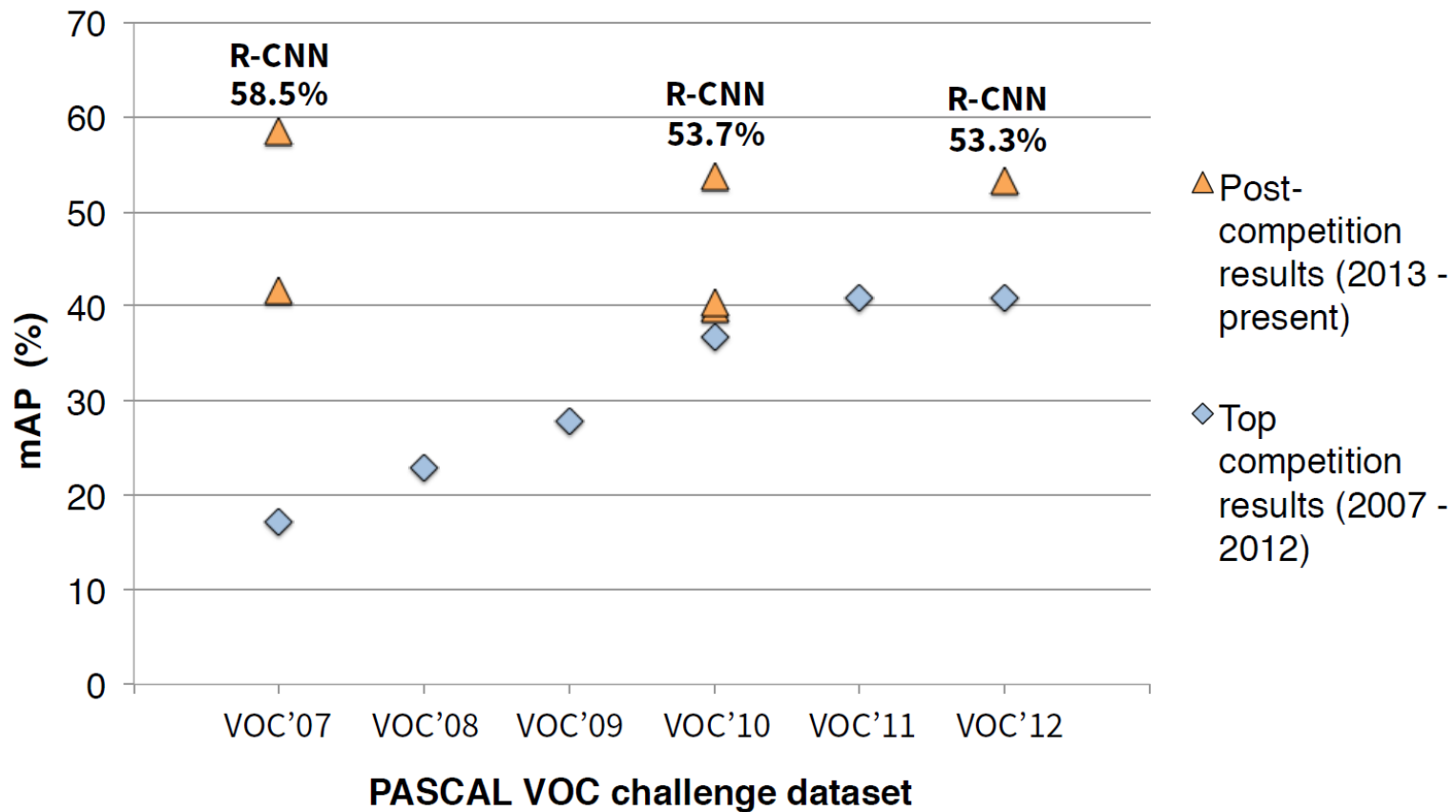
R-CNN

- Learn a regressor to align positive proposals to the ground truth



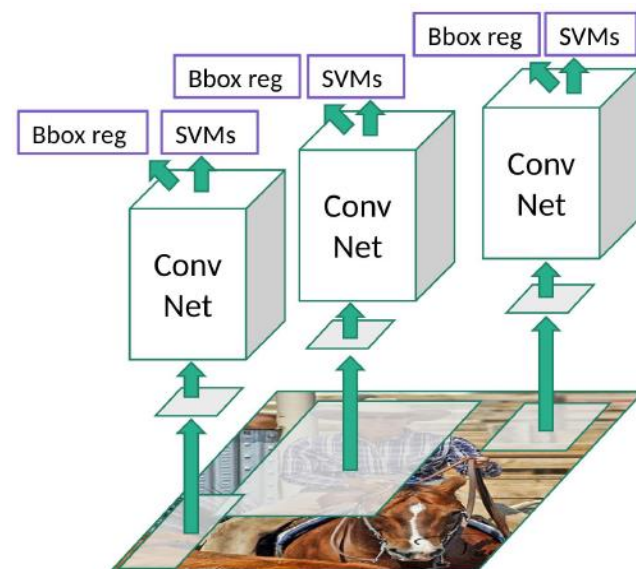
Experimental results

- Evaluation on the Pascal VOC dataset



Problems of R-CNN

- Ad hoc training objectives
 - Fine-tune network with softmax classifier (log loss)
 - Train post-hoc linear SVMs (hinge loss)
 - Train post-hoc bounding-box regressions (least squares)
- Inference (detection) is slow
 - 47s / image with VGG16 [Simonyan & Zisserman, ICLR'15]



Outline

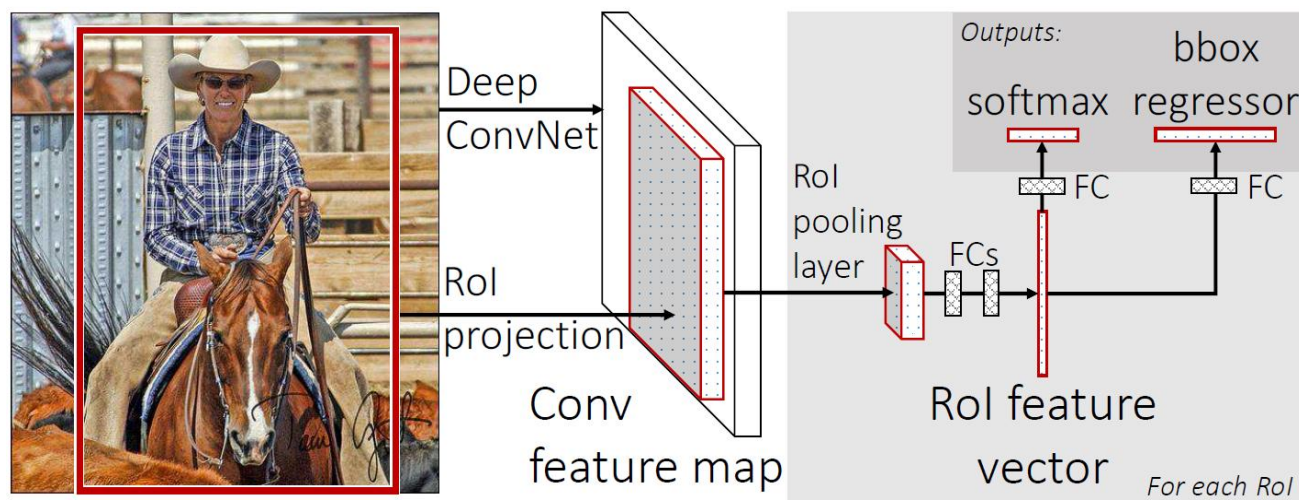
- Object detection
 - R-CNN
 - Fast R-CNN
 - Faster R-CNN
 - YOLO
 - FCOS
 - DETR
 - Deformable DETR



Fast R-CNN

[Girshick, ICCV'15]

- System overview



- Apply fully convolutional networks to **the whole image**
- RoI pooling**: each proposal is pooled into a fix-size feature map
- Classification with a softmax layer
- Regression-based bounding box refinement

Fast R-CNN

- Given an input image

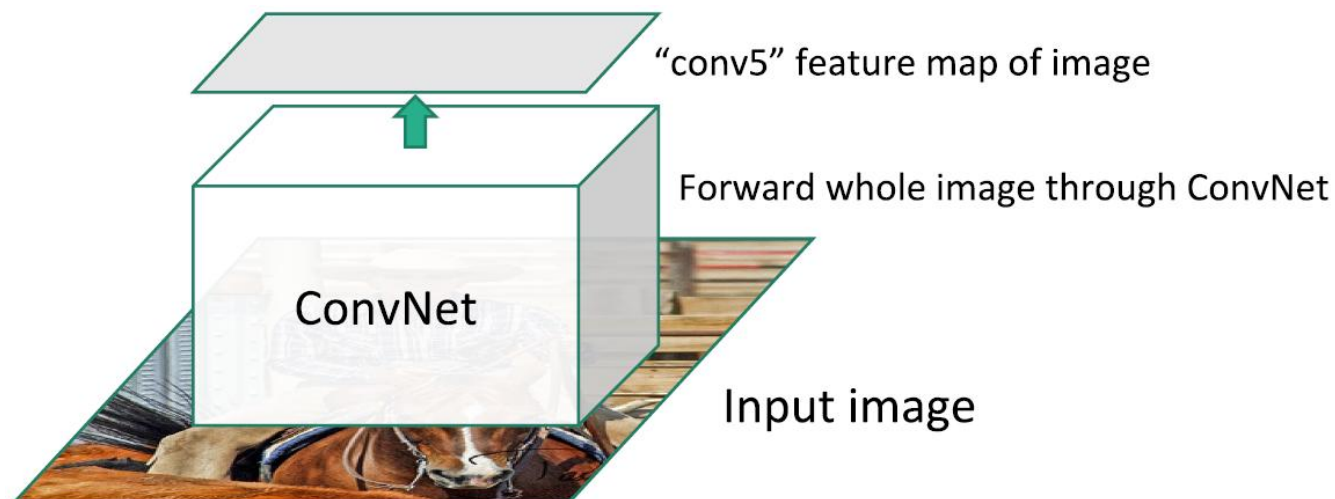


Input image



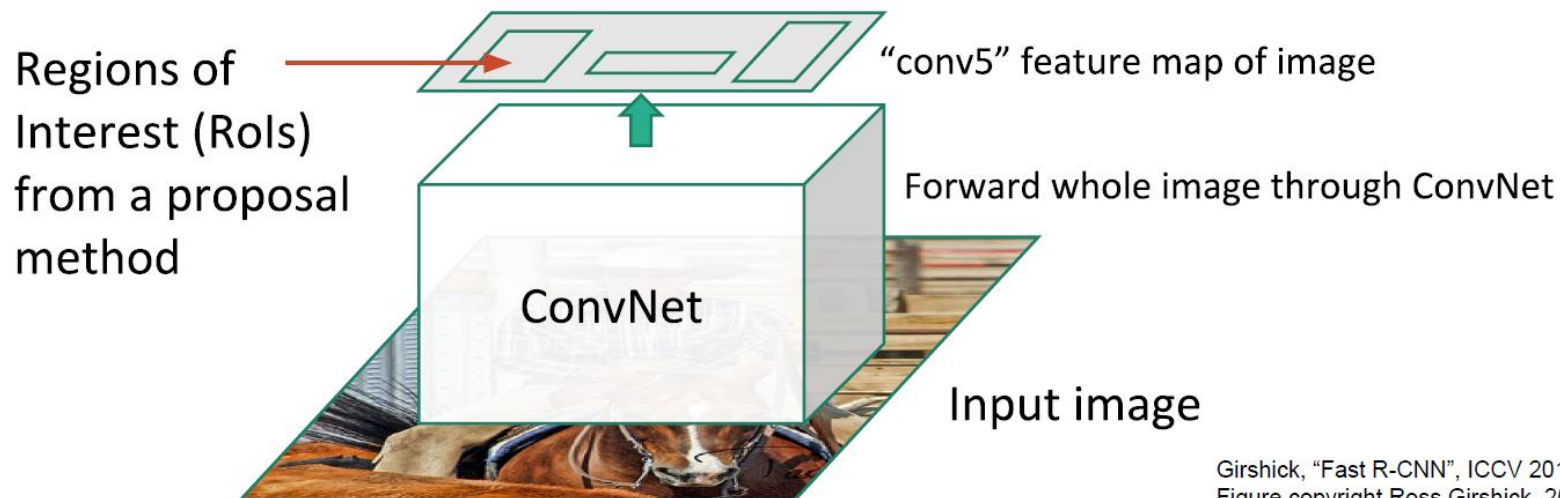
Fast R-CNN

- Apply the CNN to **the whole image** and get the feature map at layer conv5



Fast R-CNN

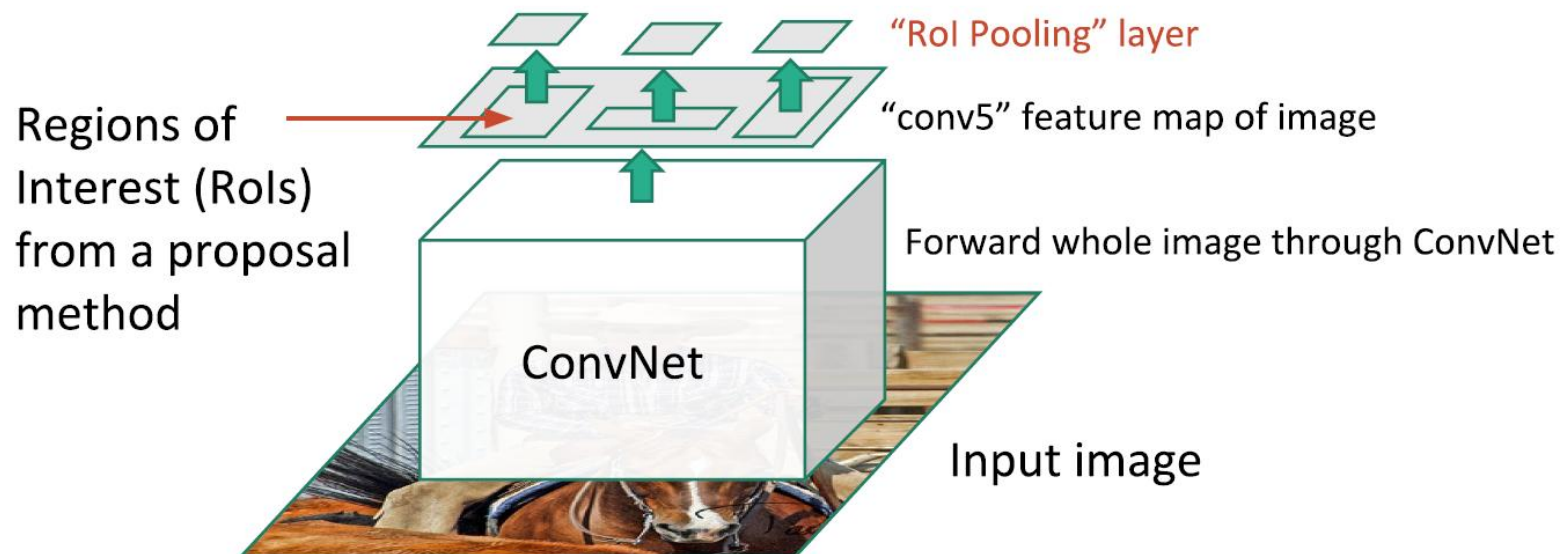
- Get the proposal regions on the feature map



Girshick, "Fast R-CNN", ICCV 2015
Figure copyright Ross Girshick, 2015

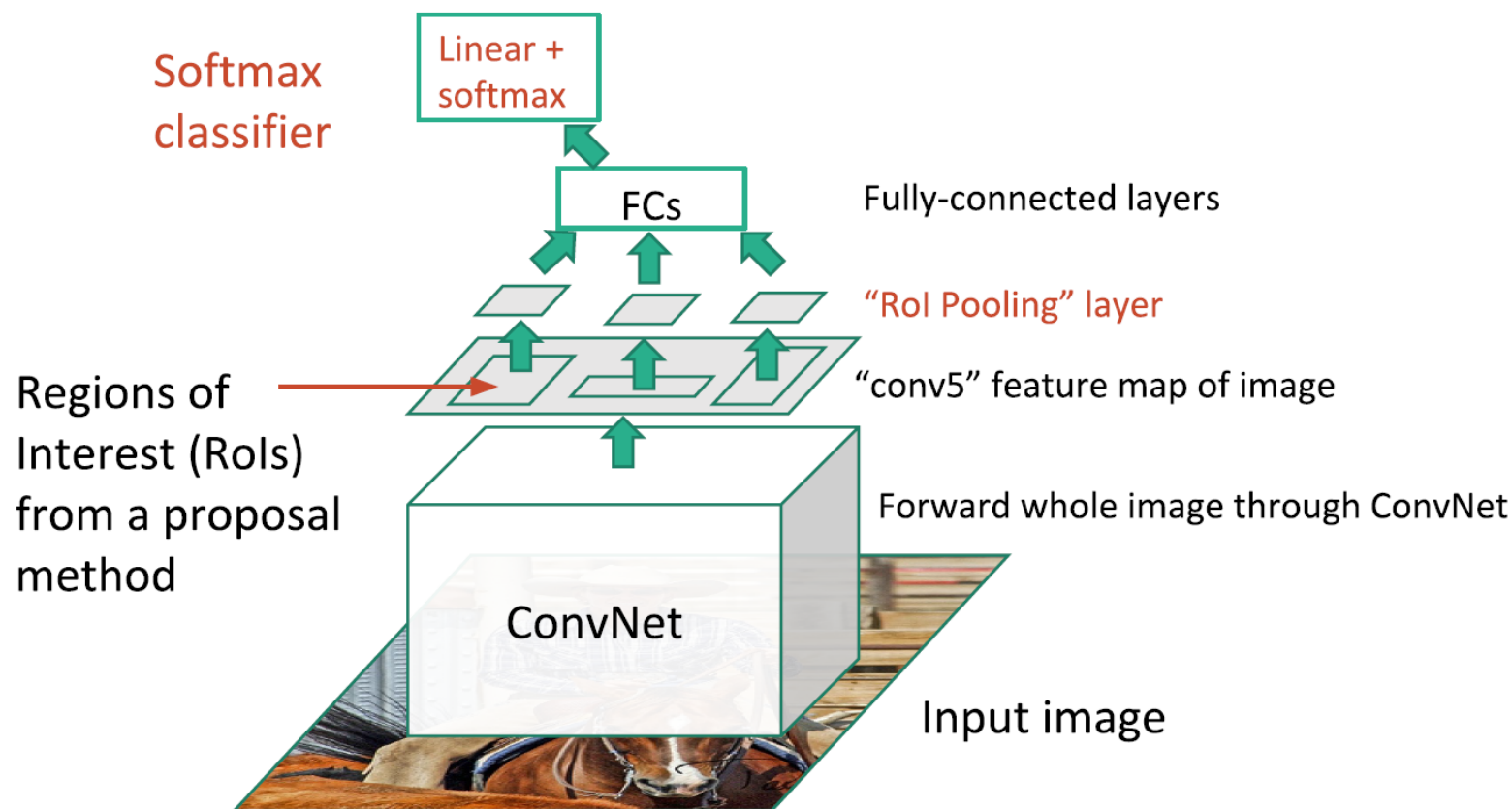
Fast R-CNN

- Apply **Rol pooling** to each region proposal
 - Pool each Rol to the region of the same size



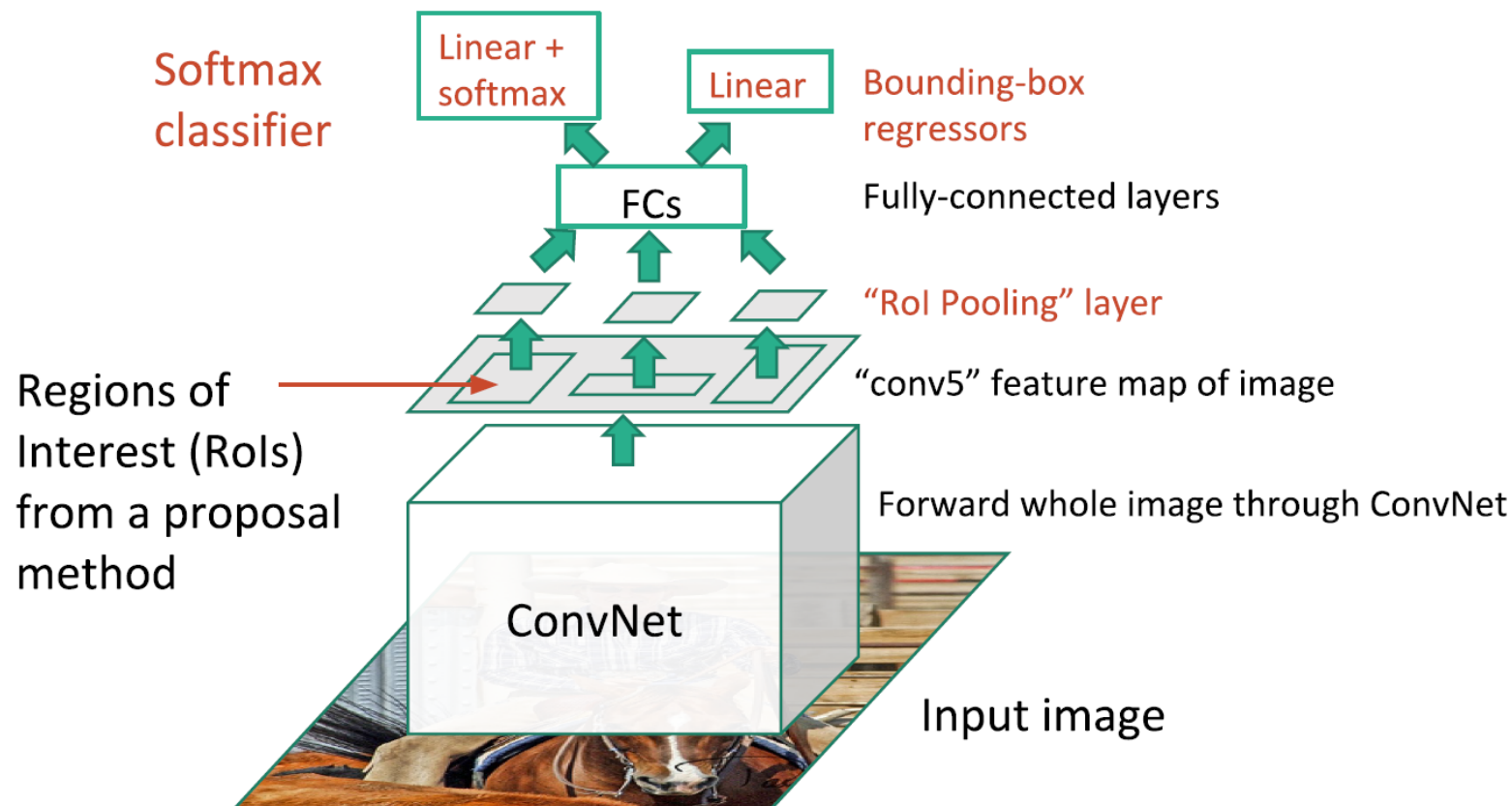
Fast R-CNN

- Learn and apply **fully connected layers (FCs)** to each region for classification



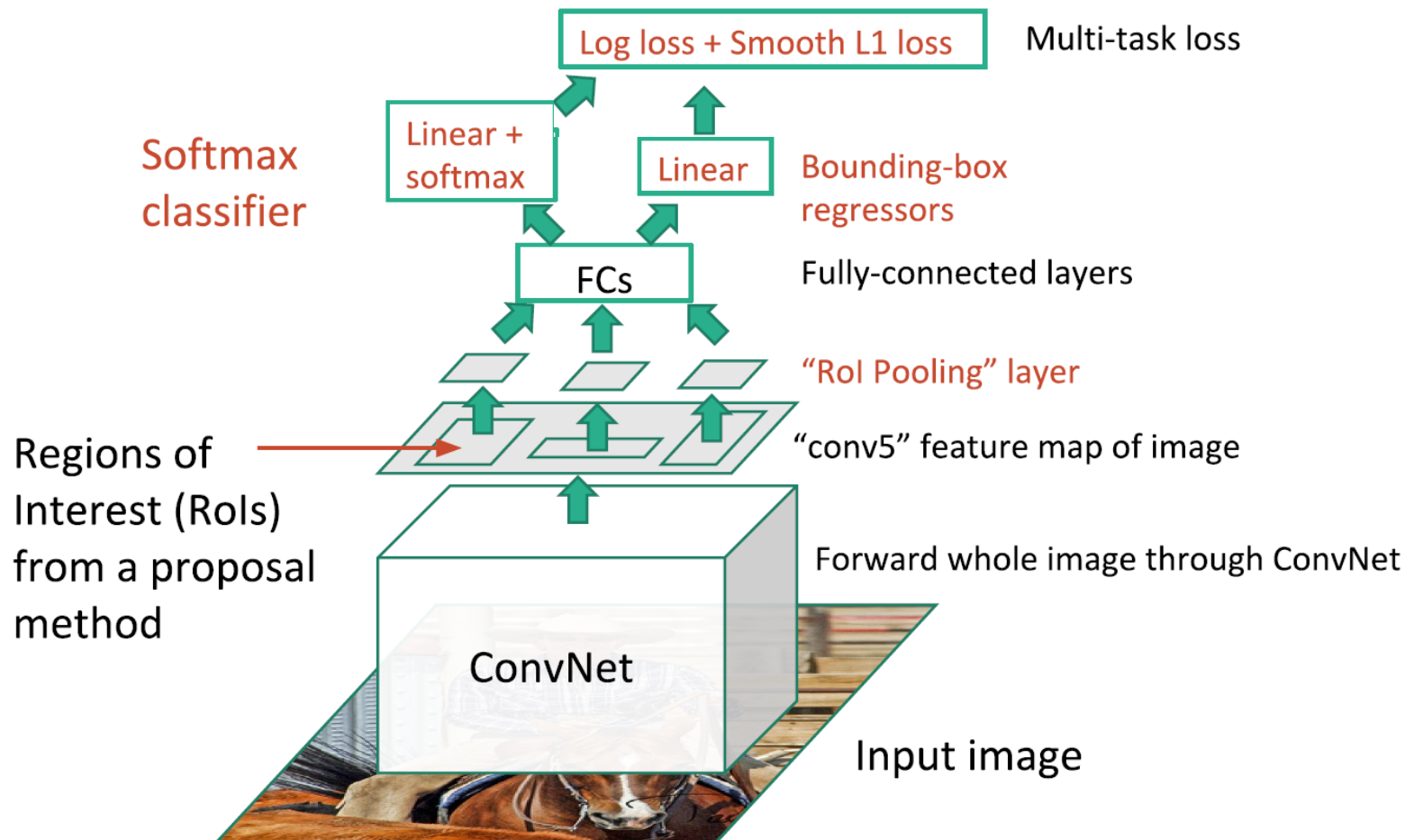
Fast R-CNN

- Learn a regressor to align the positive proposals to ground truth



Fast R-CNN

- The joint loss function



Experimental Results

- Evaluation on Pascal VOC dataset

	Fast R-CNN	R-CNN [1]	SPP-net [2]
Train time (h)	9.5	84	25
- Speedup	8.8x	1x	3.4x
Test time / image	0.32s	47.0s	2.3s
Test speedup	146x	1x	20x
mAP	66.9%	66.0%	63.1%

[Girshick et al., ICCV'15]



Problems of Fast R-CNN

- In R-CNN, applying CNN to each proposal dominates the inference time
- In Fast-CNN, running selective search for proposal generation dominates the inference time
 - The computation bottleneck is selective search



Outline

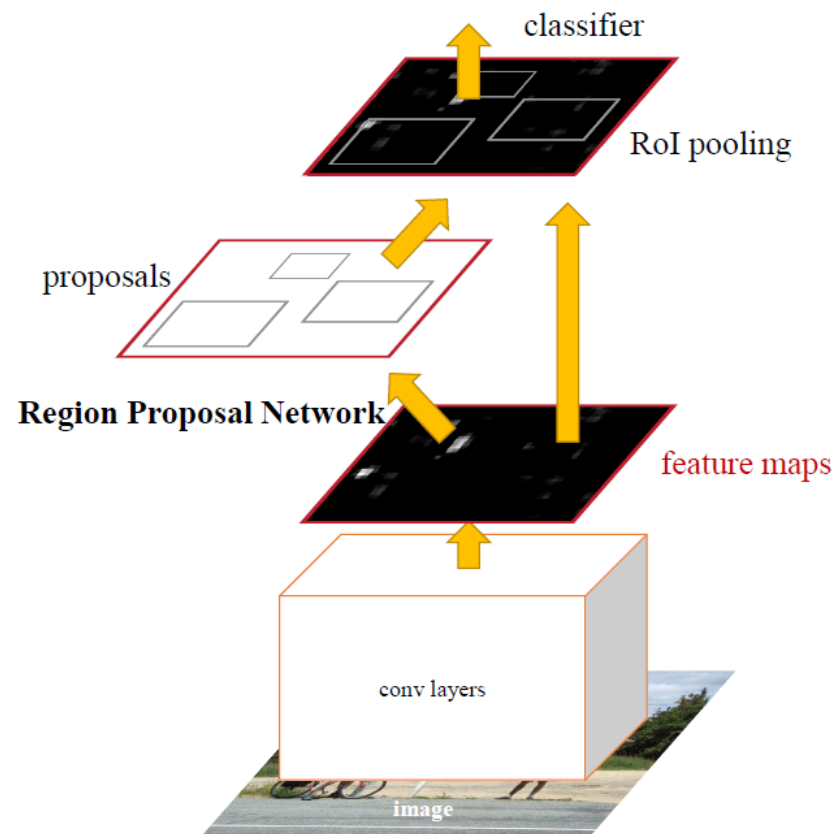
- Object detection
 - R-CNN
 - Fast R-CNN
 - Faster R-CNN
 - YOLO
 - FCOS
 - DETR
 - Deformable DETR



Faster R-CNN

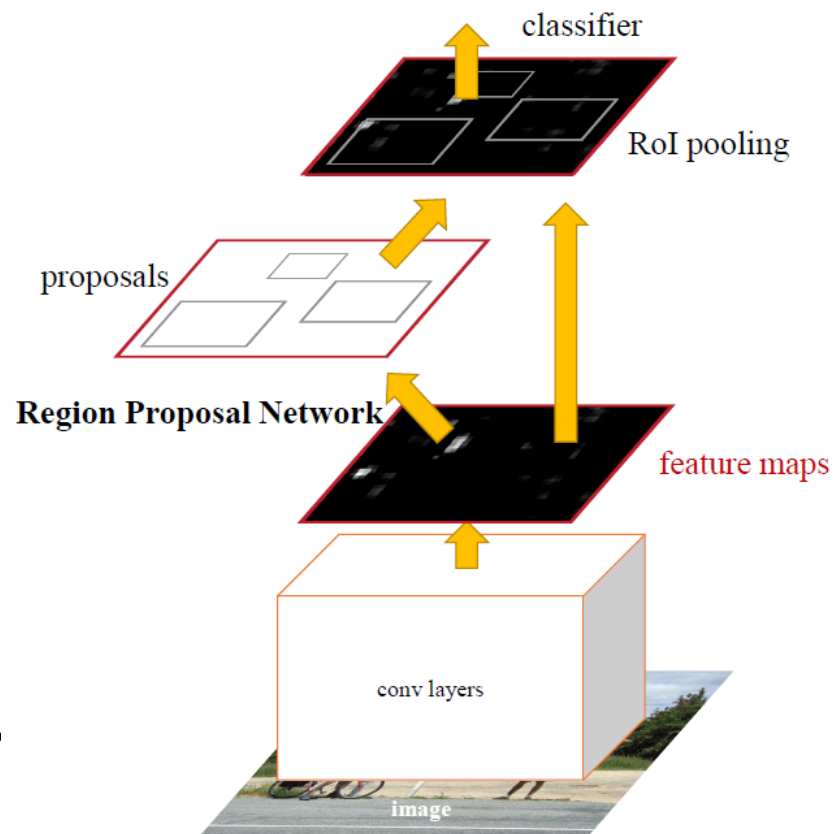
- Selective search is replaced by RPN
- **Region proposal network (RPN)**
 - Follows the last conv. layer
 - Produces proposals
- Share the computation of CNN forward path with Fast R-CNN
- GPU acceleration is allowed

[Ren et al., NIPS'15]



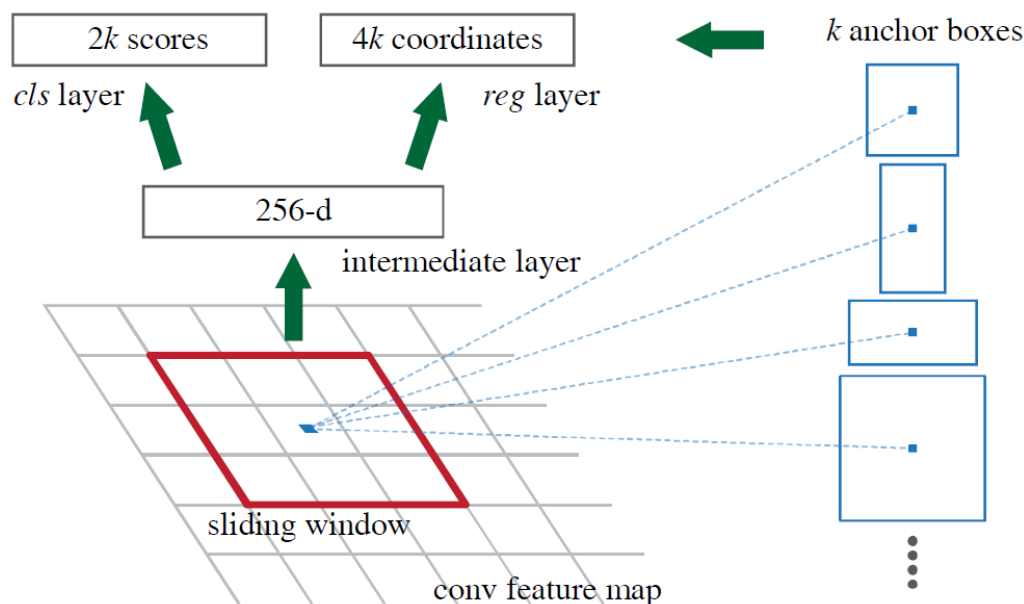
Region proposal network (RPN)

- Input: feature maps of an image
- Output: A set of **proposals**
 - Rectangular regions
 - Objectness scores
- Network: fully convolutional network (FCN)
- After running PRN, RoI pooling is used to compile the features of each proposal for object detection, just like Fast R-CNN



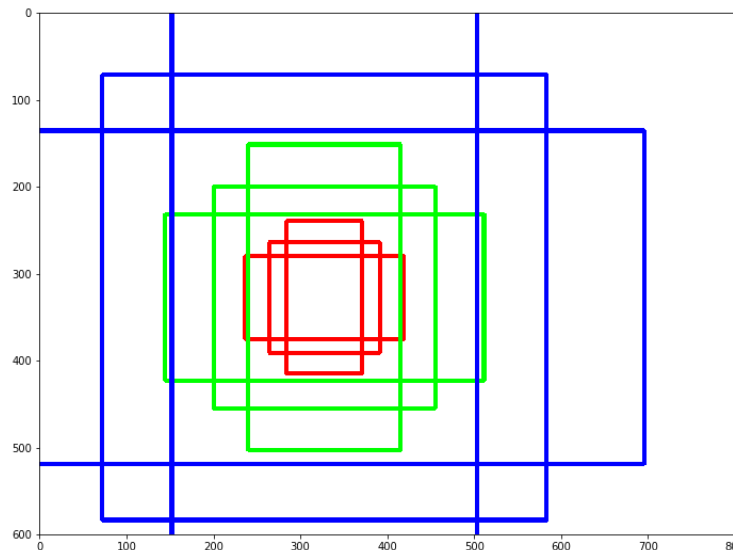
Region proposal network (RPN)

- Slide a small window on the feature maps
- At each position, k ($=9$) **anchors** are sampled
- Build a small network
 - *cls*: predict each anchor as object or non-object
 - *Reg*: regress the bounding box location of each anchor

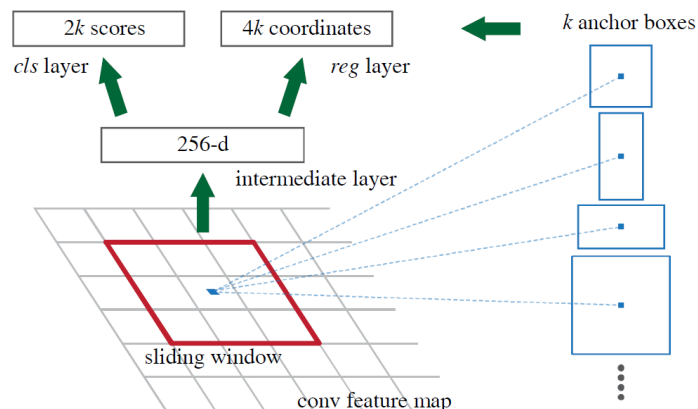


Anchor

- Resize an image such that its short side is 600 pixels
- 9 anchors at each position
 - Three scales: 128 x 128, 256 x 256, and 512 x 512
 - Three aspect ratios: 1 x 1, 2 x 1, and 1 x 2
- An image of size 800 x 600
- The 9 anchors at (320, 320)
- Suppose the total stride is 16
- There are $51 \times 39 \times 9 = 17901$ anchors



Objective function of RPN

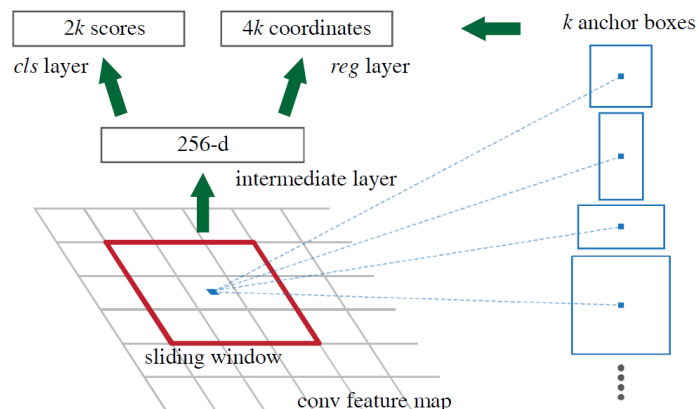


- Objective function:

$$L(\{p_i\}, \{t_i\}) = \frac{1}{N_{cls}} \sum_i L_{cls}(p_i, p_i^*) + \lambda \frac{1}{N_{reg}} \sum_i p_i^* L_{reg}(t_i, t_i^*)$$

- cls layer* predicts an anchor as object (proposal) or non-object
- An anchor is positive (object) if
 - 1) It has the highest IoU overlap with a ground-truth box, or
 - 2) It has an IoU overlap higher than 0.7 with a ground-truth box
- An anchor is negative if its IoU ratios for all ground-truth boxes are less than 0.3

Objective function of RPN



- Objective function:

$$L(\{p_i\}, \{t_i\}) = \frac{1}{N_{cls}} \sum_i L_{cls}(p_i, p_i^*) + \lambda \frac{1}{N_{reg}} \sum_i p_i^* L_{reg}(t_i, t_i^*)$$

- reg layer* regresses the bounding box location
- $t = (x, y, w, h)$ denotes the box center, width, and height
- Three boxes: x_A for anchor, x for predicted box, and x^* for ground truth

$$t_x = (x - x_a)/w_a, \quad t_y = (y - y_a)/h_a,$$

$$t_w = \log(w/w_a), \quad t_h = \log(h/h_a),$$

$$t_x^* = (x^* - x_a)/w_a, \quad t_y^* = (y^* - y_a)/h_a,$$

$$t_w^* = \log(w^*/w_a), \quad t_h^* = \log(h^*/h_a),$$

Experimental Results

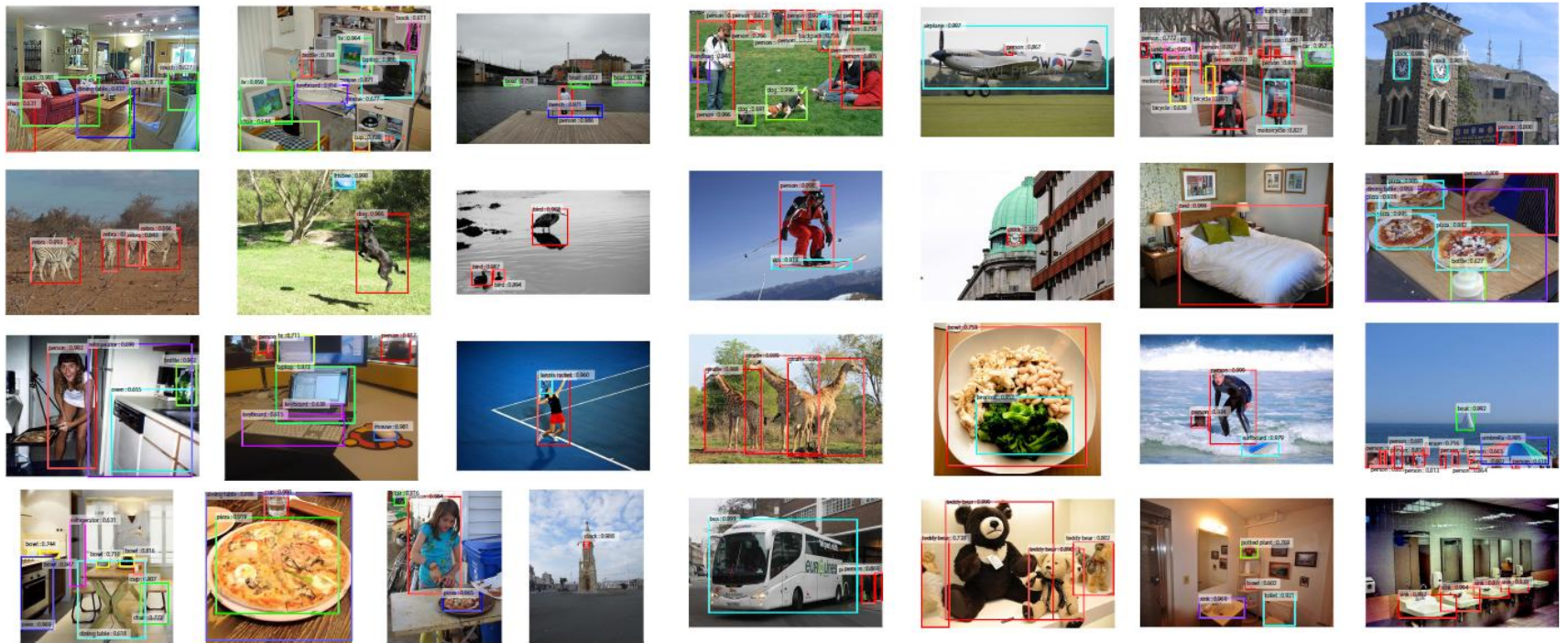
- Evaluation on Pascal VOC dataset

	R-CNN	Fast R-CNN	Faster R-CNN
Test time per image (with proposals)	50 seconds	2 seconds	0.2 seconds
(Speedup)	1x	25x	250x
mAP (VOC 2007)	66.0	66.9	66.9

slide credit: Li et al.



Experimental Results



Outline

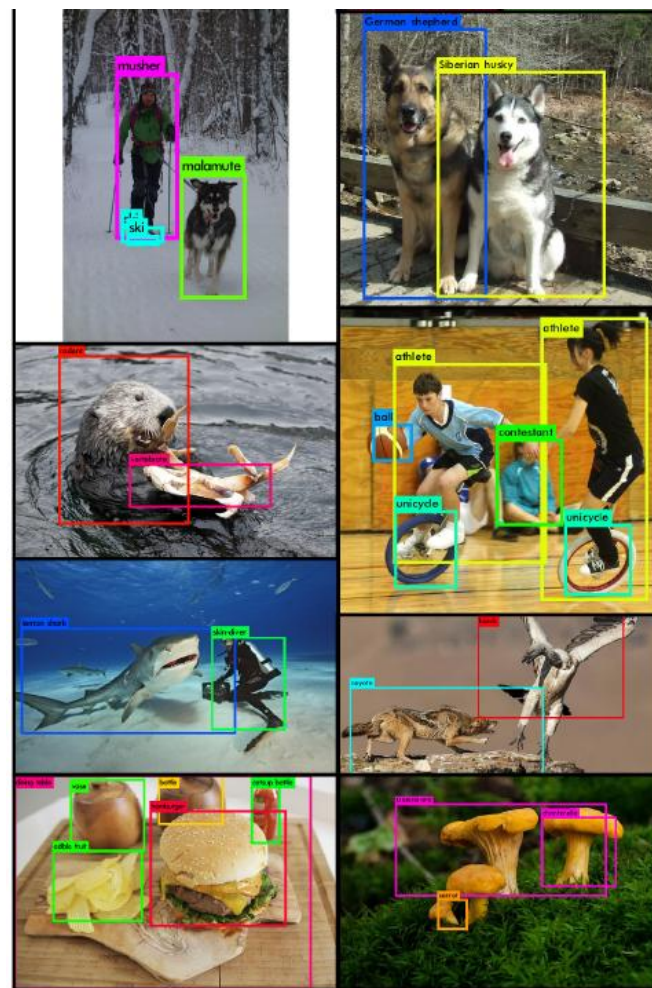
- Object detection
 - R-CNN
 - Fast R-CNN
 - Faster R-CNN
 - YOLO
 - FCOS
 - DETR
 - Deformable DETR



YOLO9000

[Redmon & Farhadi, CVPR'17]

Detection Frameworks	Train	mAP	FPS
Fast R-CNN [5]	2007+2012	70.0	0.5
Faster R-CNN VGG-16[15]	2007+2012	73.2	7
Faster R-CNN ResNet[6]	2007+2012	76.4	5
YOLO [14]	2007+2012	63.4	45
SSD300 [11]	2007+2012	74.3	46
SSD500 [11]	2007+2012	76.8	19
YOLOv2 288 × 288	2007+2012	69.0	91
YOLOv2 352 × 352	2007+2012	73.7	81
YOLOv2 416 × 416	2007+2012	76.8	67
YOLOv2 480 × 480	2007+2012	77.8	59
YOLOv2 544 × 544	2007+2012	78.6	40



YOLO (You Only Look Once)

[Redmon et al., CVPR'16]

- R-CNN, Fast R-CNN, and Faster R-CNN
 - Generate proposals
 - Classify each proposal
 - Post-processing
- In YOLO, object detection is formulated as a **regression** problem to spatially predict **bounding boxes** and **class probabilities**
 - Only one feedforward
 - Global context of the entire image
 - Robust to various background
 - Much fast (real-time detection, 45 frames per second)



YOLO (You Only Look Once)

- YOLO casts object detection as a single regression problem
 - Input: an image
 - Output: bounding boxes and class probabilities of detected objects

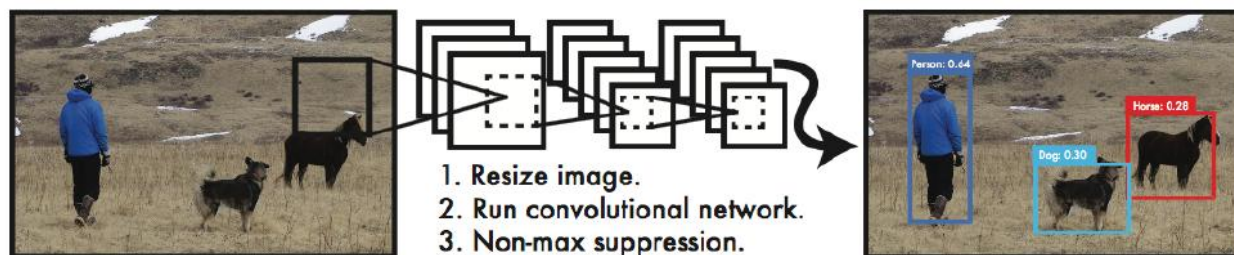


Figure 1: The YOLO Detection System. Processing images with YOLO is simple and straightforward. Our system (1) resizes the input image to 448×448 , (2) runs a single convolutional network on the image, and (3) thresholds the resulting detections by the model's confidence.

Unified detection

- An image is divided into an $S \times S$ grid
- Each grid cell
 - Predicts B bounding boxes and the confidence scores for these boxes
 - ◆ Bounding box: (x, y, w, h)
 - ◆ Confidence score: $\text{Pr}(\text{Object}) * \text{IOU}$
 - ◆ Confidence score should be zero if no object exists in the cell
 - ◆ Otherwise confidence score should equal the IOU between the predicted box and the ground truth
 - Predicts C conditional class probabilities, $\text{Pr}(\text{class}_i | \text{Object})$
- YOLO outputs an $S \times S \times (B*5+C) = 7 \times 7 \times (2 * 5 + 20)$ tensor



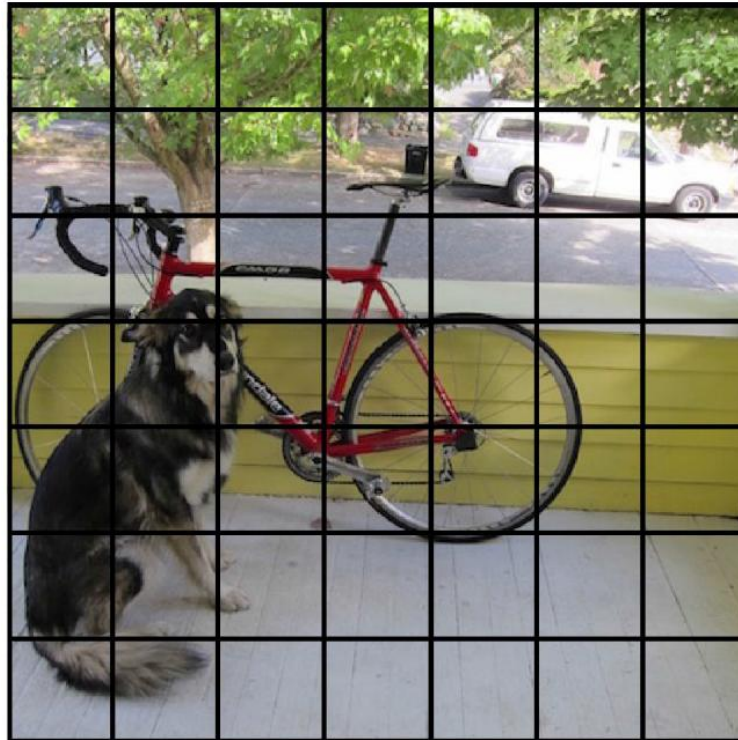
Unified detection

- An image is divided into an $S \times S$ grid
- Each grid cell
 - Predicts B bounding boxes and the confidence scores for these boxes
 - ◆ Bounding box: (x, y, w, h)
 - ◆ Confidence score: $\text{Pr}(\text{Object}) * \text{IOU}$
 - ◆ Confidence score should be zero if no object exists in the cell
 - ◆ Otherwise confidence score should equal the IOU between the predicted box and the ground truth
 - Predicts C conditional class probabilities, $\text{Pr}(\text{class}_i | \text{Object})$
- YOLO outputs an $S \times S \times (B*5+C) = 7 \times 7 \times (2 * 5 + 20)$ tensor



Splitting an image into a grid

- Split an input image into an $S \times S$ grid



a 7 x 7 grid

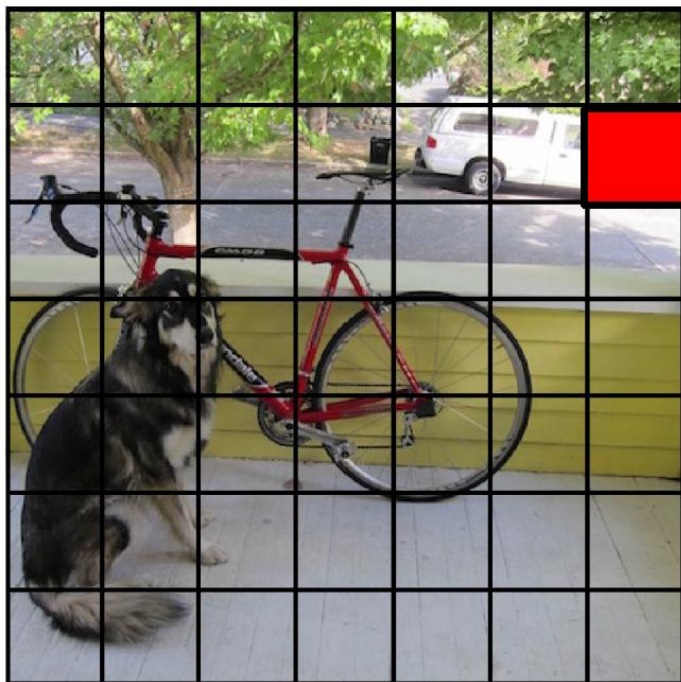
Unified detection

- An image is divided into an $S \times S$ grid
- Each grid cell
 - Predicts B bounding boxes and the confidence scores for these boxes
 - ◆ Bounding box: (x, y, w, h)
 - ◆ Confidence score: $\text{Pr}(\text{Object}) * \text{IOU}$
 - ◆ Confidence score should be zero if no object exists in the cell
 - ◆ Otherwise confidence score should equal the IOU between the predicted box and the ground truth
 - Predicts C conditional class probabilities, $\text{Pr}(\text{class}_i | \text{Object})$
- YOLO outputs an $S \times S \times (B*5+C) = 7 \times 7 \times (2 * 5 + 20)$ tensor

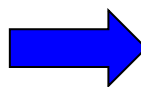


For each cell

- Each cell B boxes and their confidences
 - A box: (x, y, w, h)
 - Its confidence: $P(\text{Object})$

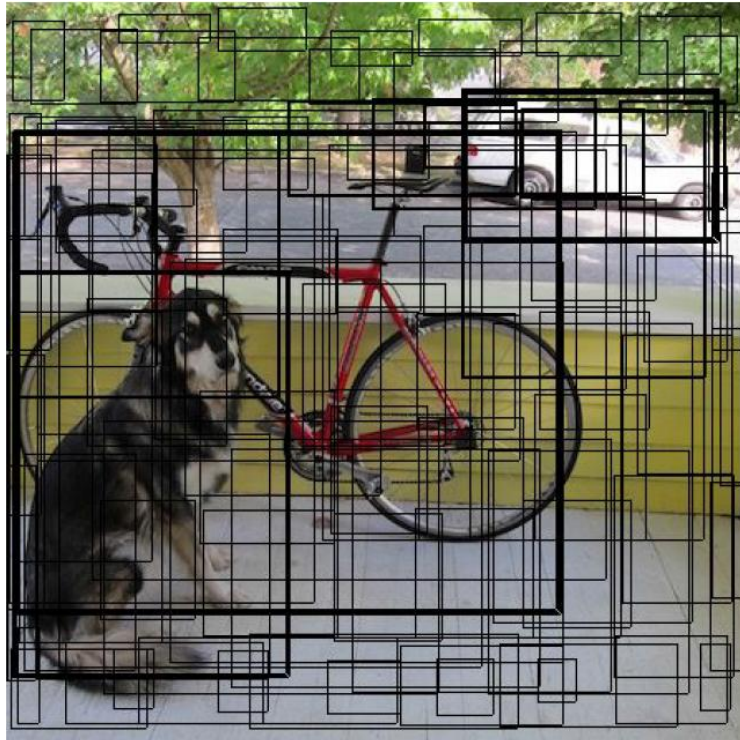


$B = 2$



All boxes with their confidences

- 98 predicted boxes
 - 49 (=7x7) cells
 - Each cell has two boxes

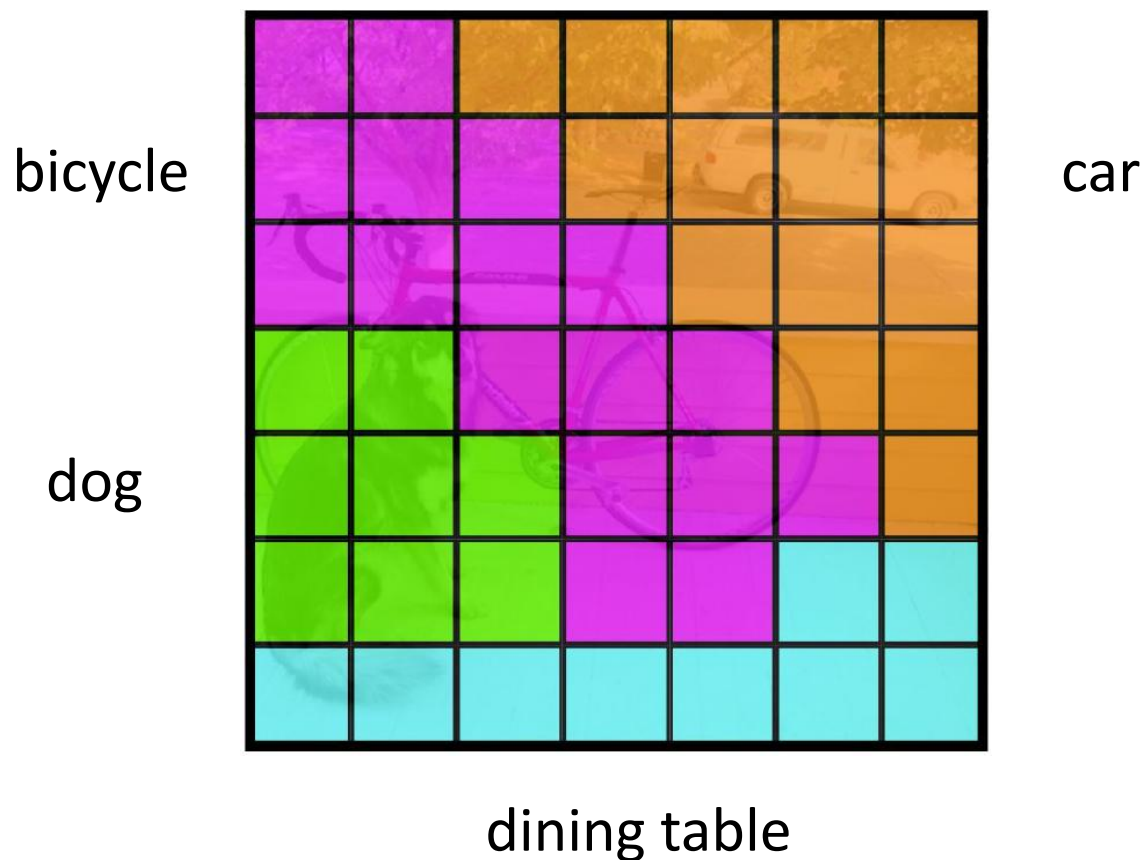


Unified detection

- An image is divided into an $S \times S$ grid
- Each grid cell
 - Predicts B bounding boxes and the confidence scores for these boxes
 - ◆ Bounding box: (x, y, w, h)
 - ◆ Confidence score: $\text{Pr}(\text{Object}) * \text{IOU}$
 - ◆ Confidence score should be zero if no object exists in the cell
 - ◆ Otherwise confidence score should equal the IOU between the predicted box and the ground truth
 - Predicts C conditional class probabilities, $\text{Pr}(\text{class}_i | \text{Object})$
- YOLO outputs an $S \times S \times (B*5+C) = 7 \times 7 \times (2 * 5 + 20)$ tensor



Each cell predicts a class probability



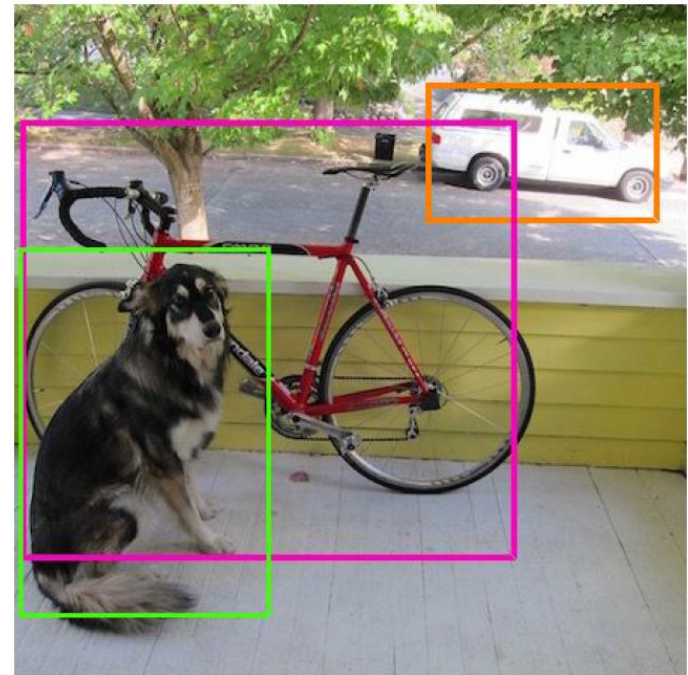
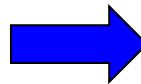
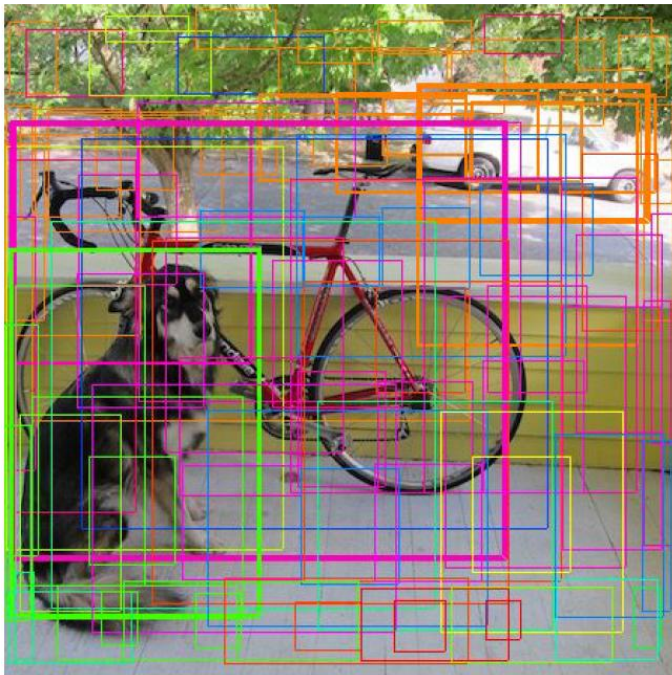
Combine predicted boxes and class probability

$$P(\text{class} | \text{Object}) * P(\text{Object}) \\ = P(\text{class})$$

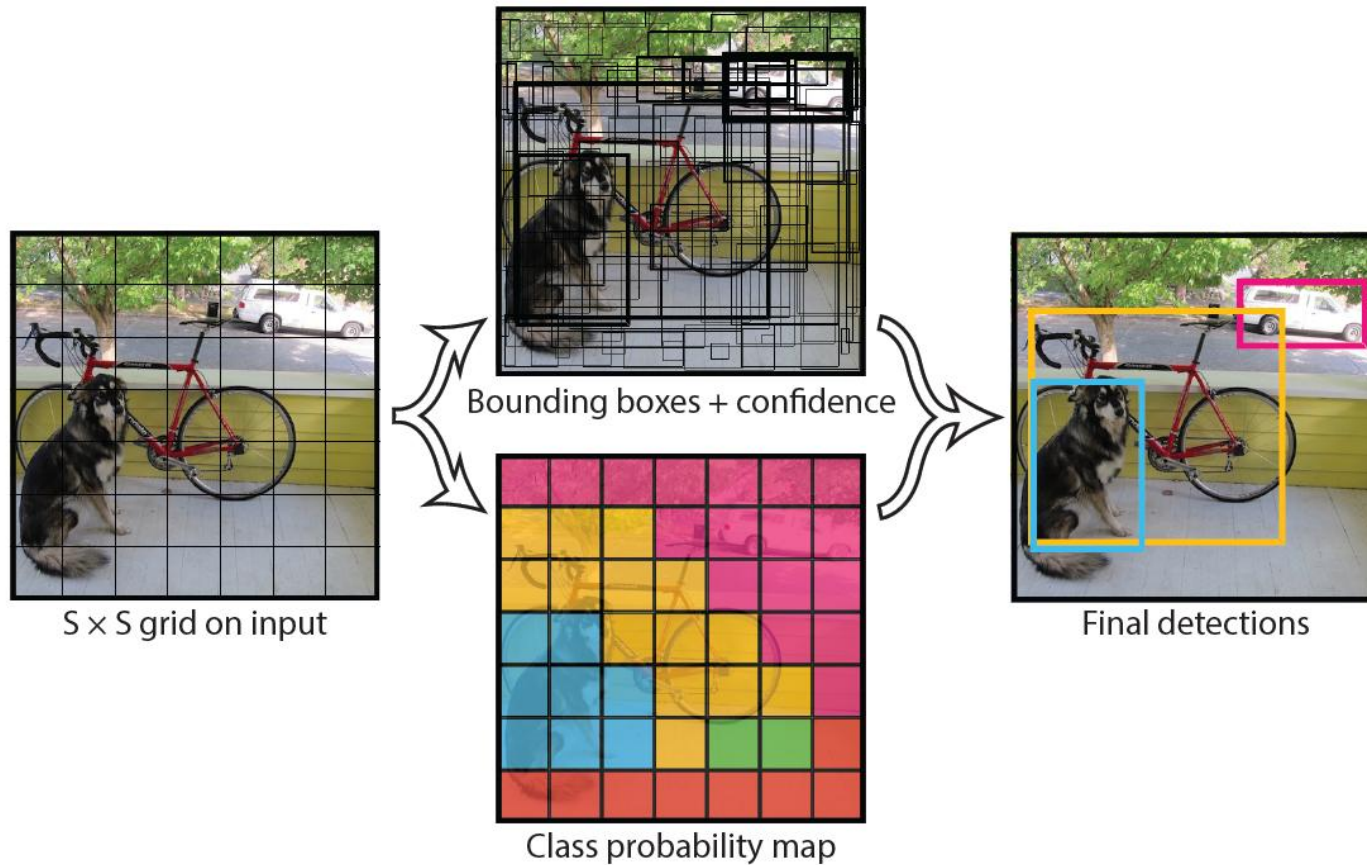


Combine predicted boxes and class probability

- Thresholding followed by NMS (non maximum suppression)
 - Thresholding: remove low-confidence boxes
 - NMS: deal with overlapping boxes



Unified detection



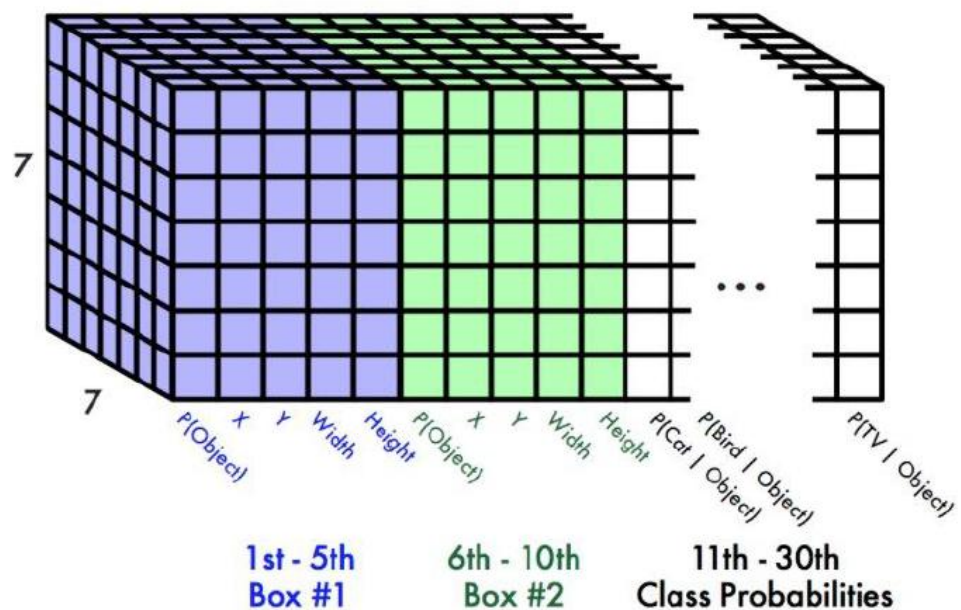
Unified detection

- An image is divided into an $S \times S$ grid
- Each grid cell
 - Predicts B bounding boxes and the confidence scores for these boxes
 - ◆ Bounding box: (x, y, w, h)
 - ◆ Confidence score: $\text{Pr}(\text{Object}) * \text{IOU}$
 - ◆ Confidence score should be zero if no object exists in the cell
 - ◆ Otherwise confidence score should equal the IOU between the predicted box and the ground truth
 - Predicts C conditional class probabilities, $\text{Pr}(\text{class}_i | \text{Object})$
- YOLO outputs an $S \times S \times (B*5+C) = 7 \times 7 \times (2 * 5 + 20)$ tensor



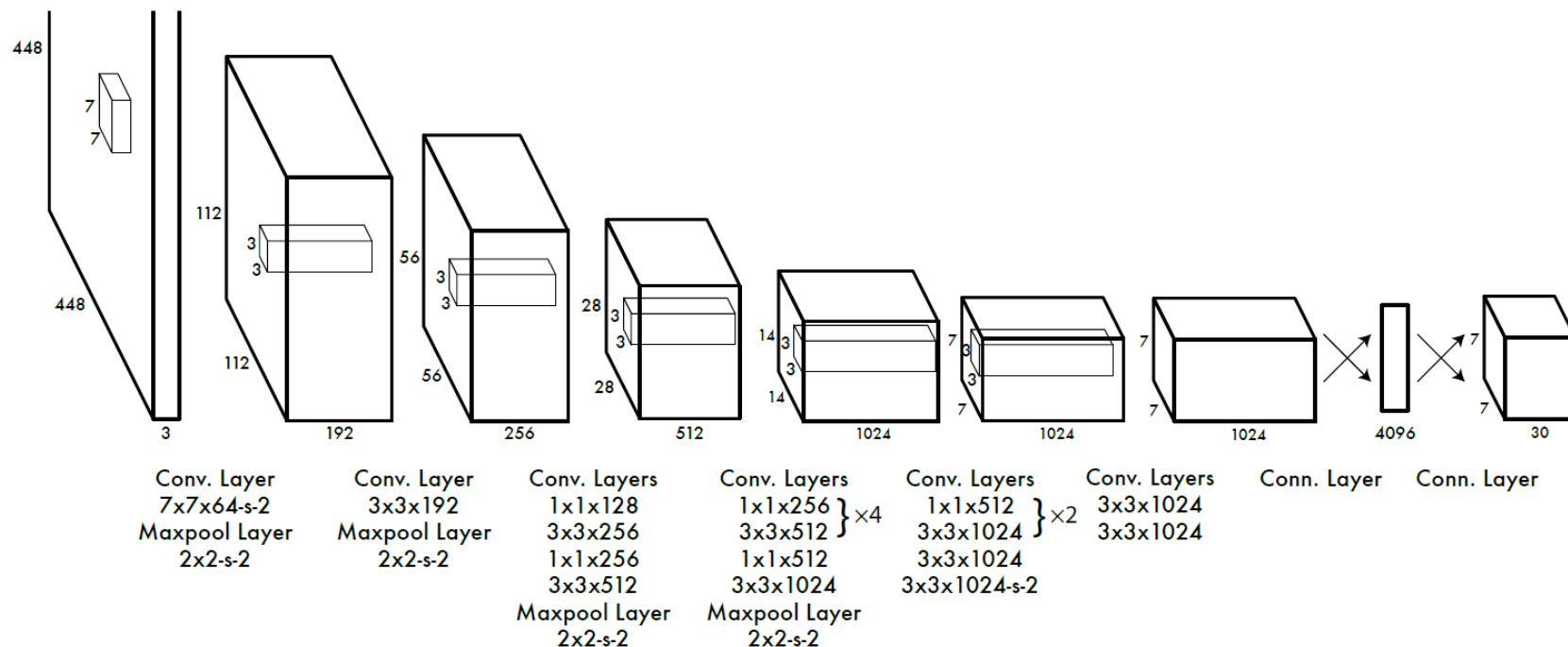
Output of YOLO

- Each cell predicts
 - 2 boxes
 - ◆ Box: (x, y, w, h)
 - ◆ Confidence
 - 20 class probabilities

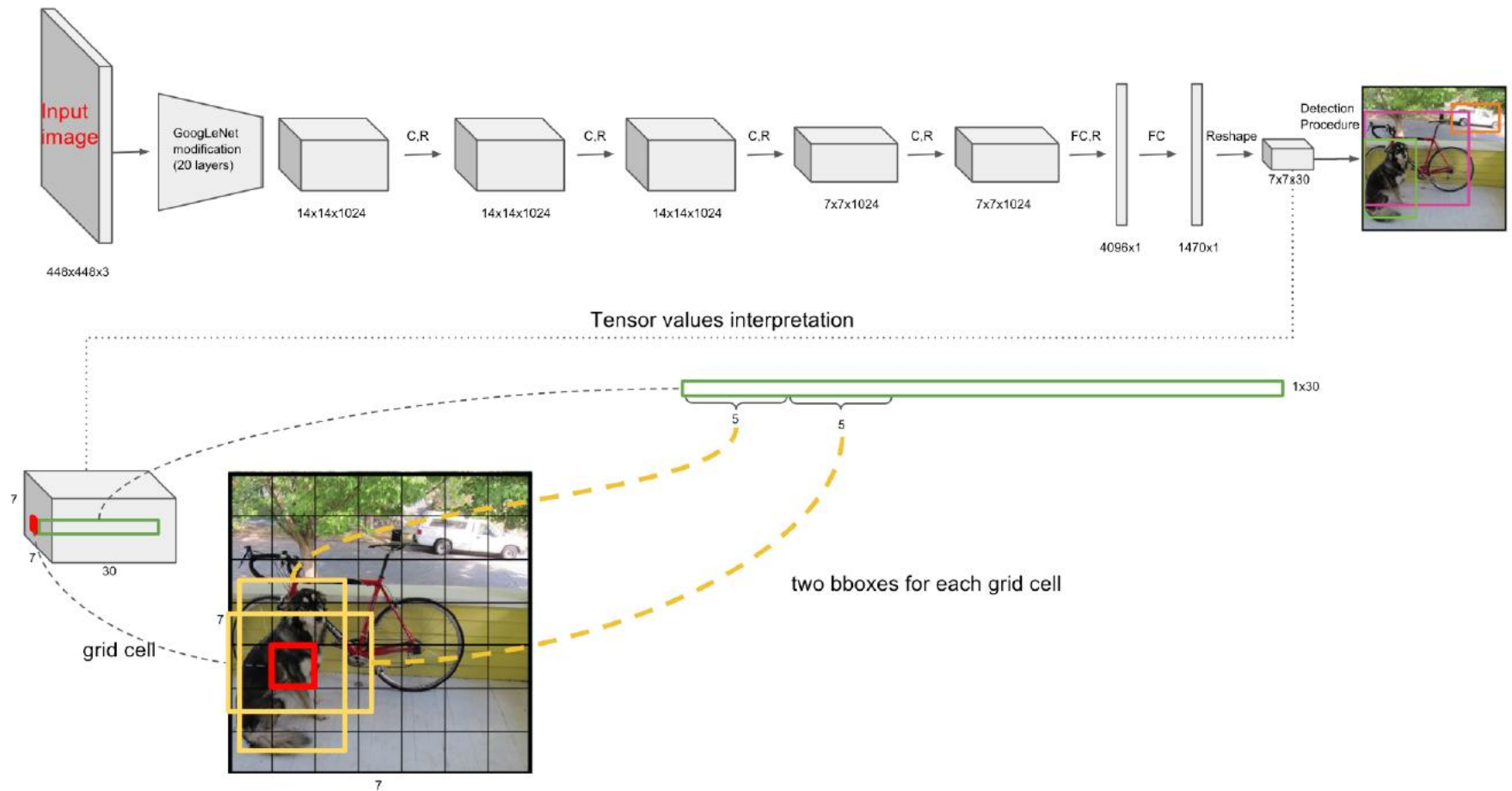


Network architecture

- The network of YOLO has 24 convolutional layers followed by 2 fully connected layers



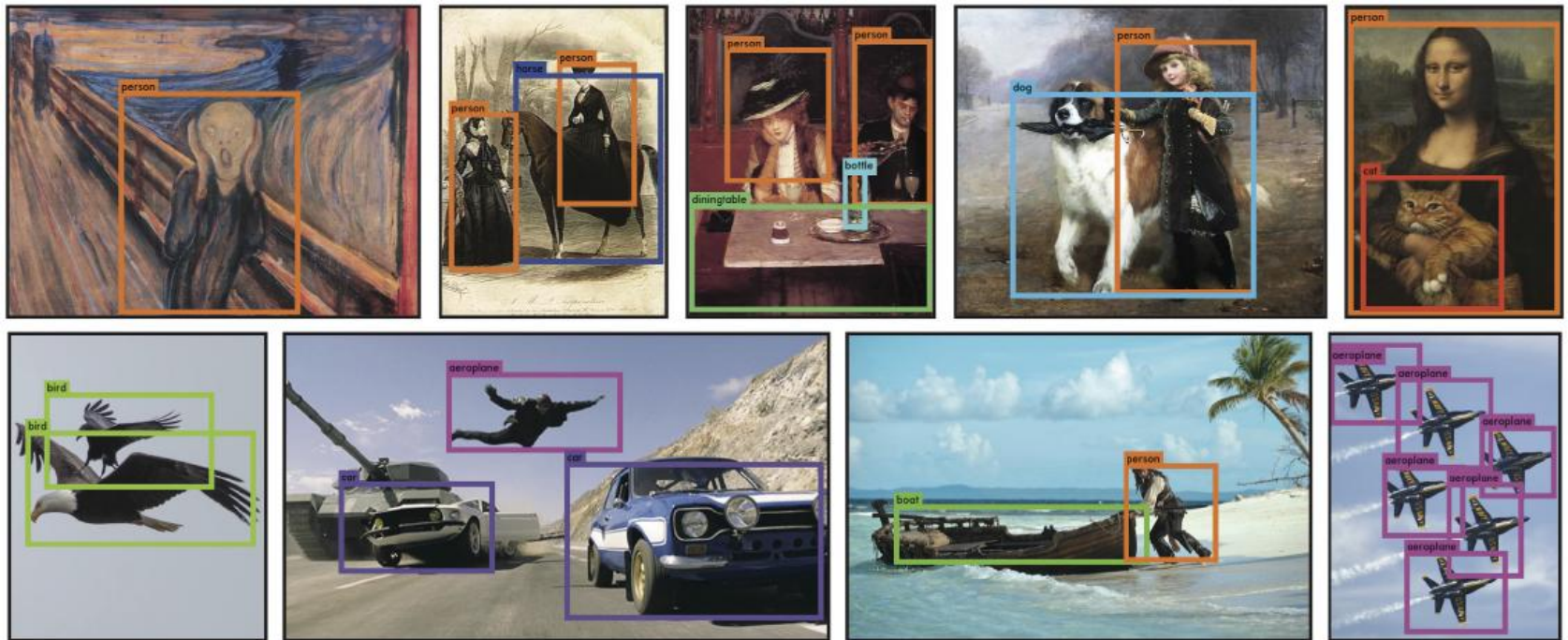
Prediction illustration



slide: T. Jeon



Experimental results



Outline

- Object detection
 - R-CNN
 - Fast R-CNN
 - Faster R-CNN
 - YOLO
 - FCOS
 - DETR
 - Deformable DETR



FCOS: Fully convolutional one-stage object detection

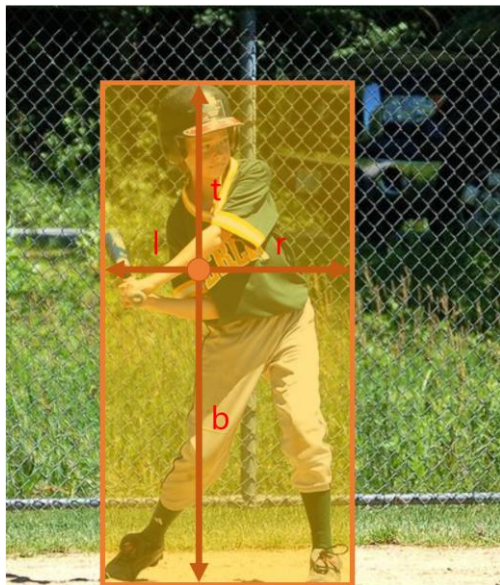
[Tian et al., ICCV'19]

- Most detectors rely on proposals or pre-defined anchors
- Drawbacks of using proposals and anchors
 - Time-consuming proposal generation
 - Sensitive to anchor box size, aspect ratio, and number
 - Difficult to handle objects with large shape variations
 - Require re-design on new detection tasks
- FCOS is anchor-free and proposal-free
 - Avoid complex computation related to anchor boxes
 - Avoid hyper-parameters related to anchor boxes
 - Achieve improved detection accuracy with a simpler framework

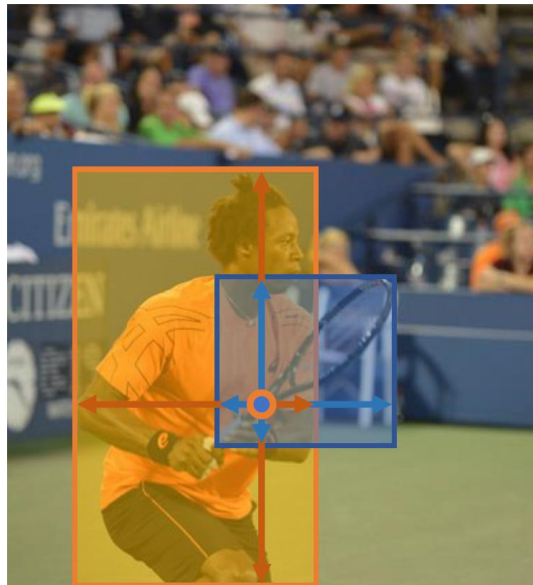


Three key techniques

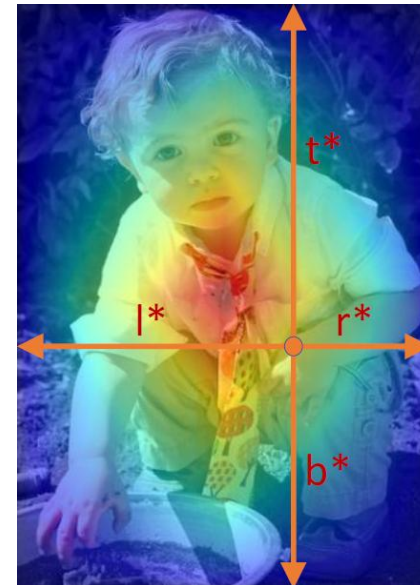
- Reformulate object detection in a **per-pixel prediction** fashion
- Uses **multi-level prediction** to resolve ambiguity
- Introduce **center-ness** to suppress low-quality detections



per-pixel prediction



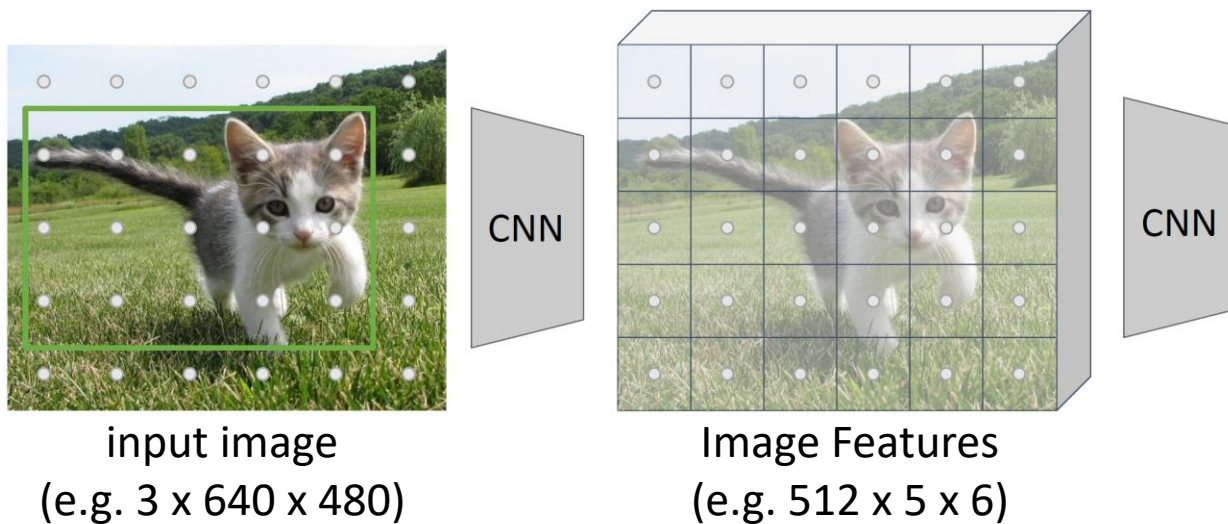
multi-level prediction



center-ness

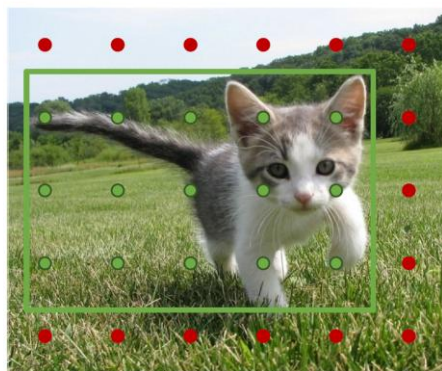
Per-pixel prediction

- Given an image with annotated ground-truth bounding boxes as input, CNN is applied for feature extraction



Per-pixel prediction

- Given an image with annotated ground-truth bounding boxes as input, CNN is applied for feature extraction
 - Classify points as **positive** if they fall into a GT box, otherwise **negative**



input image
(e.g. 3 x 640 x 480)

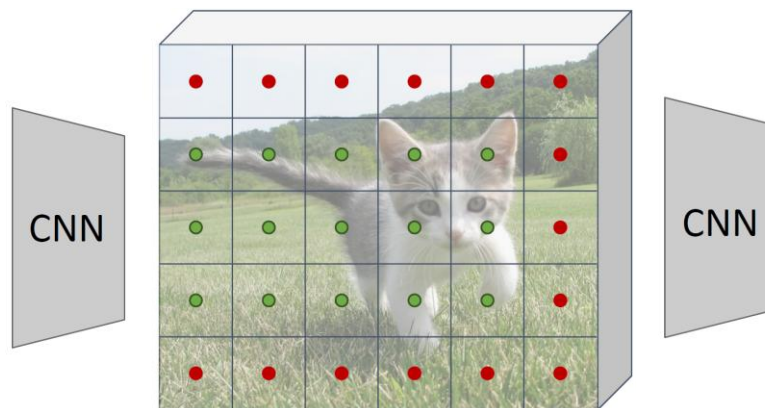
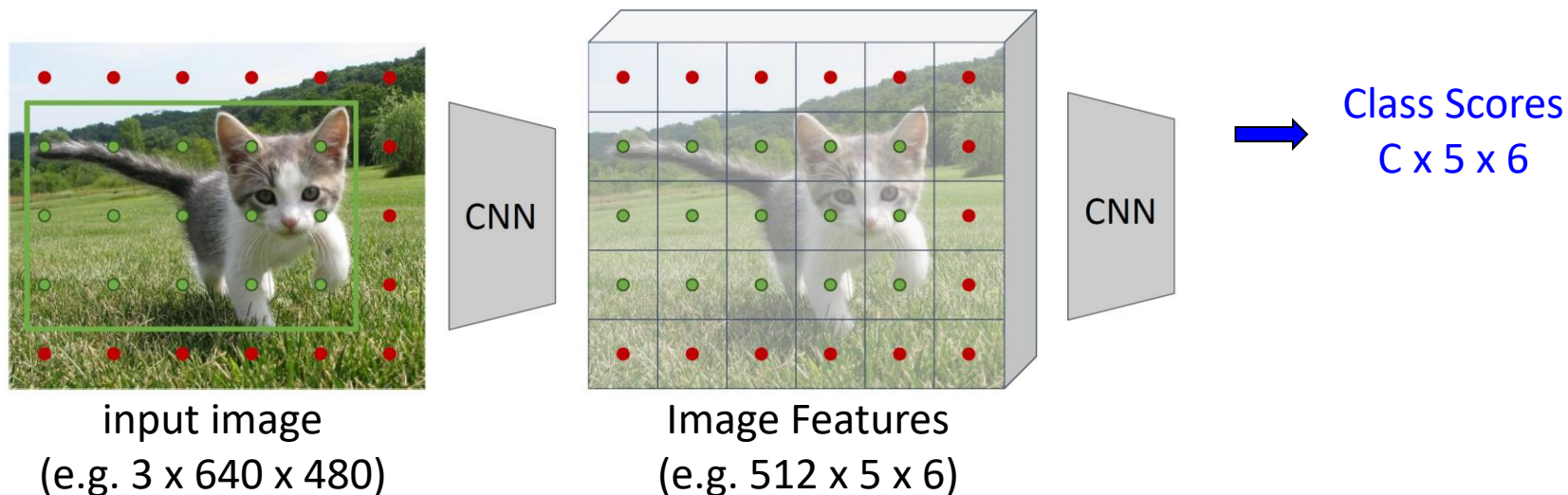


Image Features
(e.g. 512 x 5 x 6)

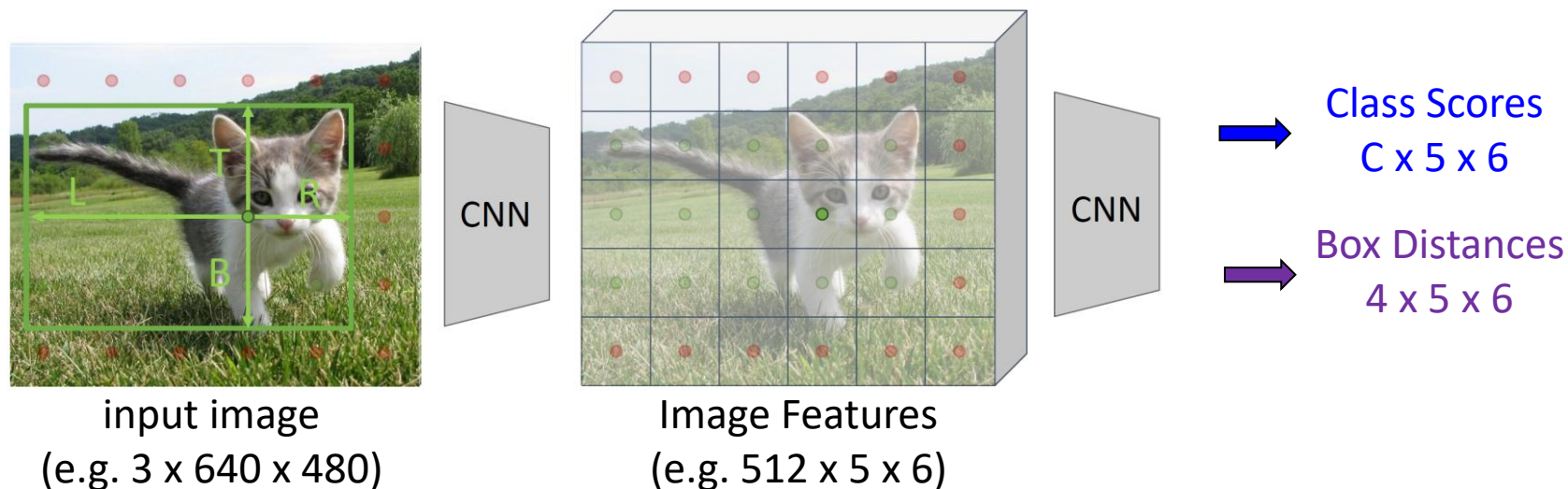
Per-pixel prediction

- Given an image with annotated ground-truth bounding boxes as input, CNN is applied for feature extraction
 - Classify points as **positive** if they fall into a GT box, otherwise **negative**
 - Train independent **per-class logistic regressors**



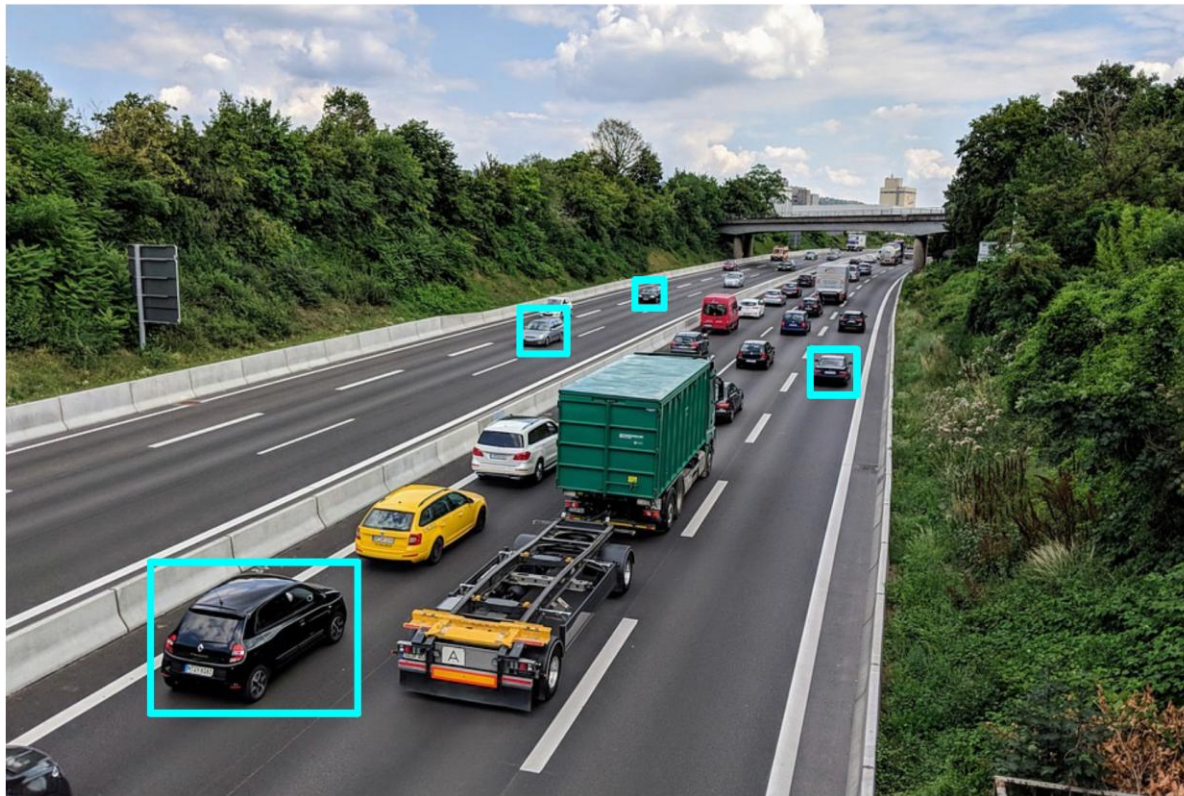
Per-pixel prediction

- Given an image with annotated ground-truth bounding boxes as input, CNN is applied for feature extraction
 - Classify points as **positive** if they fall into a GT box, otherwise **negative**
 - Train independent **per-class logistic regressors**
 - For positive points, regress **its distances to the four sides**



Multi-level prediction: Multi-scale object detection

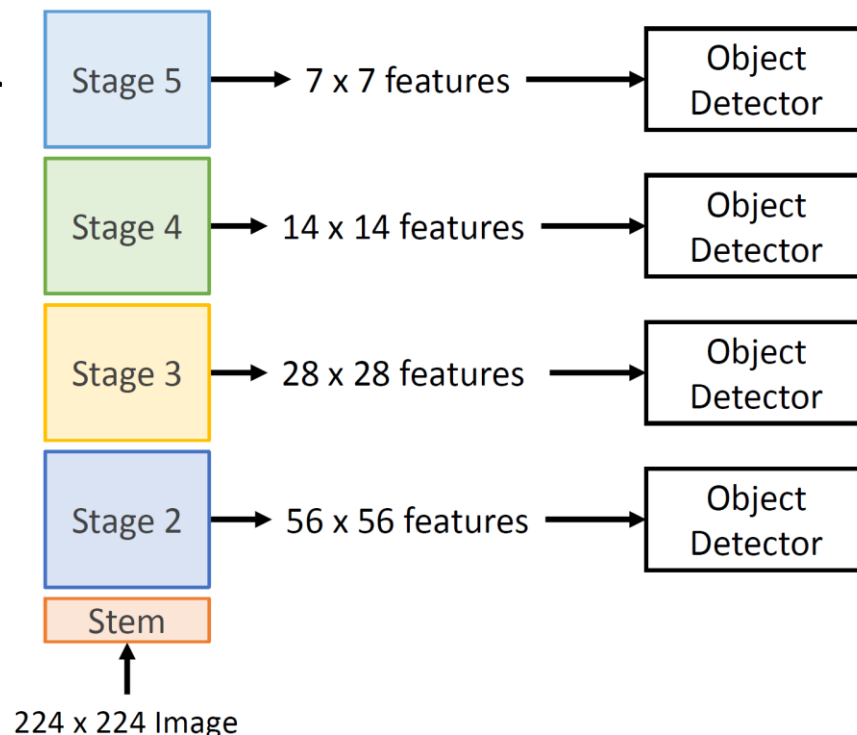
- We need to detect objects of many different scales. How can we improve the **scale invariance** of the detector?



Prior work: Feature pyramid network

[Lin et al., ICCV'17]

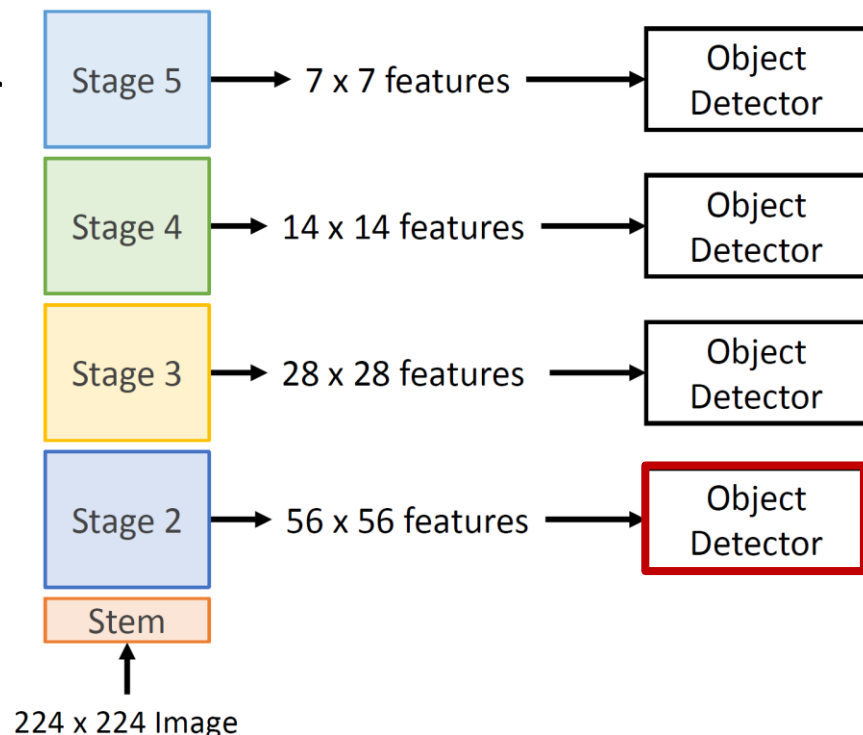
- CNNs have multiple stages that operate at different resolutions
- Attach an independent detector to the features at each level



Prior work: Feature pyramid network

[Lin et al., ICCV'17]

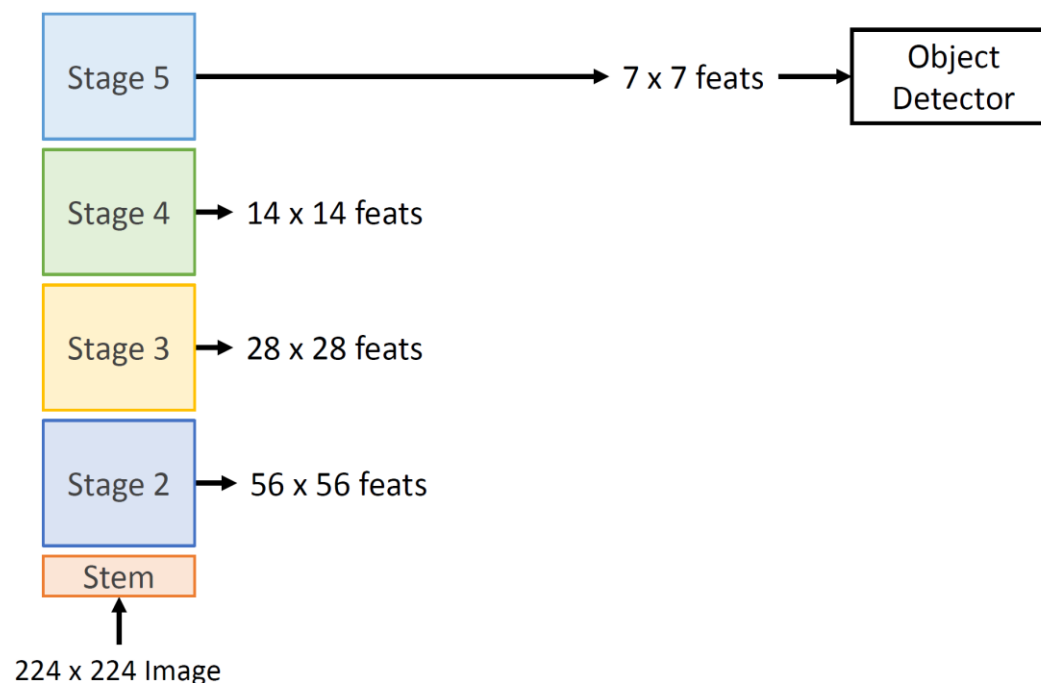
- CNNs have multiple stages that operate at different resolutions.
- Attach an independent detector to the features at each level.
- **Problem:** The detector on early features doesn't make use of the entire backbone;
 - It doesn't get access to high-level features



Prior work: Feature pyramid network

[Lin et al., ICCV'17]

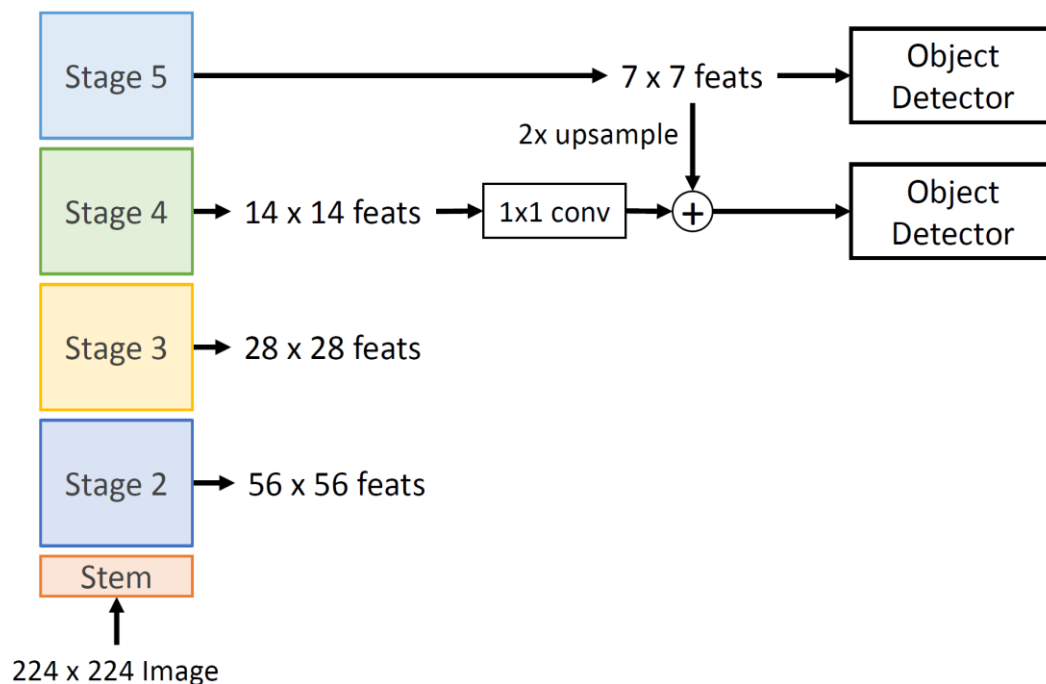
- Conventional CNN performs **bottom-up** feature extraction, from low-level features to high-level ones



Prior work: Feature pyramid network

[Lin et al., ICCV'17]

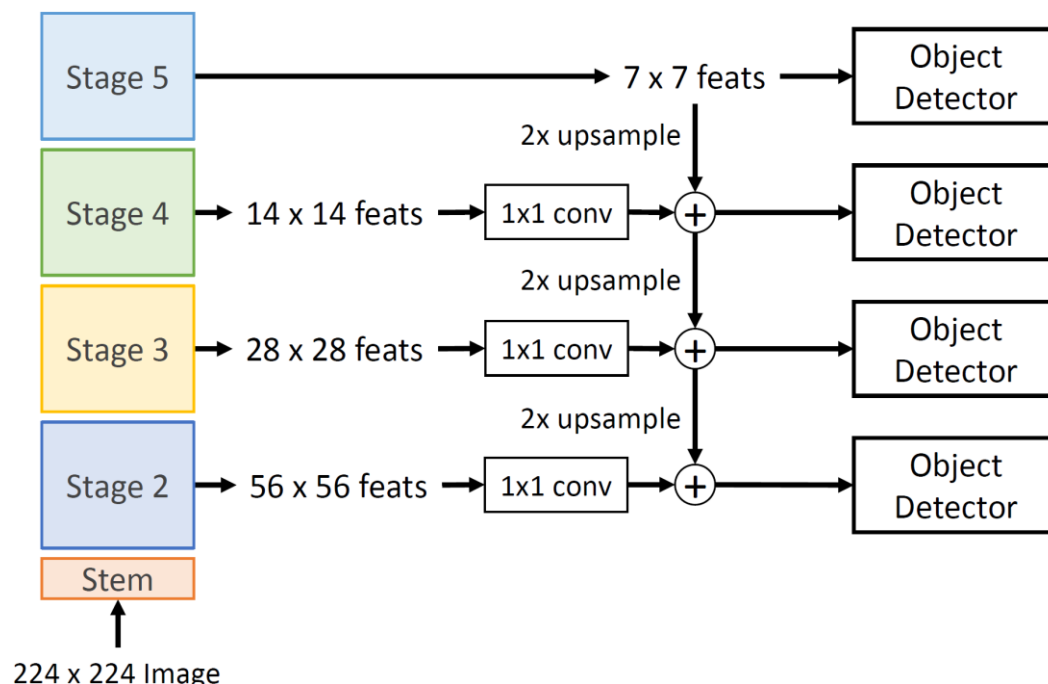
- Conventional CNN performs **bottom-up** feature extraction, from low-level features to high-level ones
- In addition to the bottom-up branch, the **feature pyramid network (FPN)** adds **top-down connections** that feed information from high-level features down to low-level ones



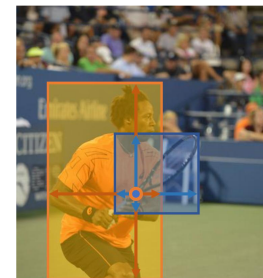
Prior work: Feature pyramid network

[Lin et al., ICCV'17]

- Conventional CNN performs **bottom-up** feature extraction, from low-level features to high-level ones
- In addition to the bottom-up branch, the **feature pyramid network (FPN)** adds **top-down connections** that feed information from high-level features down to low-level ones



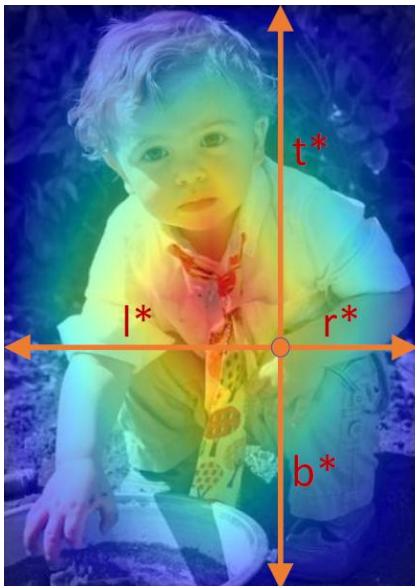
Multi-level prediction: Resolve ambiguity



- Ambiguity: A pixel may belong to multiple bounding boxes
- Idea:
 - 1. Build a feature pyramid
 - 2. Each feature level deals with objects of a particular size range
- FCOS utilizes FPN to generate multi-scale feature maps of five levels, $P_3 \sim P_7$
 - Objects with different sizes are assigned to different feature levels
 - $m_2, m_3, m_4, m_5, m_6, m_7$ are set to 0, 64, 128, 256, 512, and ∞
 - Scale P_i deals with objects of sizes between m_i and m_{i-1}

Center-ness

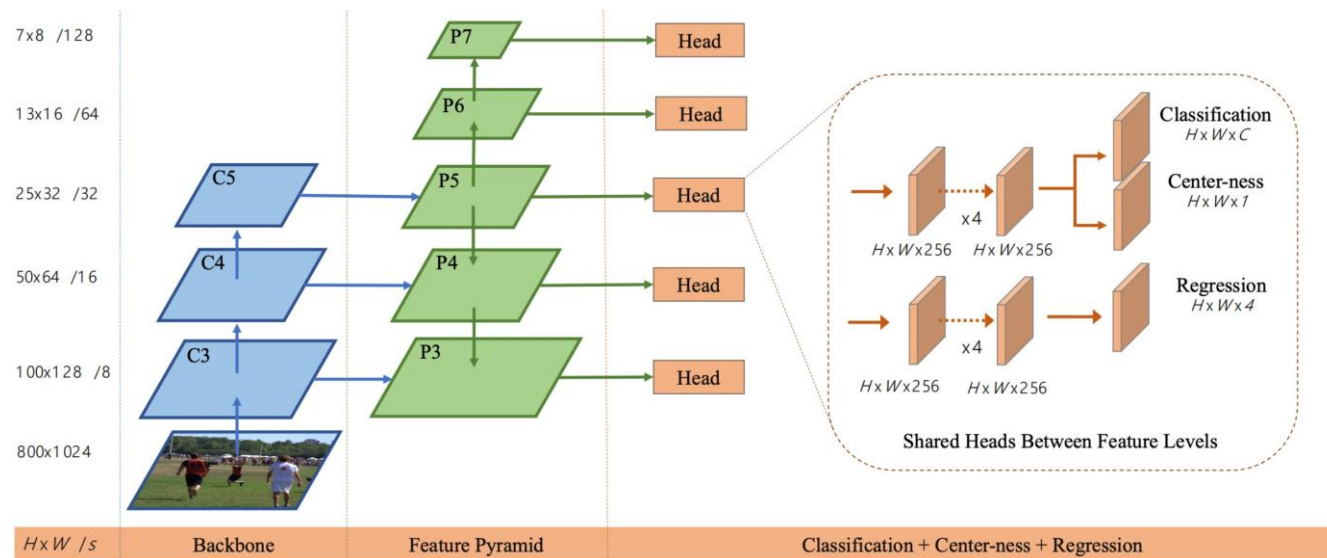
- Observation: pixels far away from object centers tend to yield low-quality predictions
- Center-ness: Assign a higher weight to a pixel closer to the center
- Add a network branch for center-ness prediction



$$\text{centerness}^* = \sqrt{\frac{\min(l^*, r^*)}{\max(l^*, r^*)} \times \frac{\min(t^*, b^*)}{\max(t^*, b^*)}}$$

Combining the three key techniques

- Network architecture of FCOS



- Inference

- Given an input image, we forward it through the network and obtain the classification scores and the regression prediction
- Choose positive sample and recover its bounding box

Experimental results: Comparisons with SoTAs

- Comparisons with SoTAs

Method	Backbone	AP	AP ₅₀	AP ₇₅	AP _S	AP _M	AP _L
Two-stage methods:							
Faster R-CNN w/ FPN [14]	ResNet-101-FPN	36.2	59.1	39.0	18.2	39.0	48.2
Faster R-CNN by G-RMI [11]	Inception-ResNet-v2 [27]	34.7	55.5	36.7	13.5	38.1	52.0
Faster R-CNN w/ TDM [25]	Inception-ResNet-v2-TDM	36.8	57.7	39.2	16.2	39.8	52.1
One-stage methods:							
YOLOv2 [22]	DarkNet-19 [22]	21.6	44.0	19.2	5.0	22.4	35.5
SSD513 [18]	ResNet-101-SSD	31.2	50.4	33.3	10.2	34.5	49.8
DSSD513 [5]	ResNet-101-DSSD	33.2	53.3	35.2	13.0	35.4	51.1
RetinaNet [15]	ResNet-101-FPN	39.1	59.1	42.3	21.8	42.7	50.2
CornerNet [13]	Hourglass-104	40.5	56.5	43.1	19.4	42.7	53.9
FSAF [34]	ResNeXt-64x4d-101-FPN	42.9	63.8	46.3	26.6	46.2	52.7
FCOS	ResNet-101-FPN	41.5	60.7	45.0	24.4	44.8	51.6
FCOS	HRNet-W32-5l [26]	42.0	60.4	45.3	25.4	45.0	51.0
FCOS	ResNeXt-32x8d-101-FPN	42.7	62.2	46.1	26.0	45.6	52.6
FCOS	ResNeXt-64x4d-101-FPN	43.2	62.8	46.6	26.5	46.2	53.3
FCOS w/ improvements	ResNeXt-64x4d-101-FPN	44.7	64.1	48.4	27.6	47.5	55.6

- AP₅₀: average precision when the IoU threshold is set to 50%
- AP_{S/M/L}: average precision for small/medium/large objects



Experimental results: Ablation studies

- Ablation study on **multi-level prediction**

w/ FPN	Amb. samples (%)	Amb. samples (diff.) (%)
	23.16	17.84
✓	7.14	3.75

- Amb. samples: ratio of ambiguous samples to all positive samples
- Amb. Samples (diff.): Similar but excludes those ambiguous samples belonging to the same category

- Ablation study on **center-ness**

	AP	AP ₅₀	AP ₇₅	AP _S	AP _M	AP _L
None	33.5	52.6	35.2	20.8	38.5	42.6
center-ness [†]	33.5	52.4	35.1	20.8	37.8	42.8
center-ness	37.1	55.9	39.8	21.3	41.0	47.8

- center-ness[†]: the center-ness computed from the regression branch
- center-ness : the center-ness predicted from the center-ness branch



Experimental results: Visualization



Outline

- Object detection
 - R-CNN
 - Fast R-CNN
 - Faster R-CNN
 - YOLO
 - FCOS
 - DETR
 - Deformable DETR



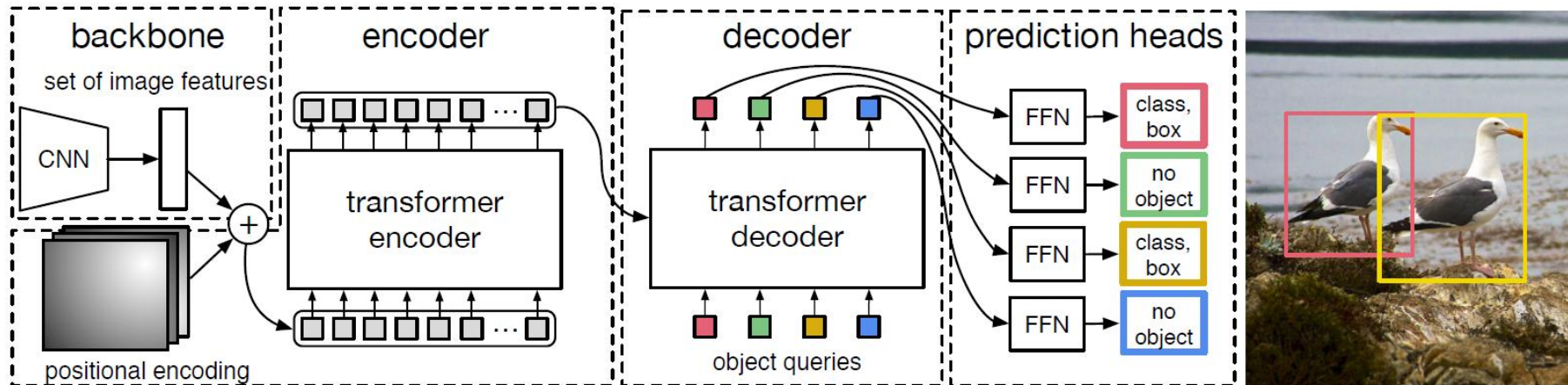
DETR (DEtection TRansformer)

[Carion et al., ECCV'20]

- DETR formulates object detection as a **set prediction** problem
 - It removes many hand-designed components
 - No need of proposal, anchor, or non maximum suppression
- DETR has a **transformer encoder-decoder architecture** with a **set-based loss** forcing unique predictions via bipartite matching
- It achieves comparable performance with Faster R-CNN in accuracy and efficiency
- It can be generalized to panoptic segmentation

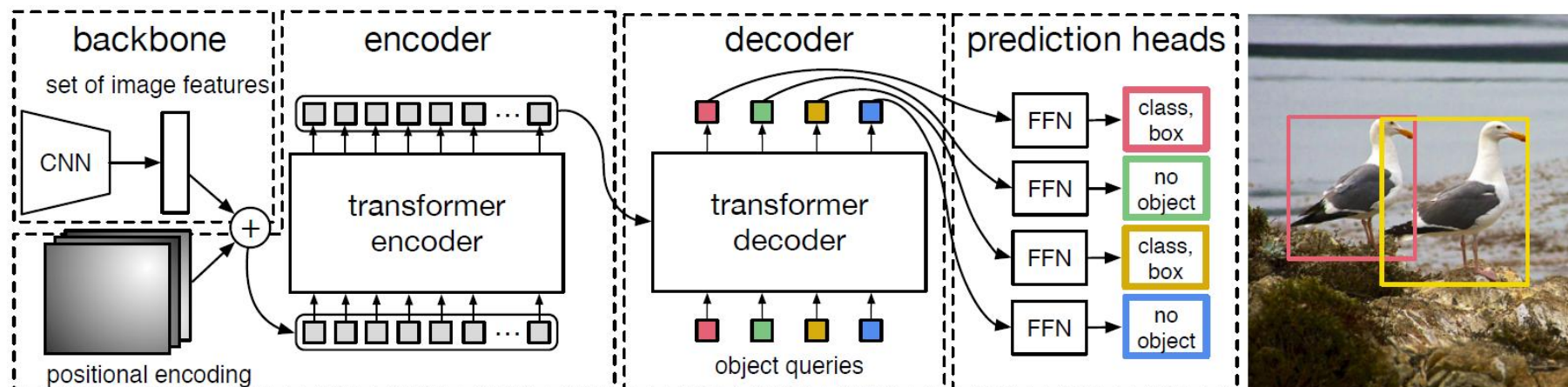


Network architecture



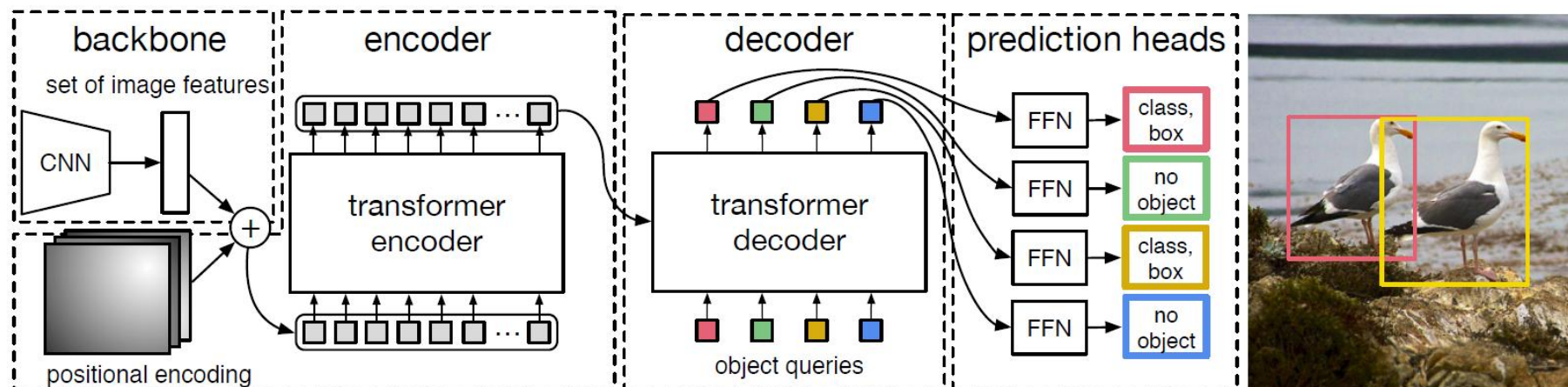
- An end-to-end trainable architecture
- CNN for feature extraction
- A transformer with an encoder and a decoder
- The transformer for making N predictions
- FFN (feed-forward network) for predicting class and box
- A set-loss serves as the objective for network optimization

Network architecture: Backbone



- Backbone: Extract a compact feature representation
- Input image: $x_{\text{img}} \in \mathbb{R}^{3 \times H_0 \times W_0}$
- Output feature maps: $f \in \mathbb{R}^{C \times H \times W}$
where $C = 2048$ and $H, W = \frac{H_0}{32}, \frac{W_0}{32}$
- Fixed positional encodings

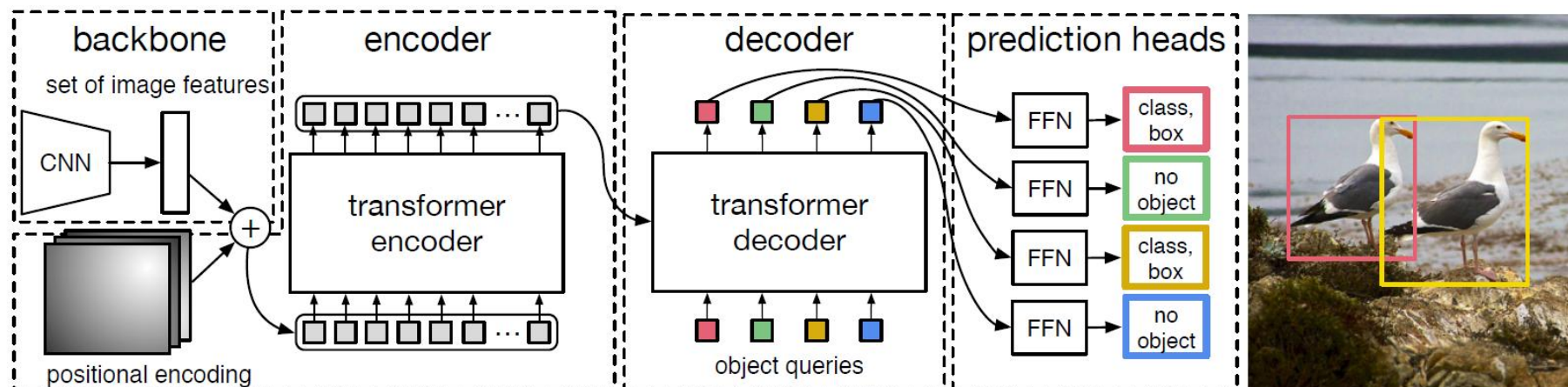
Network architecture: Transformer encoder



- Transformer encoder

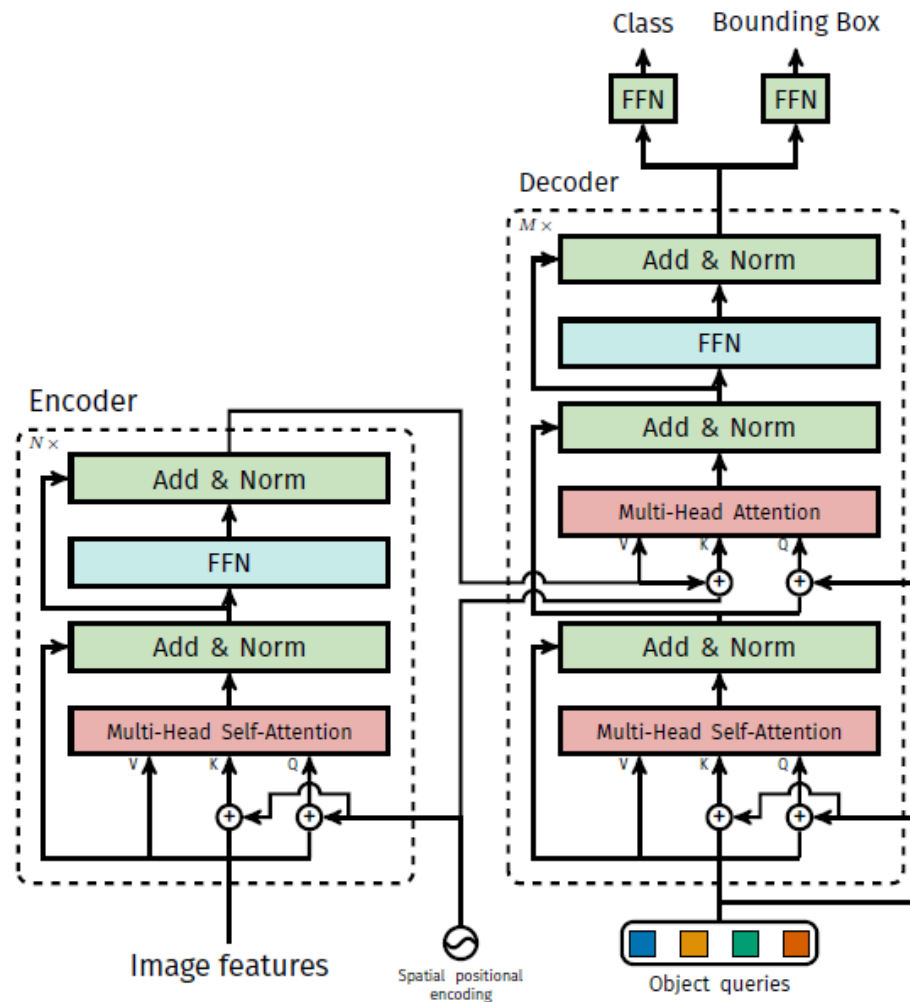
- Apply 1x1 convolution to reduce dimension from C to d
- Collapse spatial dimension to yield a one-dimensional feature map of size $d \times HW$
- Each encoder layer consists of a multi-head self-attention module and a feed-forward network

Network architecture: Transformer decoder

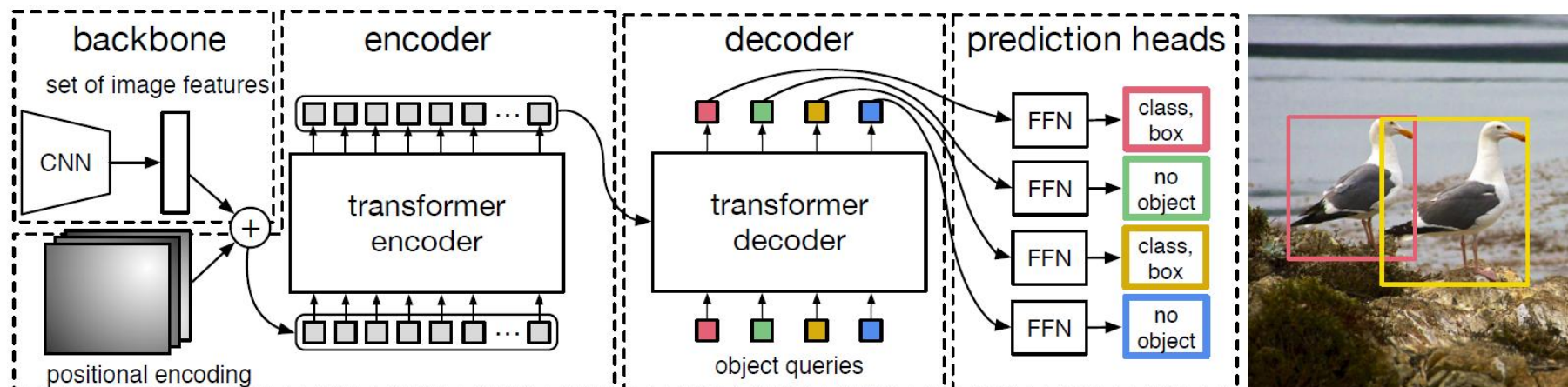


- Transformer decoder
 - N object queries of dimension d . N is larger than the numbers of objects in most images, say 50
 - Each decoder layer consist of
 - 1. Multi-head self-attention module
 - 2. Multi-head encoder-decoder attention module
 - 3. Feed forward network

Transformer in DETR



Network architecture: Prediction heads and loss



- Feed forward network (FFN)
 - 3-layer perceptron with ReLU activation function and a linear projection layer
 - It predicts **the class** using softmax and **the normalized center coordinates, height, and width of the box**
 - Classes: object classes and background $\emptyset$
 - N prediction sets, each of which is form of $y_i = (c_i, b_i)$
- c_i is class label, while $b_i \in [0, 1]^4$ is box prediction

Objective function for network optimization

- N predictions, larger than the typical number of objects in an image
- Ground truth: $y = \{y_i\}_{i=1}^N$; Predictions by DETR: $\hat{y} = \{\hat{y}_i\}_{i=1}^N$
- Bipartite matching between these two sets

$$\hat{\sigma} = \arg \min_{\sigma \in \mathfrak{S}_N} \sum_i^N \mathcal{L}_{\text{match}}(y_i, \hat{y}_{\sigma(i)})$$

➤ where $\sigma \in \mathfrak{S}_N$ is a permutation of N elements

- The pair-wise matching cost between y_i and $\hat{y}_{\sigma(i)}$ is

$$-\mathbb{1}_{\{c_i \neq \emptyset\}} \hat{p}_{\sigma(i)}(c_i) + \mathbb{1}_{\{c_i \neq \emptyset\}} \mathcal{L}_{\text{box}}(b_i, \hat{b}_{\sigma(i)})$$

- Objective function of DETR

$$\mathcal{L}_{\text{Hungarian}}(y, \hat{y}) = \sum_{i=1}^N \left[-\log \hat{p}_{\hat{\sigma}(i)}(c_i) + \mathbb{1}_{\{c_i \neq \emptyset\}} \mathcal{L}_{\text{box}}(b_i, \hat{b}_{\hat{\sigma}(i)}) \right]$$



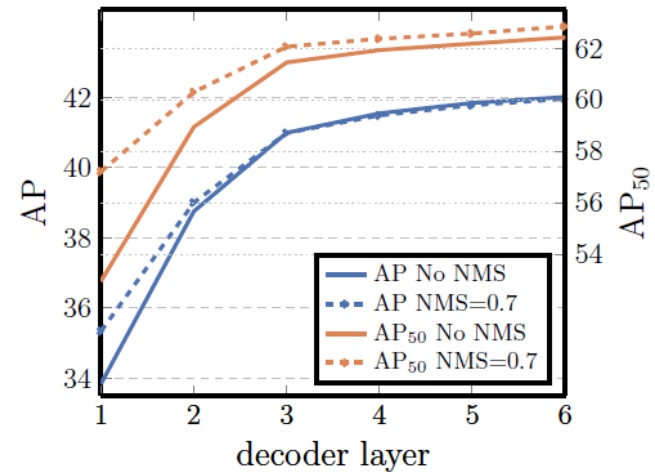
Experimental results on object detection

- COCO 2017 detection dataset

Model	GFLOPS/FPS	#params	AP	AP ₅₀	AP ₇₅	AP _S	AP _M	AP _L
RetinaNet	205/18	38M	38.7	58.0	41.5	23.3	42.3	50.3
Faster RCNN-DC5	320/16	166M	39.0	60.5	42.3	21.4	43.5	52.5
Faster RCNN-FPN	180/26	42M	40.2	61.0	43.8	24.2	43.5	52.0
Faster RCNN-R101-FPN	246/20	60M	42.0	62.5	45.9	25.2	45.6	54.6
RetinaNet+	205/18	38M	41.1	60.4	43.7	25.6	44.8	53.6
Faster RCNN-DC5+	320/16	166M	41.1	61.4	44.3	22.9	45.9	55.0
Faster RCNN-FPN+	180/26	42M	42.0	62.1	45.5	26.6	45.4	53.4
Faster RCNN-R101-FPN+	246/20	60M	44.0	63.9	47.8	27.2	48.1	56.0
DETR	86/28	41M	42.0	62.4	44.2	20.5	45.8	61.1
DETR-DC5	187/12	41M	43.3	63.1	45.9	22.5	47.3	61.1
DETR-R101	152/20	60M	43.5	63.8	46.4	21.9	48.0	61.8
DETR-DC5-R101	253/10	60M	44.9	64.7	47.7	23.7	49.5	62.3



Ablation studies



with and without NMS

spatial pos. enc. encoder	enc.	decoder	output pos. enc. decoder	AP	Δ	AP ₅₀	Δ
none	none	learned at input	learned at input	32.8	-7.8	55.2	-6.5
sine at input	sine at input	learned at input	learned at input	39.2	-1.4	60.0	-1.6
learned at attn.	learned at attn.	learned at attn.	learned at attn.	39.6	-1.0	60.7	-0.9
none	sine at attn.	learned at attn.	learned at attn.	39.3	-1.3	60.3	-1.4
sine at attn.	sine at attn.	learned at attn.	learned at attn.	40.6	-	61.6	-

different positional encodings

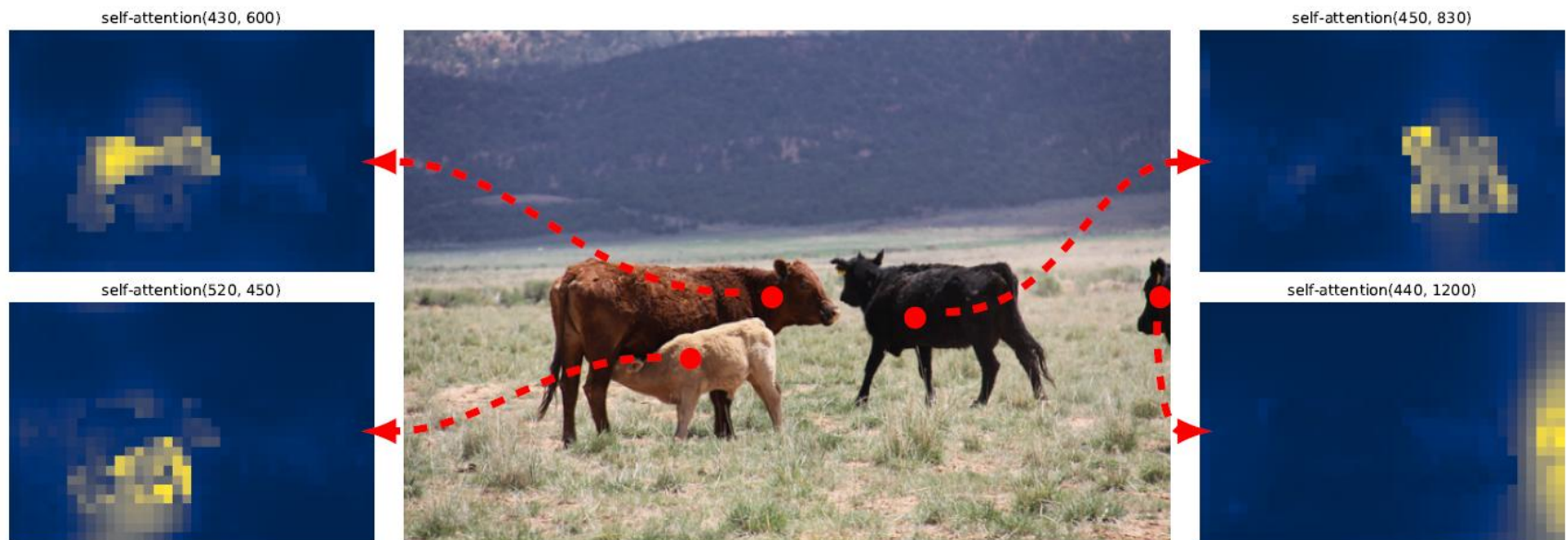
#layers	GFLOPS/FPS	#params	AP	AP ₅₀	AP _S	AP _M	AP _L
0	76/28	33.4M	36.7	57.4	16.8	39.6	54.2
3	81/25	37.4M	40.1	60.6	18.5	43.8	58.6
6	86/23	41.3M	40.6	61.6	19.9	44.3	60.2
12	95/20	49.2M	41.6	62.1	19.8	44.9	61.9

Effect of encoder size



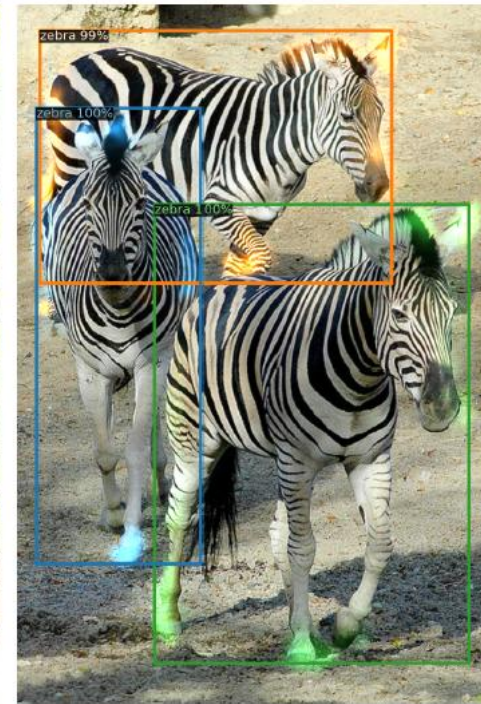
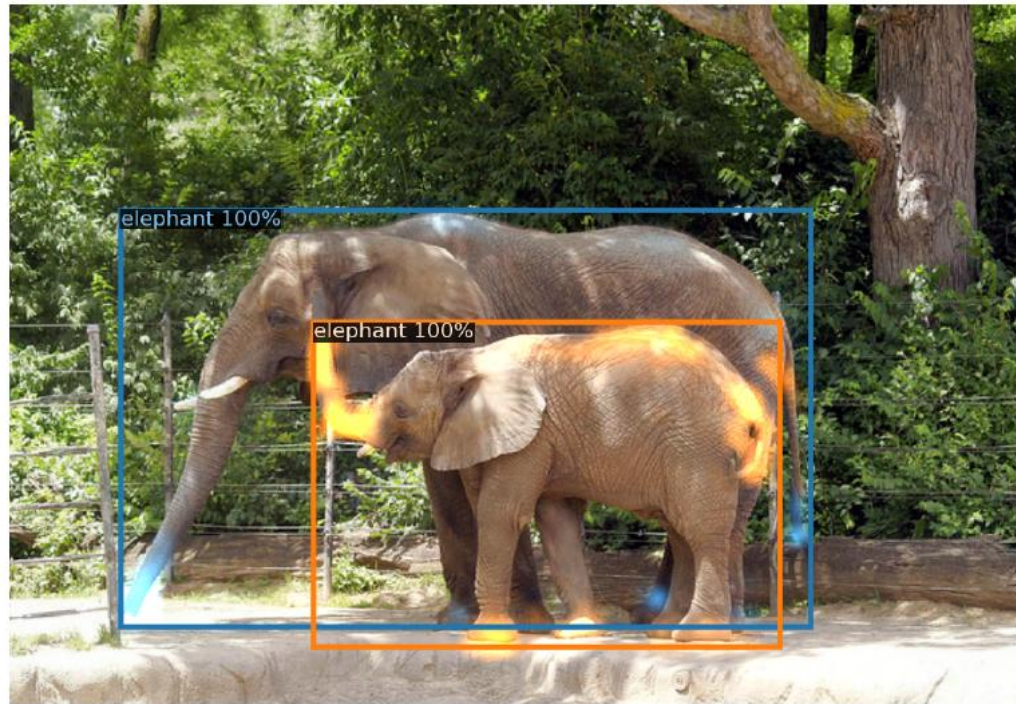
Encoder self-attention

- Self-attention on four different reference points
- The encoder is able to separate individual instances



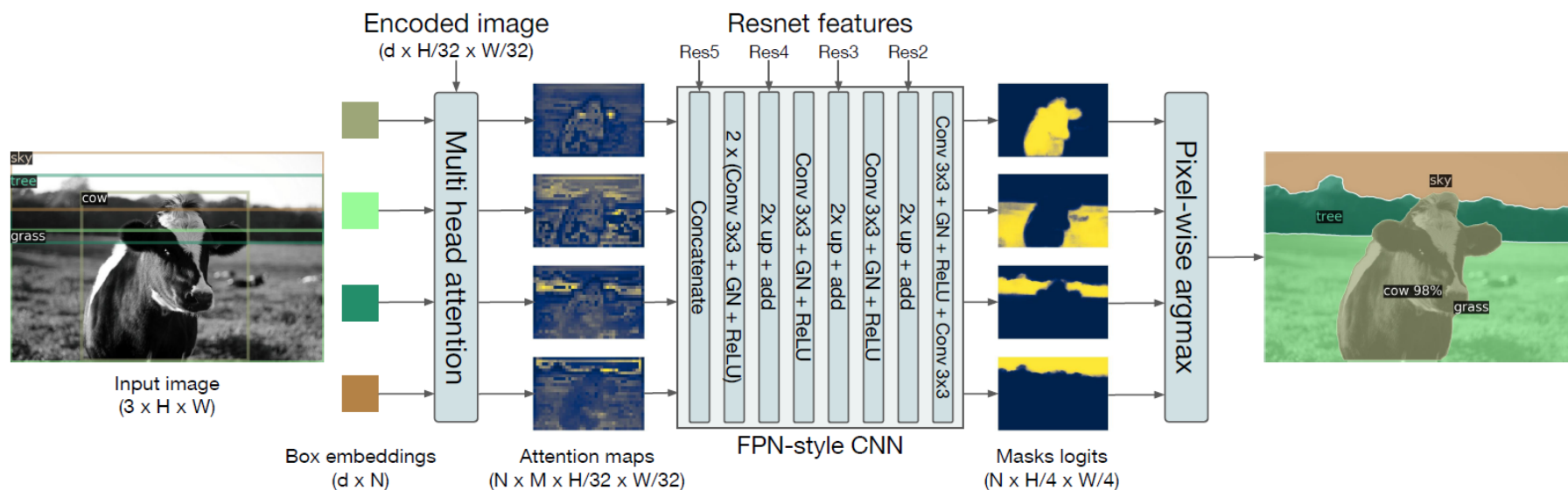
Decoder attention visualization

- Decoder attention visualization for each detected object
 - Decoder typically attends to object extremities



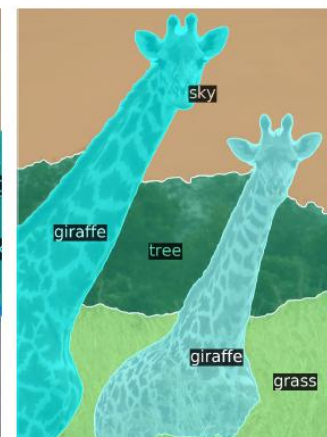
Extension to panoptic segmentation

- Add a segmentation branch to DETR



Panoptic segmentation results

Model	Backbone	PQ	SQ	RQ	PQ th	SQ th	RQ th	PQ st	SQ st	RQ st	AP
PanopticFPN++	R50	42.4	79.3	51.6	49.2	82.4	58.8	32.3	74.8	40.6	37.7
UPSnet	R50	42.5	78.0	52.5	48.6	79.4	59.6	33.4	75.9	41.7	34.3
UPSnet-M	R50	43.0	79.1	52.8	48.9	79.7	59.7	34.1	78.2	42.3	34.3
PanopticFPN++	R101	44.1	79.5	53.3	51.0	83.2	60.6	33.6	74.0	42.1	39.7
DETR	R50	43.4	79.3	53.8	48.2	79.8	59.5	36.3	78.5	45.3	31.1
DETR-DC5	R50	44.6	79.8	55.0	49.4	80.5	60.6	37.3	78.7	46.5	31.9
DETR	R101	45.1	79.9	55.5	50.5	80.9	61.7	37.0	78.5	46.0	33.0
DETR-DC5	R101	45.6	80.0	56.1	50.9	80.9	62.2	37.5	78.6	46.8	33.1



Concluding remarks

- DETR is a new object detector based on transformers
- DETR achieves comparable results to an optimized Faster R-CNN
- It can be extended to panoptic segmentation



Outline

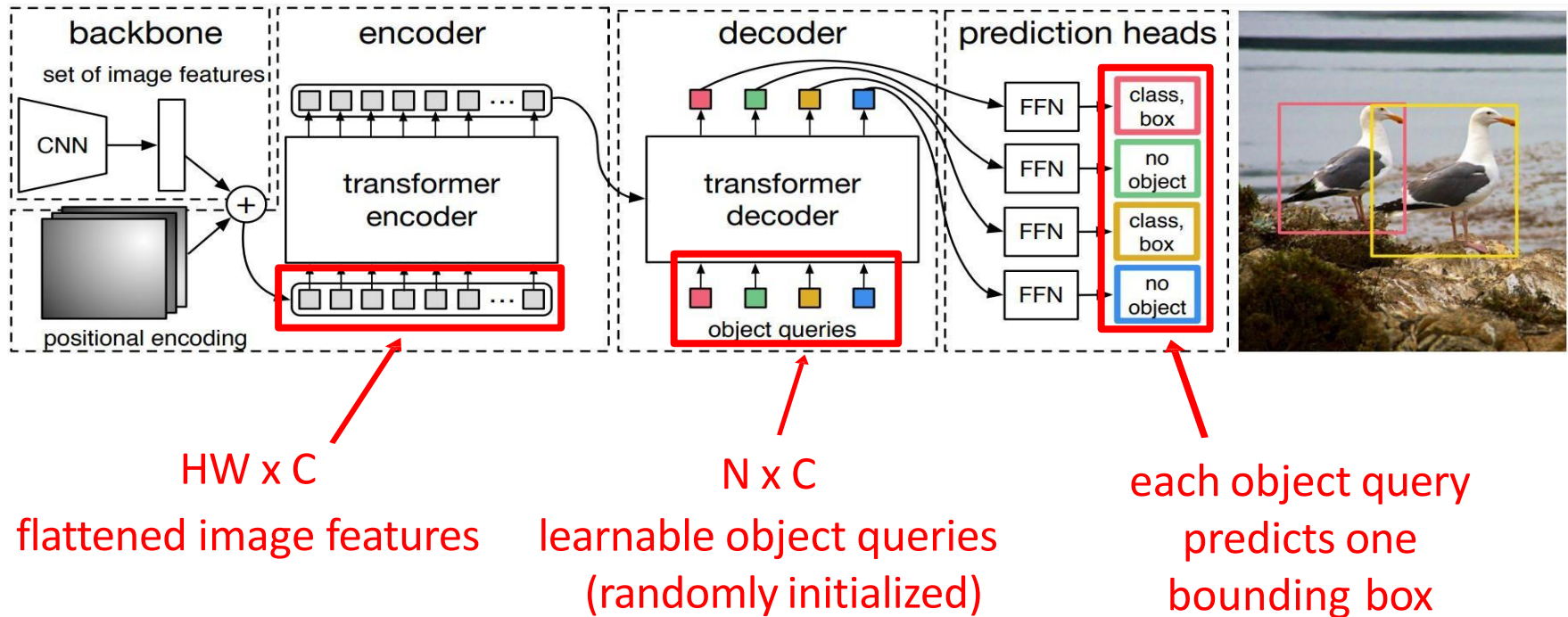
- Object detection
 - R-CNN
 - Fast R-CNN
 - Faster R-CNN
 - YOLO
 - FCOS
 - DETR
 - Deformable DETR



Deformable DETR: Deformable transformers for end-to-end object detection

[Zhu et al., ICLR'21]

- DETR architecture:

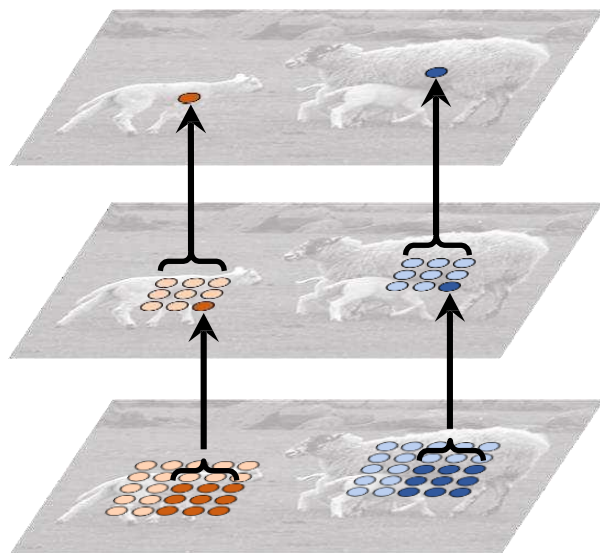


DETR

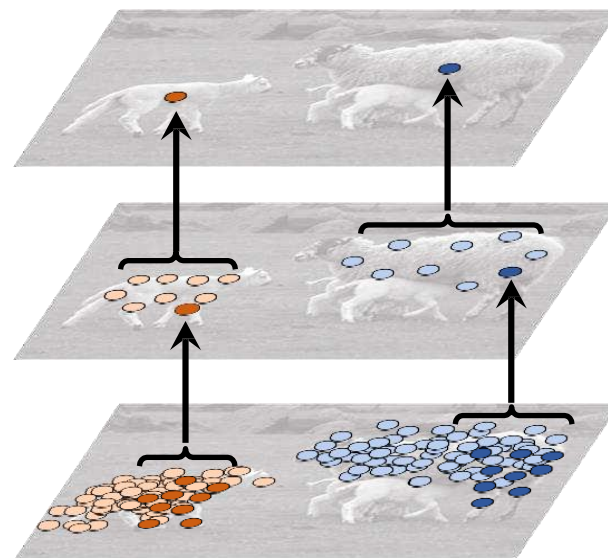
- Strengths
 - Eliminate the need for hand-crafted components, such as anchor size and NMS threshold
 - Achieve very competitive performance with previous modern object detectors
- Weaknesses
 - Slow convergence
 - ◆ 500 epochs to converge on the COCO dataset, 10x~20x slower than Faster R-CNN
 - Limited feature resolution
 - ◆ About 4.1% lower AP on small objects than Faster R-CNN+FPN
 - Both are caused by a large number of tokens



Deformable convolution



standard convolution



deformable convolution

Deformable convolution is effective and efficient in image recognition

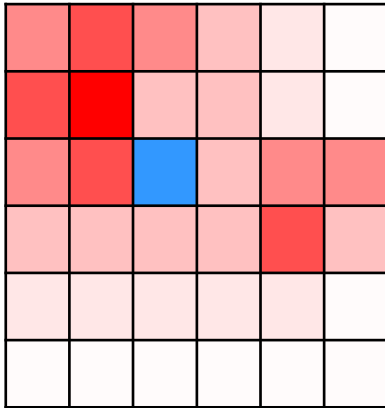
Dai, Jifeng, et al. "Deformable convolutional networks." In ICCV 2017.

Zhu, Xizhou, et al. "Deformable convnets v2: More deformable, better results." In CVPR 2019.

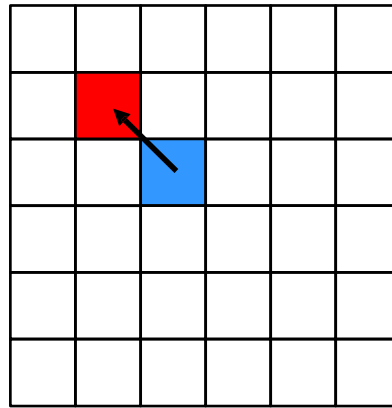


Self-attention + Deformable convolution?

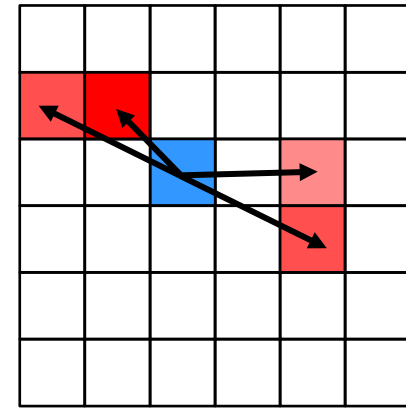
- Deformable attention: Self-attention + Deformable convolution



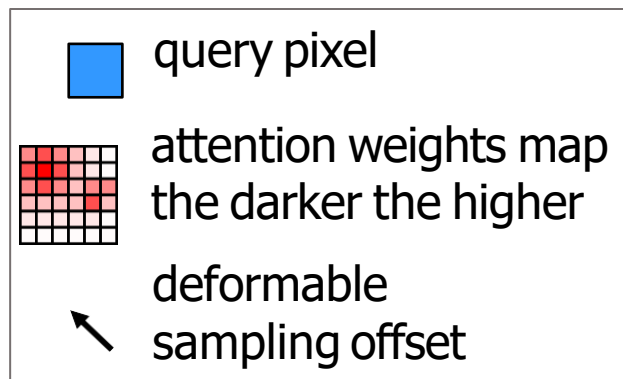
Self-Attention
inefficient



Deformable Convolution
lacks relation modeling

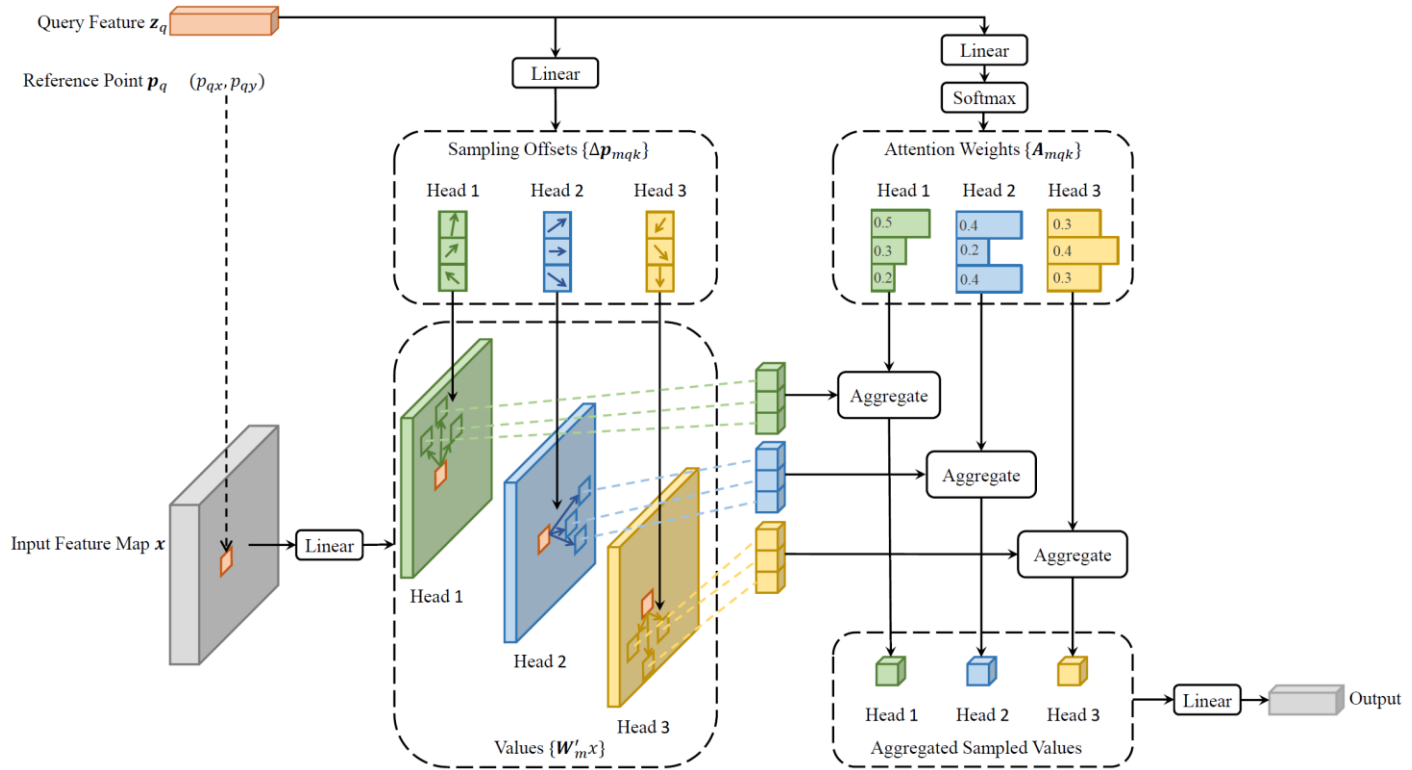


Deformable Attention



Deformable attention module

$$\text{DeformAttn}(z_q, p_q, x) = \sum_{m=1}^M \mathbf{W}_m \left[\sum_{k=1}^K A_{mqk} \cdot \mathbf{W}'_m x(p_q + \Delta p_{mqk}) \right]$$



Deformable attention module: equation

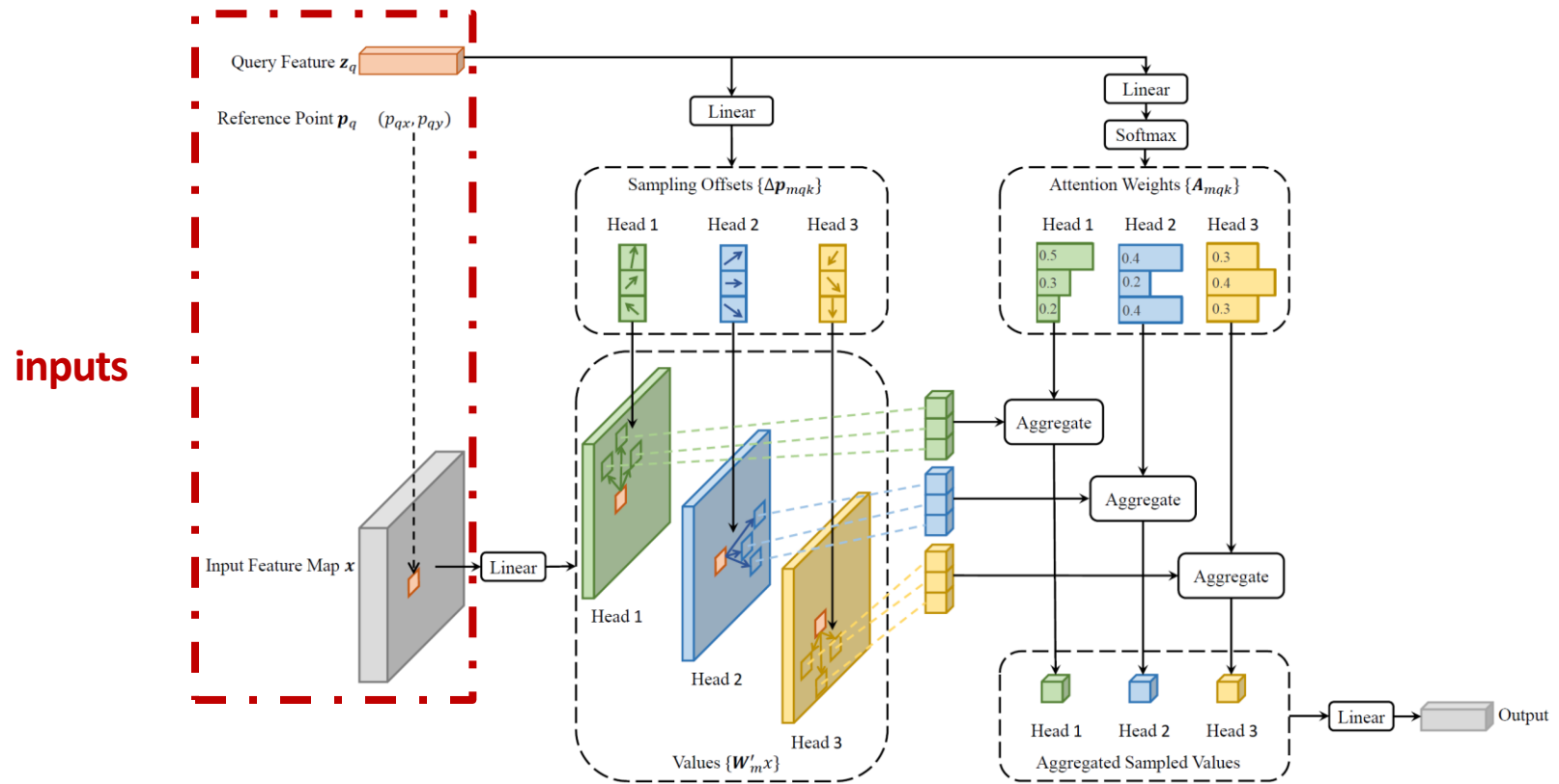
$$\text{DeformAttn}(z_q, p_q, x) = \sum_{m=1}^M W_m \left[\sum_{k=1}^K A_{mqk} \cdot W'_m x(p_q + \Delta p_{mqk}) \right]$$

Diagram illustrating the Deformable Attention Module equation and its components:

- query** (green text) points to z_q .
- position** (red text) points to p_q .
- feature map** (blue text) points to x .
- #heads** (purple text) points to M .
- #sampled points** (purple text) points to K .
- attention** (purple text) points to A_{mqk} .
- offset** (blue text) points to Δp_{mqk} .
- position of the sampled point** (red text) points to $p_q + \Delta p_{mqk}$.
- features of the sampled point** (green text) points to $x(p_q + \Delta p_{mqk})$.
- value** (purple text) points to $W'_m x(p_q + \Delta p_{mqk})$.

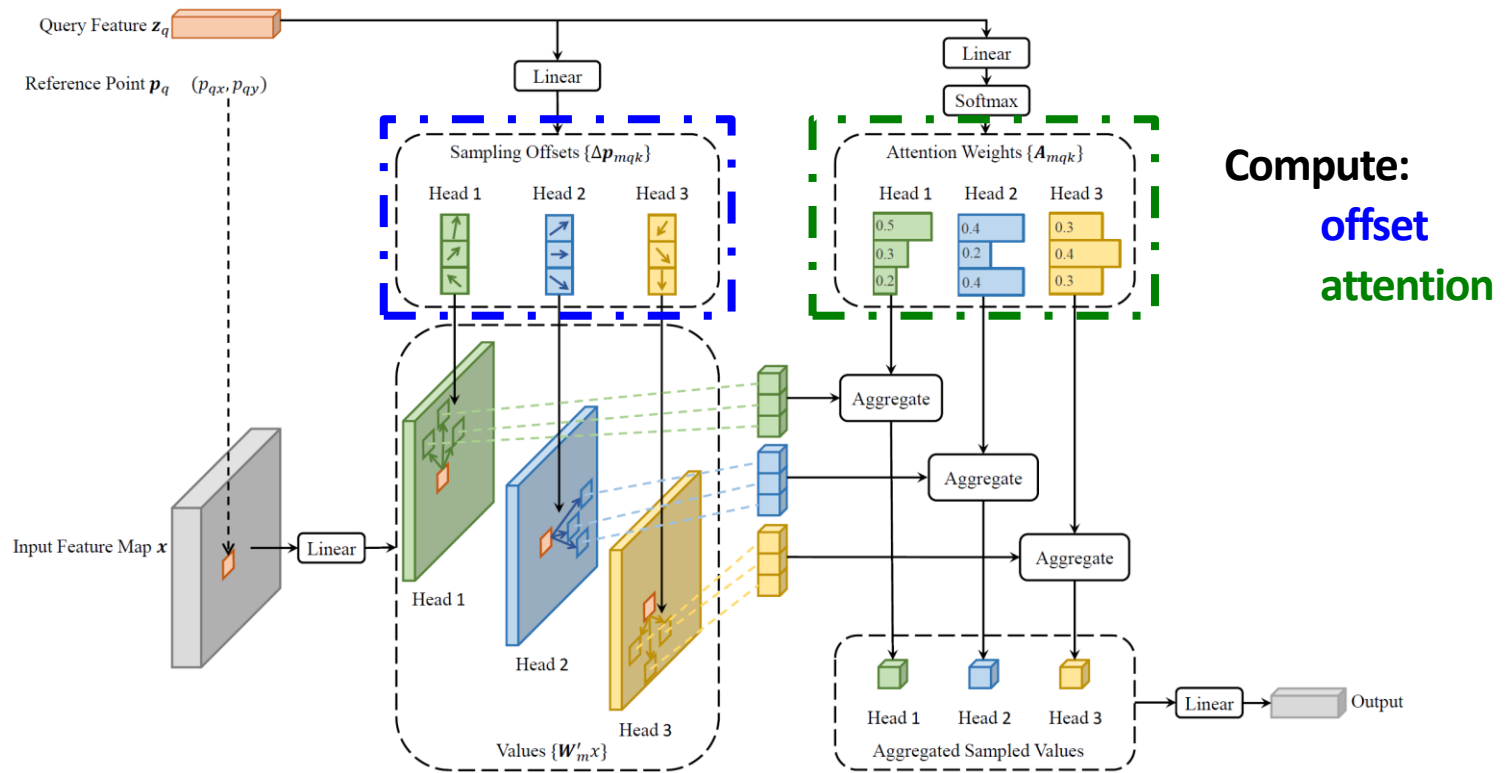
Deformable attention module

$$\text{DeformAttn}(\boxed{z_q, p_q, x}) = \sum_{m=1}^M \mathbf{W}_m \left[\sum_{k=1}^K A_{mqk} \cdot \mathbf{W}'_m \mathbf{x}(p_q + \Delta p_{mqk}) \right]$$



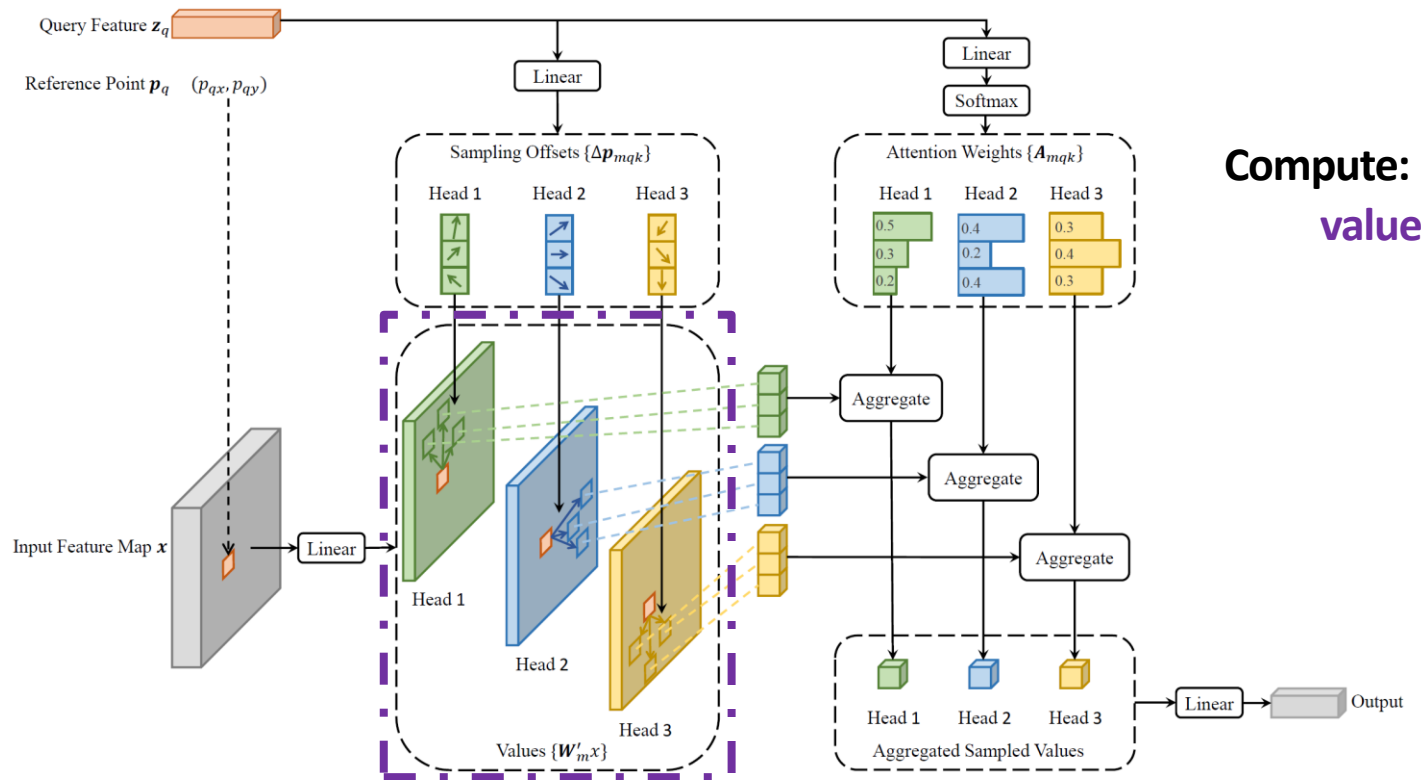
Deformable attention module

$$\text{DeformAttn}(z_q, p_q, x) = \sum_{m=1}^M W_m \left[\sum_{k=1}^K A_{mqk} \cdot W'_m x(p_q + \Delta p_{mqk}) \right]$$



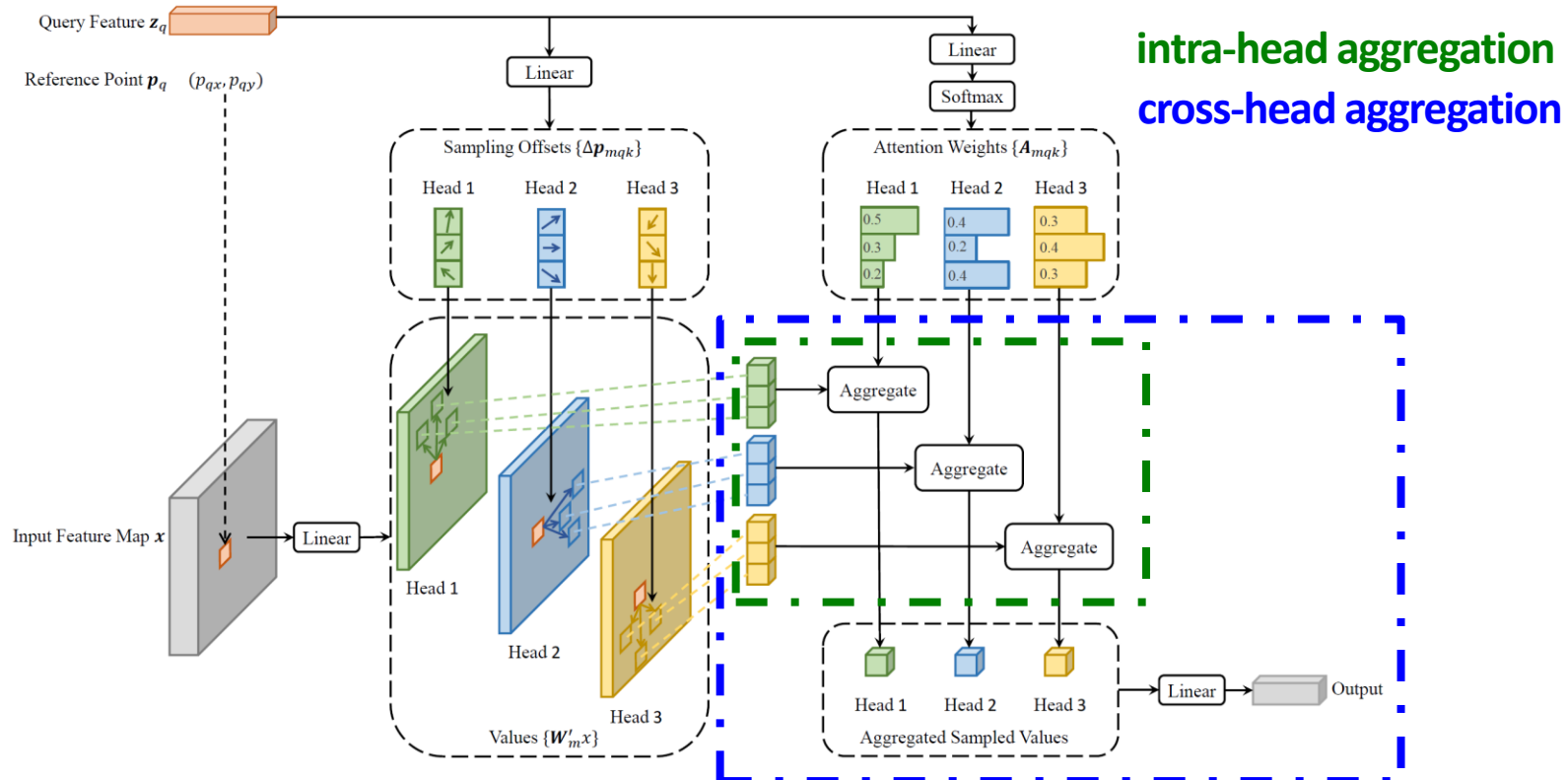
Deformable attention module

$$\text{DeformAttn}(z_q, p_q, x) = \sum_{m=1}^M \mathbf{W}_m \left[\sum_{k=1}^K A_{mqk} \cdot \mathbf{W}'_m x(p_q + \Delta p_{mqk}) \right]$$

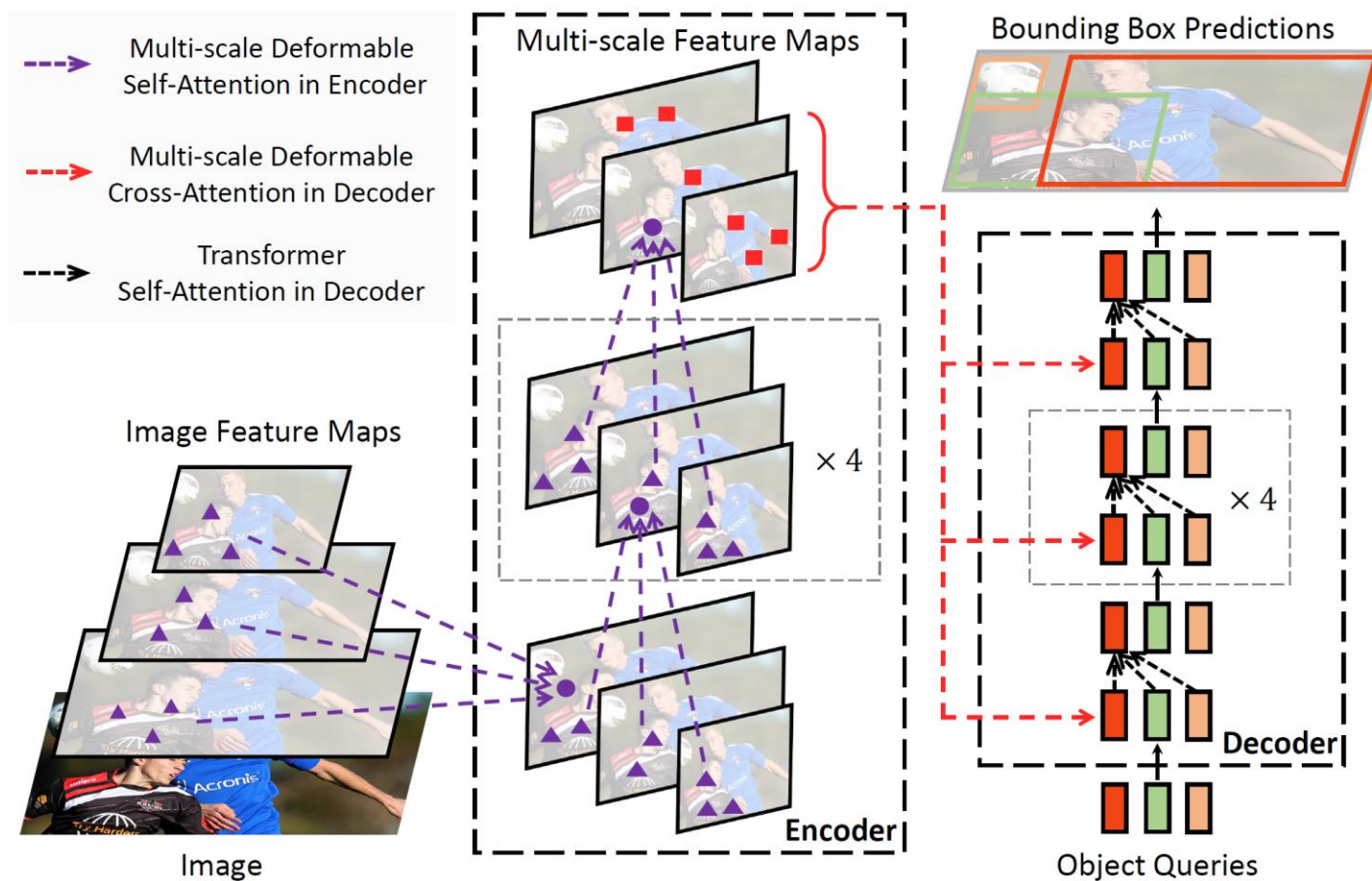


Deformable attention module

$$\text{DeformAttn}(z_q, p_q, x) = \sum_{m=1}^M W_m \left[\sum_{k=1}^K A_{mqk} \cdot W'_m x(p_q + \Delta p_{mqk}) \right]$$



Pipeline of Deformable DETR



Multi-scale extension

- Deformable attention

$$\text{DeformAttn}(z_q, p_q, \boxed{\mathbf{x}}) = \sum_{m=1}^M \mathbf{W}_m \left[\sum_{k=1}^K A_{mqk} \cdot \mathbf{W}'_m \mathbf{x}(p_q + \Delta \mathbf{p}_{mqk}) \right]$$

- Multi-scale deformable attention

$$\text{MSDeformAttn}(z_q, \hat{p}_q, \boxed{\{\mathbf{x}^l\}_{l=1}^L}) = \sum_{m=1}^M \mathbf{W}_m \left[\sum_{l=1}^L \sum_{k=1}^K A_{mlqk} \cdot \mathbf{W}'_m \mathbf{x}^l(\underbrace{\phi_l(\hat{p}_q)}_{\text{normalized coordinate}} + \Delta \mathbf{p}_{mlqk}) \right]$$

Single-scale features -> multi-scale features

normalized coordinate
coordinate in the l -th scale



Experimental results: Comparison with SoTAs

Method	Epochs	AP	AP ₅₀	AP ₇₅	AP _S	AP _M	AP _L	params	FLOPs	Training GPU hours	Inference FPS
Faster R-CNN + FPN	109	42.0	62.1	45.5	26.6	45.4	53.4	42M	180G	380	26
DETR	500	42.0	62.4	44.2	20.5	45.8	61.1	41M	86G	2000	28
DETR-DC5	500	43.3	63.1	45.9	22.5	47.3	61.1	41M	187G	7000	12
DETR-DC5	50	35.3	55.7	36.8	15.2	37.5	53.6	41M	187G	700	12
DETR-DC5 ⁺	50	36.2	57.0	37.4	16.3	39.2	53.9	41M	187G	700	12
Deformable DETR	50	43.8	62.6	47.7	26.4	47.1	58.0	40M	173G	325	19
+ iterative bounding box refinement	50	45.4	64.7	49.0	26.8	48.3	61.7	40M	173G	325	19
++ two-stage Deformable DETR	50	46.2	65.2	50.0	28.8	49.2	61.7	40M	173G	340	19

- Iterative BB refinement: each decoder layer refines the bounding boxes based on predictions from the previous layer
- Two-stage Deformable DETR
 - Stage 1: Generate proposals
 - Stage 2: Initiate object queries from the proposals



Experimental results: Learning curves

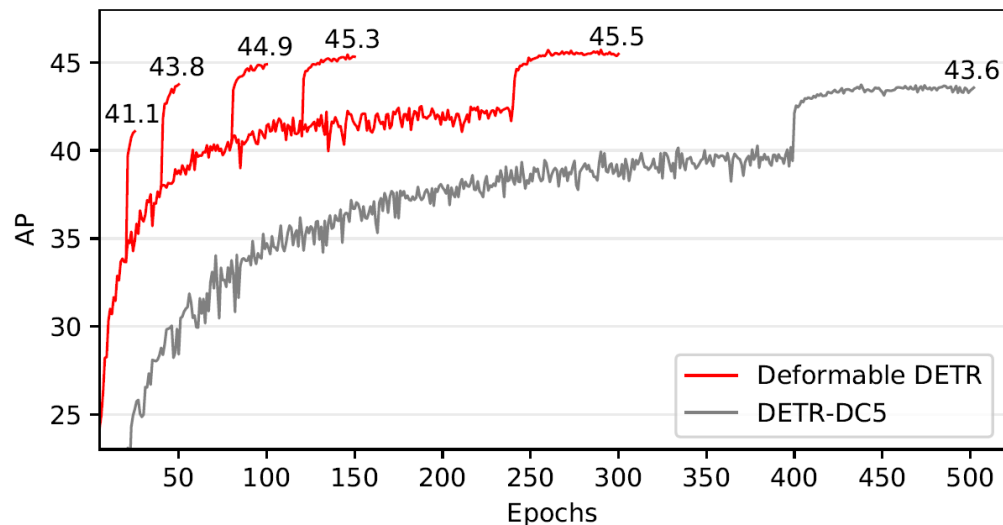


Figure 3: Convergence curves of Deformable DETR and DETR-DC5 on COCO 2017 val set. For Deformable DETR, we explore different training schedules by varying the epochs at which the learning rate is reduced (where the AP score leaps).

Concluding remarks

- Deformable DETR is an end-to-end object detector, which is efficient and fast-converging
- Compared with DETR, Deformable DETR can achieve better performance (especially on small objects) with 10× fewer training epochs
- It enables us to explore more interesting and practical variants of end-to-end object detectors



References

- **RCNN**
 - R. Girshick, J. Donahue, T. Darrell, and J. Malik: Rich feature hierarchies for accurate object detection and semantic segmentation, in CVPR 2014.
- **Fast RCNN**
 - R. Girshick: Fast R-CNN, in ICCV, 2015.
- **Faster RCNN**
 - S. Ren, K. He, R. Girshick, and J. Sun: Faster R-CNN: Towards real-time object detection with region proposal networks, in NIPS 2015.
- **YOLO**
 - J. Redmon, S. Divvala, R. Girshick, and A. Farhadi: You Only Look Once: Unified, real-time object detection, in CVPR 2016.
- **FCOS**
 - Z. Tian et al.: FCOS: Fully convolutional one-stage object detection, in ICCV 2019.
- **DETR**
 - N. Carion et al.: End-to-end object detection with Transformers, in ECCV 2020.
- **Deformable DETR**
 - X. Zhu et al.: Deformable DETR: Deformable transformers for end-to-end object detection, in ICLR 2021.



Thank You for Your Attention!

THANK YOU FOR YOUR ATTENTION!

Yen-Yu Lin (林彥宇)

Email: lin@cs.nycu.edu.tw

URL: <https://www.cs.nycu.edu.tw/members/detail/lin>

